Progressive Sieving-Style Information-Set Decoding Algorithm

Tong Yu[1], Haodong Jiang[1(✉)], Hong Wang[1], Rongmao Chen[2], Qingfeng Cheng[1], Xinyi Huang[3] and Yuefei Zhu[1]

[1] Information Engineering University, China
[2] National University of Defense Technology, China
[3] Nanjing University of Aeronautics and Astronautics, China
tongyu912@163.com, hdjiang13@163.com, wfallmoon@163.com, chromao@nudt.edu.cn, qingfengc2008@sina.com, xyhuang81@gmail.com, yfzhu17@sina.com

Abstract. Information set decoding (ISD) algorithm is the main tool to estimate the concrete bit security of code-based cryptographic schemes including Classic McEliece, HQC and BIKE. Inspired by sieving methods in lattice-based cryptoanalysis, a new type of ISD algorithm (called sieving-ISD) based on locality sensitive filter (LSF) was recently proposed by Guo, Johansson, and Nguyen [GJN24, TIT], which has been shown to achieve comparable complexity with the BJMM/MMT algorithm when attacking Classic McEliece. At EUROCRYPT 2024, Ducas, Esser, Etinski and Kirshanova extended [GJN24, TIT]'s deterministic LSF to probabilistic LSFs and provided an asymptotic worst-case complexity analysis for sieving-ISD with different LSFs in the full-distance setting, which indicates that the sieving-ISD with probabilistic LSFs can achieve better time complexity than the ones with [GJN24, TIT]'s deterministic LSF.

In this paper, we first propose a generalized sieving-ISD framework (called progressive sieving-ISD), which allows for more freedom in parameter configuration. In particular, we present a concrete complexity analysis for both our progressive sieving-ISD and its "decoding one out of many" (DOOM) variant under a binary sieve heuristic, whose validity can be verified via experiments. Then, by searching the optimal parameter configuration, we show that our progressive sieving-ISD can achieve attack time complexity improvements over the previous non-progressive version by 5-12 bits. In particular, for all the three categories of HQC to be standardized by NIST, we show that the state-of-the-art complexity results can be reduced by 7-9 bits using our progressive sieving-ISD, making their security levels 5.1/2.1/5.7 bits below the NIST requirements (143/207/272 bits). Interestingly, our results show that when considering the concrete security of Classic McEliece/HQC/BIKE, the progressive sieving-ISD with [GJN24, TIT]'s deterministic LSF can achieve a better performance than the ones with probabilistic LSFs in [DEEK24, EC]. Finally, we show the connection between progressive sieving-ISD and BJMM, and hence explain why progressive sieving-ISD can achieve a better time complexity than BJMM.

1 Introduction

As a well-established problem in coding theory, the Syndrome Decoding (SD) problem is described as follows. Given a uniformly random parity-check matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$ and a syndrome $\mathbf{s} \in \mathbb{F}_2^{n-k}$, one is asked to find $\mathbf{e} \in \mathbb{F}_2^n$ with weight ω such that $\mathbf{He} = \mathbf{s}$. The difficulty of the Syndrome Decoding (SD) problem is the security basis of the code-based key encapsulation mechanisms (KEMs) such as HQC (selected to be standardized by NIST), Classic McEliece (being considered as an ISO standard), and BIKE. Currently, information set decoding (ISD) algorithm is the main tool to estimate the concrete security of code-based cryptographic schemes.

The first ISD algorithm was proposed by Prange [23] in 1962. The key idea of the ISD algorithm is that some random permutation matrix $\mathbf{P}$ is applied to the parity-check matrix $\mathbf{H}$ such that most of the weight ω is shifted to the *information set*. Let $\mathbf{P}^{-1}\mathbf{e} = (\hat{\mathbf{e}}, \bar{\mathbf{e}}) \in \mathbb{F}_2^{k+l} \times \mathbb{F}_2^{n-k-l}$. Then, applying a Gaussian elimination $\mathbf{G} \in \mathbb{F}_2^{(n-k)\times(n-k)}$ yields

$$\mathbf{GHP} \cdot \mathbf{P}^{-1}\mathbf{e} = \begin{bmatrix} \hat{\mathbf{H}} & 0 \\ \bar{\mathbf{H}} & \mathbf{I}_{n-k-l} \end{bmatrix} (\hat{\mathbf{e}}, \bar{\mathbf{e}}) = \mathbf{Gs} = (\hat{\mathbf{s}}, \bar{\mathbf{s}}).$$

Prange's algorithm assumes $l = 0$ and $\hat{\mathbf{e}} = \mathbf{0}$. Thus, the time complexity of Prange's algorithm (also called the permutation-dominated ISD algorithm) is mainly dominated by finding a proper permutation $\mathbf{P}$. The memory complexity of Prange's algorithm is polynomial. Modern enumeration-dominated ISD algorithms such as Stern [25], MMT [20], BJMM [3], May-Ozerov [21], Both-May [6] etc., assume the weight of $\hat{\mathbf{e}}$ is $\hat{\omega}$ (i.e., $|\hat{\mathbf{e}}| = \hat{\omega} \neq 0$) and enumerate $\hat{\mathbf{e}}$. Various techniques including meet in the middle [25], representation [20,3], nearest neighbors search [21,6], etc., have been proposed to improve the efficiency of such an enumeration part at the cost of exponential memory usage.

Sieving-ISD. Inspired by sieving methods in lattice-based cryptoanalysis, a new type of enumeration-dominated ISD algorithm called Sieving-style ISD (Sieving-ISD) algorithm was recently formally proposed by Guo, Johansson and Nguyen [19],[1] and extended with a more concise framework by Ducas, Esser, Etinski and Kirshanova [11].

The key idea of Sieving-ISD is to initialize a list $L_0 \subseteq \mathcal{S}_{\omega_0}^{k+l}$ and iteratively update the list to increase the probability of the vectors in the list satisfying the parity-check equations $\hat{\mathbf{H}}\mathbf{x} = \hat{\mathbf{s}}$ with $|\mathbf{x}| = \hat{\omega}$. In details, as shown by Fig. 1.1, in the layer-i iteration, two lists L_i^0 and L_i^1 are constructed such that $L_i^0, L_i^1 \in \mathcal{S}_{\omega_i}^{k+l}$, $|L_i^0| = |L_i^1| = N_i/2$, $\hat{\mathbf{H}}_{[1\to st_i]}\mathbf{x} = \mathbf{0}$ ($\hat{\mathbf{H}}_{[1\to st_i]}\mathbf{x} = \hat{\mathbf{s}}_{[1\to st_i]}$, resp.) for any $\mathbf{x} \in L_i^0$ ($\mathbf{x} \in L_i^1$, resp.), where $\mathcal{S}_{\omega_i}^{k+l} \subset \mathbb{F}_2^{k+l}$ is the set of binary vectors of weight ω_i, $\hat{\mathbf{H}}_{[1\to st_i]}$ is the first st_i rows of $\hat{\mathbf{H}}$, and $\hat{\mathbf{s}}_{[1\to st_i]}$ is the first st_i elements of $\hat{\mathbf{s}}$. In particular, in order to make the lists have enough vectors in every iteration, new vectors are produced by combining pairs of neighbor vectors and the locality

[1] A similar idea was also initiated by Bernstein in a cryptanalysis forum cryptanalytic-algorithms@list.cr.yp.to.

sensitive filter (LSF) technique is used to search for the near neighbors [19,11], as in the lattice sieving case [2].

Definition 1.1 (Near Neighbor Search [11, Definition 1.1]). *Given a list of vectors $L \subset \mathcal{S}_\omega^n$, find all pairs $\mathbf{x}, \mathbf{y} \in L$ such that $|\mathbf{x}+\mathbf{y}| = \omega$.*

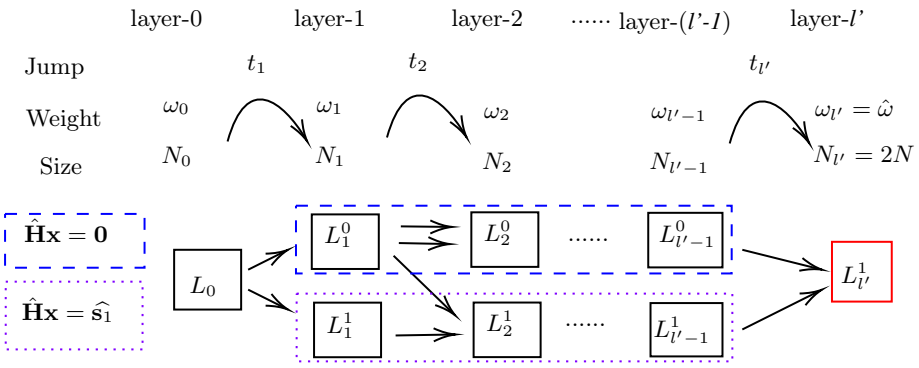

Fig. 1.1: The Framework of Progressive Sieving-ISD

The idea of LSF is first to assign each $\mathbf{x} \in L$ into a *bucket*: $\text{Bucket}_{\mathbf{c},\alpha} = \{\mathbf{x} \in L : |\mathbf{x} \wedge \mathbf{c}| = \alpha\}$, where $\mathbf{c} \in \mathcal{C}_f$ ($\mathcal{C}_f$ is the filter set), and then to perform the near neighbor search only in the buckets. Well-chosen $\mathcal{C}_f$ will guarantee that every pair of vectors in the same bucket is the solution to the near neighbor search problem with a very high probability. Currently, there are mainly two types of LSFs, used for the near neighbor search in the Hamming metric.[2] The first type is deterministic LSFs proposed by Guo, Johansson, and Nguyen in [19], which we call GJN-LSF in this paper. In GJN-LSF, $\mathcal{C}_f$ is chosen as $\mathcal{C}_f = \mathcal{S}_{\omega/2}^{k+l}$ and $\alpha = \omega/2$. Thus, every pair $\mathbf{x}, \mathbf{y} \in L$ such that $|\mathbf{x}+\mathbf{y}| = \omega$ will exactly only lie in the $\text{Bucket}_{\mathbf{x}\wedge\mathbf{y},\alpha}$. The second type is probabilistic LSFs given by Ducas, Esser, Etinski, and Kirshanova [11], where $\mathcal{C}_f$ is chosen as a subset of $\mathcal{S}_v^{k+l}$ ($v < \omega/2$). In this case every pair $\mathbf{x}, \mathbf{y} \in L$ such that $|\mathbf{x}+\mathbf{y}| = \omega$ will lie in $\binom{\omega/2}{v}$ buckets. In order to improve performance, $\mathcal{C}_f$ is set as a random subset in $\mathcal{S}_v^{k+l}$ such that every pair $\mathbf{x}, \mathbf{y} \in L$ with $|\mathbf{x}+\mathbf{y}| = \omega$ will exactly lie in one bucket with a high probability. This random subset is picked via two methods, coded hashing (CH) and random product code (RPC). We denote these two LSFs by CH-LSF and RPC-LSF. Additionally, both have a memory-optimized version via a simple repetition technique (details can be found in Appendix D). We call these two variants CH-LSF-OPT and RPC-LSF-OPT.

The asymptotic complexities of sieving-ISD with the aforementioned five LSFs have been analyzed in the full distance setting by [11]. In particular,

[2] There are many variants of the near neighbor search problem that have been widely researched, e.g., [17,14,21], which we do not discuss in this paper.

all the sieving-ISDs with probabilistic LSFs including CH-LSF, CH-LSF-OPT, RPC, RPC-LSF-OPT have better asymptotic worst-case time complexities than the sieving-ISD with deterministic GJN-LSF and the conventional ISDs such as BJMM and MMT. Different from the worst-case full distance setting $\omega = \Theta(n)$, the hardness of NIST-KEMs including Classic McEliece, BIKE and HQC relies on the SD problem with smaller weight $\omega = \Theta(n/\log(n))$ or $\Theta(\sqrt{n})$. In [19], the authors show that the sieving-ISD with GJN-LSF achieves comparable attack complexity to MMT/BJMM when applied to Classical McEliece, see Table 1.1. For the quasi-cyclic setting like HQC and BIKE, the cyclic structure will make the underlying SD problem produce many related SD problems, which usually leads to a "decoding one out of many" (DOOM) speedup in the attack. In the DOOM setting, the ISD algorithms including Stern, MMT and BJMM, can achieve a speedup of $\sqrt{k}$ [24,18]. For the sieving-ISD, the authors in [19] only sketched the idea of how to adapt the sieving-ISD algorithm to benefit from the DOOM technique. However, due to the lack of concrete complexity and success probability analysis, they did not give the bit security results for BIKE and HQC in the DOOM setting.

Thus, two natural questions appear

Q1: Whether the probabilistic LSFs can help the sieving-style ISD achieve a better attack performance when considering the concrete bit security of Classic McEliece, BIKE and HQC?

Q2: How to strictly analyze the DOOM speedup of the sieving-style ISD and give bit security results for BIKE and HQC in the DOOM setting?

In the current sieving-ISDs [19,11], the number of the parity-check equations $\hat{\mathbf{H}}_{[1\to st_i]}\mathbf{x} = \hat{\mathbf{s}}_{[1\to st_i]}$ is only added 1 in every iteration. That is, the jump value $t_i = st_i - st_{i-1} = 1$ for any $1 \le i \le l'$. In addition, in every iteration, the lists have the same size and the elements have the same weight. That is, for any $0 \le i \le l'$, all ω_i in Fig. 1.1 are set to the same value, and likewise N_i. Thus, the third natural question appears

Q3: How to evaluate the concrete complexity of the generalized sieving-ISD with different t_i, ω_i and N_i? What are the optimal parameter configuration and optimal attack complexity for Classic McEliece, BIKE and HQC?

1.1 Our Contributions

Our main contributions are as follows.

1. First, we extend the Near Neighbor Search problem (Definition 1.1) to the *asymmetric* near neighbor search with *predicate* (ANNS-predicate) problem, where "*asymmetric*" means that the size and element weight of the input list can be different from the ones of the output list and the "*predicate*" means that the check of parity-check equations is built-in into the sieving (this trick is indispensable for our improvement since only the elements in buckets that

Table 1.1: Bit security estimates for Classic McEliece

	Category 1 ($n = 3488$)		Category 3 ($n = 4068$)		Category 5 ($n = 6688$)		Category 5 ($n = 6960$)		Category 5 ($n = 8192$)	
	T	M	T	M	T	M	T	M	T	M
Stern [25]	152.9	41	194.7	43	270.7	54	271.4	64	306.7	83
MMT [20]	152.8	45	194.4	62	267.7	95	267.8	110	301.3	127
BJMM [3]	148.5	77	188.9	101	258.3	109	258.5	110	292.1	114
GJN [19]	150.3	53	191.3	60	264.5	86	264.6	92	298.6	95
NO-PRO-GJN	148.7	43	190.1	74	263.6	94	264.1	87	298.1	89
PRO-GJN	**141.9**	**77**	**182.5**	**86**	**254.1**	**119**	**254.0**	**127**	**286.9**	**131**
PRO-CH	149.0	86	188.9	89	263.7	97	263.9	98	301.5	99
PRO-CH-OPT	144.7	86	183.8	90	254.9	96	254.8	98	288.5	100
PRO-RPC	193.7	75	236.0	79	311.9	85	312.7	85	348.7	88
PRO-RPC-OPT	193.7	75	236.0	78	311.9	85	312.7	85	348.7	88

pass the parity-check equations will be checked whether their combinations satisfy the weight requirement.). Based on the LSFs proposed in [19,11], we give our progressive sieving for the ANNS-predicate problem.

2. Then, embedding the ANNS-predicate sieving into the sieving-ISD, we further present a generalized framework, which we call progressive sieving-ISD in this paper. Different from [19,11], we remove the equality constraint in t_i, ω_i and N_i. In addition, compared with [19], we remove several optimization techniques including collision-filtering[3] and vector-inheriting,[4] since they bring very little performance improvement but will make the complexity analysis complicated [11]. In particular, extending the asymptotic complexity analysis in [11], we give the concrete complexity analysis for our progressive sieving-ISD under a binary sieve heuristic,[5] whose validity can be verified via experiments.
3. Thirdly, tweaking [19]'s sketched idea, we employ an unbalanced weight configuration (vectors in L_i^0 have weight $\omega_i + 1$ while vectors in other lists have weight ω_i) to adapt our progressive sieving-ISD algorithm to benefit from the DOOM technique. Thanks to our simplified framework used in the sieving

[3] This technique is proposed to reduce the memory complexity by classifying the list elements according to the collision in some certain positions and only filtering pairs of vectors according to the remaining positions.

[4] The vectors in layer-$(i-1)$ that already satisfy the parity-check equations in layer-i are directly inherited and forwarded to layer-i.

[5] Roughly speaking, the binary sieve heuristic is the assumption that the elements in input list provided at any layer behave like uniformly random and independent vectors from $\mathcal{S}_{\omega_i}^{k+l}$. This heuristic was first proposed by [11], and required by both analyses in [19,11]. As discussed by [11], similar assumptions arose in lattices [2], and other contexts [16,26,22,10].

step and our concise analysis method, we can strictly analyze the concrete complexity of the DOOM variant of our algorithm.

4. Fourthly, by searching the optimal parameter configuration based on the well-established complexity estimation method, we give the attack complexities using our progressive sieving-ISD for Classic McEliece, BIKE and HQC in the RAM model with unit-time memory access. The results are shown by Tables 1.1 and 1.2, where PRO-GJN, PRO-CH, PRO-CH-OPT, PRO-RPC, and PRO-RPC-OPT denote our progressive sieving-ISD with GJN-LSF, CH-LSF, CH-LSF-OPT, RPC-LSF and RPC-LSF-OPT. In particular, we also present the results of the sieving-ISD with GJN-LSF without our progressive trick (denoted by NO-PRO-GJN) based on the complexity estimation method[6] in this paper. As we can see, our PRO-GJN achieves attack time complexity improvements of 5-12 bits over NO-PRO-GJN for all three schemes. In particular, the state-of-the-art complexity results of HQC will be reduced by 7-9 bits[7], making their security levels 5.1/2.1/5.7 bits below the NIST requirements (143/207/272 bits). In addition, the security levels of Classic McEliece (excluding category 5) and BIKE are also slightly below the NIST requirements under our PRO-GJN attack.
5. Finally, we show the connection between our progressive sieving-ISD and BJMM. In particular, the BJMM algorithm can be interpreted with different ANNS-predicate algorithms in our progressive sieving-ISD framework. In BJMM, a collision-searching with the simple quadratic sieve is used to solve the underlying ANNS-predicate problem. While, roughly speaking, the quadratic sieve is replaced by advanced LSF-based sieving in our progressive sieving-ISD, which hence brings complexity improvements over BJMM when attacking NIST-KEMs. The performance of different ANNS-predicate algorithms in BJMM and our sieving-ISD is thoroughly compared in both asymptotic and concrete settings.

1.2 Interpretation of numerical results

According to Tables 1.1 and 1.2, the progressive sieving-ISD with deterministic GJN-LSF significantly outperforms the ones based on the aforementioned probabilistic LSFs, which is inconsistent with the asymptotic worst results[8] for full distance decoding (large error regime $\omega = \Theta(n)$) in [11]. This is due to the fact that when considering the concrete bit security of real-world NIST-KEM parameter regions (sparse error regime $\omega = \Theta(n/\log(n))$ or $\sqrt{n}$), the optimization of v and α (the main source of the asymptotic improvements) is limited and some

[6] In [19], the memory complexity is estimated only with the list size. While in sieving-ISD, one vector in the list will be stored in many buckets, which is neglected in [19] and included in our analyses in this paper.

[7] As noted by Remark 4.2 and Footnote 20, the time complexities of PRO-GJN can be further slightly reduced by about 1 bit via some small tricks, which are ignored to make the presentation concise.

[8] The maximum complexity under all the code rates.

Table 1.2: Bit security estimates for HQC and BIKE

		Category 1 ($n = 35338$)		Category 3 ($n = 71702$)		Category 5 ($n = 115247$)	
		T	M	T	M	T	M
HQC	Stern [25]	146.0	53	213.1	57	274.5	60
	MMT [20]	147.6	39	215.5	42	277.6	44
	BJMM [3]	146.3	57	213.1	62	274.2	65
	GJN*[1][19]	150.8	39	219.1	52	281.3	55
	NO-PRO-GJN	145.4	44	213.4	36	275.6	50
	PRO-GJN	**137.9**	**46**	**204.9**	**49**	**266.3**	**51**
	PRO-CH	152.7	56	219.8	59	282.3	63
	PRO-CH-OPT	152.7	56	219.7	59	282.3	63
	PRO-RPC	197.1	56	268.5	60	333.2	63
	PRO-RPC-OPT	197.1	56	268.5	60	333.2	63
		Category 1 ($n = 24646$)		Category 3 ($n = 49318$)		Category 5 ($n = 81946$)	
		T	M	T	M	T	M
BIKE Key Recovery	Stern [25]	147.6	51	210.4	55	277.5	58
	MMT [20]	149.5	38	213.0	62	280.0	66
	BJMM [3]	147.4	54	210.0	59	276.7	63
	GJN [19]	145.6	37	209.2	50	275.4	53
	NO-PRO-GJN	144.7	42	208.0	45	275.7	47
	PRO-GJN	**139.6**	**42**	**202.3**	**45**	**269.2**	**47**
	PRO-CH	153.9	52	216.7	56	284.4	59
	PRO-CH-OPT	153.9	52	216.7	56	284.4	58
	PRO-RPC	197.6	51	271.8	56	343.0	59
	PRO-RPC-OPT	197.6	50	271.8	55	343.0	59
BIKE Message Recovery	Stern [25]	146.8	51	211.0	55	275.4	58
	MMT [20]	148.5	38	213.6	41	278.0	66
	BJMM [3]	146.7	54	210.6	59	274.7	63
	GJN* [19]	151.5	37	217.0	40	281.0	53
	NO-PRO-GJN	145.4	43	210.5	46	276.1	48
	PRO-GJN	**139.2**	**44**	**203.2**	**47**	**267.5**	**49**
	PRO-CH	153.3	54	217.5	58	294.1	61
	PRO-CH-OPT	153.3	54	217.5	58	282.5	60
	PRO-RPC	195.8	55	264.0	58	331.6	61
	PRO-RPC-OPT	195.8	54	264.0	58	331.6	61

[1] The "*" is used to highlight that these results are derived without the DOOM improvement.

included polynomial overheads that are neglected in the asymptotic analyses are high.

1.3 Future Work

In this paper, we only analyze the concrete applications of our progressive sieving-ISD to code-based KEMs. However, analyzing the asymptotic complexity in the worst-case full distance setting is intractable due to the variables w_i, N_i, t_i. We leave the asymptotic complexity analysis of our algorithms as an open problem. In addition, implementing the progressive sieving-ISD with a large-scale server is also a non-trivial task, especially when we need to take the acceleration of sophisticated instruction sets into consideration. Thus, we leave the optimized implementation of our algorithm as a future work.

Recently, Carrier et al. [7,8] showed that dual attack can achieve a better worst-case asymptotic complexity than the ISD algorithms for the low-rate case. Given the recent remarkable advancements [9] of dual attack to NIST FIPS 203 (Kyber, a lattice-based scheme), we think it is also interesting to analyze the concrete application of dual attack to code-based KEMs, such as HQC.

Acknowledgements. We thank anonymous reviewers from ASIACRYPT 2025 and EUROCRYPT 2026 for their insightful comments and suggestions. In particular, we sincerely thank one of the anonymous ASIACRYPT 2025 reviewers for pointing out the implicit parameter constraint (3.9), and one of the anonymous EUROCRYPT 2026 reviewers for helping refine the gate complexity model and pointing out the attack parameter issues in the previous versions.

2 Preliminaries

Symbol description. Let $\mathbb{F}_2$ be the binary finite field and $\mathbb{F}_2^n$ be the vector space of dimension n. We use capital bold for matrices, small bold for vectors, and small non-bold for scalars. For a vector $\mathbf{y}$, we usually omit the transposition and write $\mathbf{y}$ instead of $\mathbf{y}^T$ to ease the notation, which should be clear from the context unless otherwise mentioned. The operation $+$ in $\mathbf{x}+\mathbf{y}$ ($\mathbf{x},\mathbf{y}\in\mathbb{F}_2^n$) means the bit-by-bit XOR. An n-dimensional identity matrix is denoted by $\mathbf{I}_n$. $\mathbf{H}_{[i]}$ ($\mathbf{s}_{[i]}$, resp.) denotes the first i rows (elements, resp.) of the matrix $\mathbf{H}$ (vector $\mathbf{s}$, resp.). In addition, we use $[\mathbf{H}_1,\mathbf{H_2}]$ ($[\mathbf{s}_1,\mathbf{s}_2]$, resp.) to denote the concatenation of matrices $\mathbf{H}_1$ and $\mathbf{H}_2$ (vectors $\mathbf{s}_1$ and $\mathbf{s}_2$, resp.). The notation log denotes a logarithmic function with base 2. We define $|\mathbf{x}|$ to be the Hamming weight of a vector $\mathbf{x}$. A sphere of radius ω in $\mathbb{F}_2^n$ is defined as $\mathcal{S}_\omega^n$ and its size is $\binom{n}{\omega}$. The term $x\leftarrow Bern(p)$ means that x is sampled according to the Bernoulli distribution, that is $\Pr[x=1]=p$ and $\Pr[x=0]=1-p$.

Lemma 2.1 (Chernoff bound). *Let $X_1,\ldots,X_n$ be n independent and identically distributed random variables, where $X_i\leftarrow Bern(p)$. Let $Z=\sum_{i=1}^n X_i$, $\mu=E[Z]=np$ is the expectation of Z, then the probability bound is $\Pr[Z\leq(1-\delta)\mu]\leq\exp\left(-\frac{\mu\delta^2}{2}\right)$, for all $0<\delta<1$.*

2.1 The Framework of Information Set Decoding

A binary linear $[n, k]$ code $\mathcal{C}$ is a subspace of $\mathbb{F}_2^n$, where n is its length and k is its dimension. There exists a matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$ called parity check matrix such that for each $\mathbf{c} \in \mathcal{C}$, we have $\mathbf{Hc} = \mathbf{0}$. Then the code $\mathcal{C}$ can be represented as $\mathcal{C} := \{\mathbf{c} \in \mathbb{F}_2^n : \mathbf{Hc} = \mathbf{0}\}$. The security basis of many code-based cryptographic primitives is the hardness of the Syndrome Decoding (SD) problem on a random code, which is defined as follows.

Definition 2.1 (Syndrome Decoding (SD) Problem). *Let $\mathcal{C} \subseteq \mathbb{F}_2^n$ be a binary linear $[n, k]$ code defined by a uniformly random parity check matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$. Given a syndrome $\mathbf{s} \in \mathbb{F}_2^{n-k}$, find a vector $\mathbf{e} \in \mathbb{F}_2^n$ with weight ω satisfying $\mathbf{He} = \mathbf{s}$.*

Currently, the method that produces the best asymptotic complexity for solving the SD problem is the Information Set Decoding (ISD) algorithm, which is also the main tool to analyze the concrete security of code-based cryptographic algorithms. The framework of the ISD algorithm consists of the following steps.[9] Firstly, choose a random permutation matrix $\mathbf{P}$ and compute $\mathbf{HP}$. Thus, we expectantly make the weight of the target solution vector $\mathbf{e}$ change from random to a fixed assignment i.e. $\mathbf{e}' = \mathbf{P}^{-1}\mathbf{e} = (\hat{\mathbf{e}}, \bar{\mathbf{e}})$ where $|\hat{\mathbf{e}}| = \hat{\omega}$ and $|\bar{\mathbf{e}}| = \omega - \hat{\omega}$. Secondly, we perform a Gaussian elimination $\mathbf{G} \in \mathbb{F}_2^{(n-k)\times(n-k)}$ and compute $\mathbf{H}' = \mathbf{GHP}$ and $\mathbf{s}' = \mathbf{Gs}$. After this step, we can get

$$\mathbf{H}'\mathbf{e}' = \mathbf{GHP} \cdot \mathbf{P}^{-1}\mathbf{e} = \begin{bmatrix} \hat{\mathbf{H}} & 0 \\ \bar{\mathbf{H}} & \mathbf{I}_{n-k-l} \end{bmatrix} (\hat{\mathbf{e}}, \bar{\mathbf{e}}) = \mathbf{s}' = (\hat{\mathbf{s}}, \bar{\mathbf{s}})$$

where $\hat{\mathbf{H}} \in \mathbb{F}_2^{l\times(k+l)}$ and $\bar{\mathbf{H}} \in \mathbb{F}_2^{(n-k-l)\times(k+l)}$.

Definition 2.2 ((Syndrome Multi-Decoding (SMD) Problem). *Let $\mathcal{C}$ be a linear $[k + l, k]$ code given by a parity check matrix $\hat{\mathbf{H}} \in \mathbb{F}^{l\times(k+l)}$. Given a weight parameter $\hat{\omega} \in \mathbb{N}$ and a syndrome $\hat{\mathbf{s}} \in \mathbb{F}^l$, find a list of vectors $L \subseteq \mathcal{S}_{\hat{\omega}}^{k+l}$ with size $|L| = N$ such that for any $\hat{\mathbf{e}} \in L$ we have $\hat{\mathbf{H}}\hat{\mathbf{e}} = \hat{\mathbf{s}}$.*

Then, we execute an oracle to find a list of vectors $\hat{\mathbf{e}}$ with weight $\hat{\omega}$ and satisfying $\hat{\mathbf{H}}\hat{\mathbf{e}} = \hat{\mathbf{s}}$, which can be formally defined as Syndrome Multi-Decoding (SMD) Problem (see Definition 2.2). This step is a key step in the ISD algorithm, and improvements of the modern ISD algorithms, such as MMT/BJMM, mainly come from this step. In this work, we instantiate this oracle with the progressive sieve. Finally, we compute $\bar{\mathbf{e}} = \bar{\mathbf{H}}\hat{\mathbf{e}} + \bar{\mathbf{s}}$ to check whether $|\bar{\mathbf{e}}| = \omega - \hat{\omega}$ holds. We give a complete description of the ISD algorithm in Algorithm 1.

Theorem 2.1 (Complexity of the ISD algorithm). *Let $\mathcal{C}$ be a binary linear $[n, k]$ code. Given a uniformly random parity check matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$, a syndrome $\mathbf{s} \in \mathbb{F}_2^{n-k}$, the size N of the output list in the SMD problem, parameters ω, $\hat{\omega}$ and l. Let T_O (M_O, resp.) denote the expected time (expected memory, resp.)*

[9] In this paper, we follow Dumer's framework [12].

Algorithm 1: The ISD Algorithm

Input : A binary linear $[n, k]$ code, parity check matrix $\mathbf{H}$, syndrome $\mathbf{s}$, parameters ω, $\hat{\omega}$ and l

Output: $\mathbf{e} \in \mathcal{C}$ with weight ω and satisfying $\mathbf{He} = \mathbf{s}$

1 **repeat**

2 Choose a random permutation matrix $\mathbf{P}$ and perform a Gaussian elimination $\mathbf{G}$;

3 $\mathbf{H}' = \mathbf{GHP} = \begin{bmatrix} \hat{\mathbf{H}} & 0 \\ \bar{\mathbf{H}} & \mathbf{I}_{n-k-l} \end{bmatrix}$,$\mathbf{e}' = \mathbf{P}^{-1}\mathbf{e} = (\hat{\mathbf{e}}, \bar{\mathbf{e}})$,$\mathbf{s}' = \mathbf{Gs} = (\hat{\mathbf{s}}, \bar{\mathbf{s}})$;

4 $L \longleftarrow \text{Oracle}(\hat{\mathbf{H}}, \hat{\mathbf{s}}, \hat{\omega})$;

5 **for** $\hat{\mathbf{e}} \in L$ **do**

6 $\bar{\mathbf{e}} \longleftarrow \bar{\mathbf{H}}\hat{\mathbf{e}} + \bar{\mathbf{s}}$;

7 **if** $|\bar{\mathbf{e}}| = \omega - \hat{\omega}$ **then**

8 **return** $\mathbf{P}^{-1}(\hat{\mathbf{e}}, \bar{\mathbf{e}})$;

9 **until** *solution* $\mathbf{e}$ *is found*;

of the oracle executed in Algorithm 1, T_{Gauss} be the expected time of the Gaussian elimination and T_{check} be the expected time of the last step in the Algorithm 1. Then, the algorithm can solve the SD problem in the expected time and expected memory $T = (P_1 \cdot P_2)^{-1} \cdot (T_{Gauss} + T_O + T_{check})$, $M = M_O + M_{gauss}$, where $P_1 := \binom{k+l}{\hat{\omega}}\binom{n-k-l}{\omega-\hat{\omega}}/\binom{n}{\omega}$, $P_2 = \min\left\{1, N \cdot 2^l/\binom{k+l}{\hat{\omega}}\right\}$.

The proof of Theorem 2.1 is similar to the proof of [11, Theorem 3.1] and we present it in Appendix C.

2.2 Locality Sensitive Filter

In the sieving-ISD algorithms, the oracle (line 4 of Algorithm 1) is instantiated using the sieving that proceeds interactively with a tower of codes $\mathcal{C}_0 \subset \mathcal{C}_1 \subset \cdots \subset \mathcal{C}_l$, see [19,11]. Here, $\mathcal{C}_i$'s parity-check matrix consists of the first i rows of $\hat{\mathbf{H}}$. In each interaction i, a new list of short codewords belonging to code $\mathcal{C}_i$ is constructed by summing elements from the current list with a near neighbor algorithm. In particular, the list size and the weight of codewords in the list remain constant during the interaction. Locality sensitive filter (LSF) is the basis for many of the best known algorithms to find near neighbors [2,11], including the near neighbor search in the sieving-ISD. In Appendix A, we introduce some common concepts used in LSF.

3 Progressive sieving for the SMD problem

3.1 The Framework of Progressive Sieving

Let matrix $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_1, \hat{\mathbf{H}}_2, ..., \hat{\mathbf{H}}_{l'}] \in \mathbb{F}_2^{l \times (k+l)}$ and vector $\hat{\mathbf{s}} = [\hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2, ..., \hat{\mathbf{s}}_{l'}] \in \mathbb{F}_2^{l}$, where $\hat{\mathbf{H}}_i \in \mathbb{F}_2^{t_i \times (k+l)}$ and $\hat{\mathbf{s}}_i \in \mathbb{F}_2^{t_i}$ for any $1 \leq i \leq l'$. Obviously, we have

$\sum_{i=1}^{l'} t_i = l$. Denote $\hat{\mathbf{H}}_{1\to i} = [\hat{\mathbf{H}}_1, \hat{\mathbf{H}}_2, ..., \hat{\mathbf{H}}_i]$ and $\hat{\mathbf{s}}_{1\to i} = [\hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2, ..., \hat{\mathbf{s}}_i]$. In the following, we assume that N_i $(0 \le i \le l')$ is an even number for simplicity.[10] Let $L_0 \subseteq S_{\omega_0}^{k+l}$ be a list with size N_0 such that the elements in L_0 are uniformly and independently distributed in $S_{\omega_0}^{k+l}$. The goal of our progressive sieving is to find two list towers $L_1^0, \ldots, L_{l'-1}^0$ and $L_1^1, \ldots, L_{l'-1}^1$ such that for any $i \in \{1, \ldots, l'-1\}$ we have $L_i^0, L_i^1 \subseteq S_{\omega_i}^{k+l}$, $|L_i^0| = |L_i^1| = N_i/2$, and $\hat{\mathbf{H}}_{1\to i}\mathbf{x} = \mathbf{0}$ ($\hat{\mathbf{H}}_{1\to i}\mathbf{x} = \hat{\mathbf{s}}_{1\to i}$, resp.) for any $\mathbf{x} \in L_i^0$ ($\mathbf{x} \in L_i^1$, resp.). For the layer-l', the final list $L_{l'}^1 \subseteq S_{\omega_{l'}}^{k+l}$ $(\omega_{l'} = \hat{\omega})$ with size N is generated by combining elements in the lists of layer-$(l'-1)$ such that for any $\mathbf{x} \in L_{l'}^1$ we have $\hat{\mathbf{H}}\mathbf{x} = \hat{\mathbf{s}}$.

Algorithm 2: Progressive Sieving for the SMD problem

Input : List $L_0 \subseteq \mathcal{S}_{\omega_0}^{k+l}$ with size N_0, matrix $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_1, \hat{\mathbf{H}}_2, ..., \hat{\mathbf{H}}_{l'}]^T \in \mathbb{F}_2^{l\times(k+l)}$ and vector $\hat{\mathbf{s}} = (\hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2, ..., \hat{\mathbf{s}}_{l'})^T \in \mathbb{F}_2^l$ where $\hat{\mathbf{H}}_i \in \mathbb{F}_2^{t_i\times(k+l)}$, $\hat{\mathbf{s}}_i \in \mathbb{F}_2^{t_i}$ for any $1 \le i \le l'$, and $\sum_{i=1}^{l'} t_i = l$

Output: List $L_{l'}^1 \subseteq \mathcal{S}_{\hat{\omega}}^{k+l}$ with size $|L_{l'}^1| = N$ such that $\hat{\mathbf{H}}\mathbf{z} = \hat{\mathbf{s}}$ for any $\mathbf{z} \in L_{l'}^1$

$L_1^0, L_1^1 \leftarrow$ ANNS-Predicate-1$(L_0, \hat{\mathbf{H}}_1, \hat{\mathbf{s}}_1)$;

for $i = 2$ *to* $l'-1$ **do**

  $L_i^0, L_i^1 \leftarrow$ ANNS-Predicate-2$(L_{i-1}^0, L_{i-1}^1, [\hat{\mathbf{H}}_{i-1}, \hat{\mathbf{H}}_i], [\hat{\mathbf{s}}_{i-1}, \hat{\mathbf{s}}_i])$;

$L_{l'}^1 \leftarrow$ ANNS-Predicate-3$(L_{l'-1}^0, L_{l'-1}^1, [\hat{\mathbf{H}}_{l'-1}, \hat{\mathbf{H}}_{l'}], [\hat{\mathbf{s}}_{l'-1}, \hat{\mathbf{s}}_{l'}])$;

return $L_{l'}^1$

The idea of the sieving is iteratively maintaining two lists of vectors with moderate size and combining pairs of vectors in lists to produce enough vectors with some desired property (i.e., some specific parity-check equations are satisfied) in each layer. The framework of our progressive sieving is shown by Fig. 1.1, which is an extension of [19,11]. The initial list L_0 is obtained by uniformly sampling elements from $\mathcal{S}_{\omega_0}^{k+l}$ at random.

In the layer-1, the elements in both L_1^0 and L_1^1 are obtained by combining pairs of elements in L_0. That is,

$$L_1^0 = \{\mathbf{x}+\mathbf{y} : \mathbf{x}, \mathbf{y} \in L_0, \hat{\mathbf{H}}_1(\mathbf{x}+\mathbf{y}) = \mathbf{0} \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_1\},$$

$$L_1^1 = \{\mathbf{x}+\mathbf{y} : \mathbf{x}, \mathbf{y} \in L_0, \hat{\mathbf{H}}_1(\mathbf{x}+\mathbf{y}) = \hat{\mathbf{s}}_1 \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_1\}.$$

In the layer-i $(2 \le i \le l'-1)$, the elements in L_i^0 are obtained by summing pairs of vectors in list L_{i-1}^0 or L_{i-1}^1, and the elements in L_i^1 are obtained by combining the elements in list L_{i-1}^0 and the elements in list L_{i-1}^1. That is, for $2 \le i \le l'-1$, we have

$$L_i^0 = \{\mathbf{x}+\mathbf{y} : \text{both } \mathbf{x}, \mathbf{y} \in L_{i-1}^0 \text{ or } L_{i-1}^1, \hat{\mathbf{H}}_{1\to i}(\mathbf{x}+\mathbf{y}) = \mathbf{0} \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_i\},$$

[10] If N_i is odd, the term $N_i/2$ in the context should be $\lfloor N_i/2 \rfloor$.

$$L_i^1 = \{\mathbf{x}+\mathbf{y} : \mathbf{x} \in L_{i-1}^0, \mathbf{y} \in L_{i-1}^1, \hat{\mathbf{H}}_{1\to i}(\mathbf{x}+\mathbf{y}) = \hat{\mathbf{s}}_{1\to i} \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_i\}.$$

In the layer-l', the list $L_{l'}^1$ is obtained by summing the elements in list $L_{l'-1}^0$ and the elements in list $L_{l'-1}^1$. That is,

$$L_{l'}^1 = \{\mathbf{x}+\mathbf{y} : \mathbf{x} \in L_{l'-1}^0, \mathbf{y} \in L_{l'-1}^1, \hat{\mathbf{H}}_{1\to l'}(\mathbf{x}+\mathbf{y}) = \hat{\mathbf{s}}_{1\to l'} \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_{l'}\}.$$

The whole progressive sieving is given by Algorithm 2, where the embedded ANNS-Predicate algorithms are given by Algorithms 3. These ANNS-Predicate algorithms will be discussed in detail in Sec. 3.2. The complexity of Algorithm 2 is given by Theorem 3.1, which can trivially be obtained via the following Binary Sieve Heuristic, initially proposed in [11].

Binary Sieve Heuristic [11] .[11] The expected time complexity, memory complexity and success probability of the ANNS-Predicate algorithm in Line 1, Line 3 and Line 4 of Algorithm 2 are not affected when L_{i-1}^0 (L_{i-1}^1, resp.) is sampled uniformly and independently from $\{\mathbf{x} \in \mathcal{S}_{\omega_{i-1}}^{k+l} : \hat{\mathbf{H}}_{1\to(i-1)}(\mathbf{x}+\mathbf{y}) = \mathbf{0}\}$ ($\{\mathbf{x} \in \mathcal{S}_{\omega_{i-1}}^{k+l} : \hat{\mathbf{H}}_{1\to(i-1)}(\mathbf{x}+\mathbf{y}) = \hat{\mathbf{s}}_{1\to i}\}$, resp.) for any $1 \le i \le l'$.

Theorem 3.1 (Complexity of Progressive Sieving). *Let matrix* $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_1, ..., \hat{\mathbf{H}}_{l'}] \in \mathbb{F}_2^{l\times(k+l)}$, *vector* $\hat{\mathbf{s}} = (\hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2, ..., \hat{\mathbf{s}}_{l'}) \in \mathbb{F}_2^l$, *where* $\hat{\mathbf{H}}_i \in \mathbb{F}_2^{t_i\times(k+l)}$ *for each* $1 \le i \le l'$ *and* $\sum_{i=1}^{l'} t_i = l$. *For any* $1 \le i \le l'$, *denote* T_i, M_i *and* p_i *as the expected time, expected memory and success probability of the ANNS-Predicate algorithm in layer-i. Then, under Binary Sieve Heuristic, Algorithm 2 can solve the SMD problem in expected time* $T = \sum_{i=1}^{l'} T_i$ *and expected memory* $M = \max_{1\le i\le l'}\{M_i\}$ *with the probability* $\prod_{i=1}^{l'} p_i$.

3.2 Asymmetric near neighbor search with predicate

Now, we show the algorithm for asymmetric near neighbor search with predicate (ANNS-predicate) problem (see Definitions 3.1) in single-layer iteration and its complexity.

Definition 3.1 (ANNS-Predicate-Problem). *Given matrix* $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_{t'}, \hat{\mathbf{H}}_t] \in \mathbb{F}_2^{(t'+t)\times(k+l)}$, *vector* $\hat{\mathbf{s}} = [\hat{\mathbf{s}}_{t'}, \hat{\mathbf{s}}_t] \in \mathbb{F}_2^{t'+t}$, *and two lists* $L_0', L_1' \subseteq \mathcal{S}_{\omega'}^{k+l}$ *of uniformly and independently distributed vectors of weight* ω' *such that* $\hat{\mathbf{H}}_{t'}\mathbf{x} = \mathbf{0}$ *holds for each* $\mathbf{x} \in L_0'$ *and* $\hat{\mathbf{H}}_{t'}\mathbf{x} = \hat{\mathbf{s}}_{t'}$ *holds for each* $\mathbf{x} \in L_1'$, *find two lists* $L_0, L_1 \subseteq \mathcal{S}_{\omega}^{k+l}$ *such that for any* $\mathbf{z} \in L_0$ *we have* $\hat{\mathbf{H}}\mathbf{z} = \mathbf{0}$, *and for any* $\mathbf{z} \in L_1$ *we have* $\hat{\mathbf{H}}\mathbf{z} = \hat{\mathbf{s}}$.

Algorithm 3 can be used to solve the following three specific ANNS-Predicate problems:

[11] Binary Sieve Heuristic is only verified via experiments for the symmetric case in [11]. In Appendix B, by extending [11]'s experiments, we show that such a heuristic also holds for the asymmetric case, on which the progressive sieving relies.

Algorithm 3: ANNS-Predicate

Input : matrix $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_{t'}, \hat{\mathbf{H}}_t] \in \mathbb{F}_2^{(t'+t)\times(k+l)}$, vector $\hat{\mathbf{s}} = [\hat{\mathbf{s}}_{t'}, \hat{\mathbf{s}}_t] \in \mathbb{F}_2^{t'+t}$, lists $L'_0, L'_1 \subseteq \mathcal{S}_{\omega'}^{k+l}$ satisfying $\hat{\mathbf{H}}_{t'}\mathbf{x} = \mathbf{0}$ for any $\mathbf{x} \in L'_0$, $\hat{\mathbf{H}}_{t'}\mathbf{x} = \hat{\mathbf{s}}_{t'}$ for any $\mathbf{x} \in L'_1$, the filter set $\mathcal{C}_f$ and the bucket parameter α

Output: lists $L_0, L_1 \subseteq \mathcal{S}_{\omega}^{k+l}$ such that $\hat{\mathbf{H}}\mathbf{z} = \mathbf{0}$ for any $\mathbf{z} \in L_0$ and $\hat{\mathbf{H}}\mathbf{z} = \hat{\mathbf{s}}$ for any $\mathbf{z} \in L_1$

// ANNS-Predicate-1: $t' = 0$, $\hat{\mathbf{H}}_{t'} = \mathbf{0}$, $L'_1 = \emptyset$

// ANNS-Predicate-2: $t' \neq 0$, $|L'_0| = |L'_1| = N'/2$, $|L_0| = |L_1| = N/2$

// ANNS-Predicate-3: $t' \neq 0$, $L_0 = \emptyset$

1 Bucketing Phase

2 **for** $\mathbf{x} \in L'_0$ **do**

3 **for** $\mathbf{c} \in$ ValidFilters$(\mathcal{C}_f, \mathbf{x}, \alpha)$ **do**

4 add $(0, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$ to Bucket$_\alpha(\mathbf{c})$;

5 **for** $\mathbf{x} \in L'_1$ **do**

6 **for** $\mathbf{c} \in$ ValidFilters$(\mathcal{C}_f, \mathbf{x}, \alpha)$ **do**

7 add $(1, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$ to Bucket$_\alpha(\mathbf{c})$;

8 Checking Phase

9 $L_0 \leftarrow \emptyset$,$L_1 \leftarrow \emptyset$;

10 **for** $\mathbf{x} \in L'_0$ **do**

11 **for** $\mathbf{c} \in$ ValidFilters$(\mathcal{C}_f, \mathbf{x}, \alpha)$ **do**

12 **for** $(0, \mathbf{y}, \hat{\mathbf{H}}_t\mathbf{x}) \in Bucket_\alpha(\mathbf{c})$ **do**

13 **if** $|\mathbf{x} + \mathbf{y}| = \omega$ **then**

14 store $\mathbf{x} + \mathbf{y}$ in L_0

15 **for** $(j, \mathbf{y}, \hat{\mathbf{s}}_t + \hat{\mathbf{H}}_t\mathbf{x}) \in Bucket_\alpha(\mathbf{c})$// $j = 0$ for ANNS-Predicate-Problem-1, and $j = 1$ for ANNS-Predicate-Problem-2/3

16 **do**

17 **if** $|\mathbf{x} + \mathbf{y}| = \omega$ **then**

18 store $\mathbf{x} + \mathbf{y}$ in L_1

19 Discard some elements if $|L_1| > N/2$;

20 **for** $\mathbf{x} \in L'_1$ **do**

21 **for** $\mathbf{c} \in$ ValidFilters$(\mathcal{C}_f, \mathbf{x}, \alpha)$ **do**

22 **for** $(1, \mathbf{y}, \hat{\mathbf{H}}_t\mathbf{x}) \in Bucket_\alpha(\mathbf{c})$ **do**

23 **if** $|\mathbf{x} + \mathbf{y}| = \omega$ **then**

24 store $\mathbf{x} + \mathbf{y}$ in L_0

25 Discard some elements if $|L_0| > N/2$;

// Lines 2-4, 12-14 are removed for ANNS-Predicate-Problem-3

// Lines 20-25 are removed for ANNS-Predicate-Problem-1/3

26 **return** L_0, L_1

– **ANNS-Predicate-1:** $t' = 0$, $\hat{\mathbf{H}}_{t'} = \mathbf{0}$, $L'_1 = \emptyset$
– **ANNS-Predicate-2:** $t' \neq 0$, $|L'_0| = |L'_1|$, $|L_0| = |L_1|$
– **ANNS-Predicate-3:** $t' \neq 0$, $L_0 = \emptyset$

The main difference between ANNS-Predicate-1 and ANNS-Predicate-2 is the source lists from which the two vectors are combined to produce the new vector. ANNS-Predicate-3 is a simplified version of ANNS-Predicate-2 by removing the generation of the list L_0. That is, these three algorithms can be analyzed in the same way. Note that ANNS-Predicate-2 is more complicated than the other two cases. In the following, we choose to mainly analyze Algorithm 3 for the ANNS-Predicate-2 case, and directly give analysis results for ANNS-Predicate-1/3 accordingly.

Algorithm 3 consists of two phases: the bucketing phase and the checking phase. In the bucketing phase, we first find all valid filters by the **ValidFilters** function for each $\mathbf{x}$ in L'_b ($b \in \{0,1\}$) and add $(b, \mathbf{x}, \hat{\mathbf{H}}\mathbf{x})$ to the bucket. In the checking phase, for each $\mathbf{x} \in L'_0$ and each $\mathbf{c} \in \texttt{ValidFilters}(\mathcal{C}_f, \mathbf{x}, \alpha)$, we successively check whether $|\mathbf{x}+\mathbf{y}| = \omega$ holds for $\mathbf{y}$ in $(0, \mathbf{y}, \hat{\mathbf{H}}\mathbf{x})$ (and $(1, \mathbf{y}, \hat{\mathbf{s}} + \hat{\mathbf{H}}\mathbf{x})$, resp.) in $\text{Bucket}_\alpha(\mathbf{c})$. If it holds, we add $\mathbf{x}+\mathbf{y}$ to L_0 (and L_1, resp.). In addition, for each $\mathbf{x} \in L'_1$ and each $\mathbf{c} \in \texttt{ValidFilters}(\mathcal{C}_f, \mathbf{x}, \alpha)$, we also store $\mathbf{x}+\mathbf{y}$ in L_0 for any $(1, \mathbf{y}, \hat{\mathbf{H}}\mathbf{x})$ in $\text{Bucket}_\alpha(\mathbf{c})$.

Let d be the overlap of a near neighbor pair $(\mathbf{x}, \mathbf{y})$. Since $|\mathbf{x}| = |\mathbf{y}| = \omega'$ and $|\mathbf{x}+\mathbf{y}| = \omega$, we have $2\omega' - 2d = \omega$. Thus, $d = (2\omega' - \omega)/2$. For any $\mathbf{x}, \mathbf{y} \in S^{k+l}_{\omega'}(\mathbf{x} \neq \mathbf{y})$, the probability that $\mathbf{x}+\mathbf{y} \in S^{k+l}_{\omega}$, i.e. $(\mathbf{x}, \mathbf{y})$ overlaps exactly at d positions is $\binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'}$. Due to the restrictions of the parity-check equations, i.e. $\hat{\mathbf{H}}_t\mathbf{x} = \mathbf{0}$ (and $\hat{\mathbf{H}}_t\mathbf{x} = \hat{\mathbf{s}}_t$ resp.), only $1/2^t$-parts of the vectors such that $\mathbf{x}+\mathbf{y} \in S^{k+l}_{\omega}$ are reserved on expectation. For any $\mathbf{x}, \mathbf{y} \in S^{k+l}_{\omega'}$ such that $|\mathbf{x}+\mathbf{y}| = \omega$, we denote $q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$ as the probability that $\mathbf{x}$ and $\mathbf{y}$ are added to the same bucket. Thus, we have

$$q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha) = \Pr[\exists \mathbf{c} \in \mathcal{C}_f : \mathbf{c} \in \mathcal{B}_\alpha(\mathbf{x}) \cap \mathcal{B}_\alpha(\mathbf{y})]. \tag{3.1}$$

We now give Theorem 3.2 to evaluate the complexity of Algorithm 3.

Theorem 3.2 (Complexity of Algorithm 3). [12] *Given a uniformly random matrix* $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_{t'}, \hat{\mathbf{H}}_t] \in \mathbb{F}_2^{(t'+t)\times(k+l)}$, *vector* $\hat{\mathbf{s}} = [\hat{\mathbf{s}}_{t'}, \hat{\mathbf{s}}_t] \in \mathbb{F}_2^{t'+t}$, *and two lists* $L'_0, L'_1 \subseteq \mathcal{S}^{k+l}_{\omega'}$ *containing uniform and independent vectors of weight* ω' *such that* $\hat{\mathbf{H}}_{t'}\mathbf{x} = \mathbf{0}$ *holds for each* $\mathbf{x} \in L'_0$ *and* $\hat{\mathbf{H}}_{t'}\mathbf{x} = \hat{\mathbf{s}}_{t'}$ *holds for each* $\mathbf{x} \in L'_1$. *Let* $T_{ValidFilters}$ *denote the time to identify the valid filters for each* $\mathbf{x} \in L'_0$ *(*$\mathbf{x} \in L'_1$, *resp.). Then Algorithm 3 can solve ANNS-Predicate-Problem (Definition 3.1) in*

[12] For conciseness, we overlook the impact of duplicated vectors in the analyses of success probability in Theorems 3.2. But, in the concrete bit security analysis of NIST code-based cryptographic algorithms, we take this into consideration.

expected time and expected memory

$$T = N'\left(C_1 \cdot T_{ValidFilters} + \frac{C_2 \cdot E[|\mathcal{B}_\alpha(\mathbf{x})|] \cdot E[|Bucket_\alpha(\mathbf{c})|] \cdot \omega' \log(k+l)}{2^t}\right) + T_{table} + N'(\omega'-1)t + N' \cdot C_1 \cdot E[|\mathcal{B}_\alpha(\mathbf{x})|] \cdot (\omega' \log(k+l) + t + \bar{b}) \quad (3.2)$$

$$M = N'((\omega' \log(k+l) + t + \bar{b}) + C_3 E[|\mathcal{B}_\alpha(\mathbf{x})|] \cdot (\omega' \log(k+l) + t + \bar{b})) + C_4 N \cdot \omega \log(k+l) + 2^{t+1} \cdot t \quad (3.3)$$

with probability

$$p \geq (1 - \exp(-\bar{N} q \delta^2/2))^2 \quad (3.4)$$

and $T_{table} = \min\{C_5 N', 2^t\} \cdot t$*, where* $\bar{N} \approx C_6 N'^2$*,* $\delta = 1 - N/(2\bar{N}q)$ *and* $q = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'} \cdot q(\mathbf{x}, \mathbf{y}, C_f, \alpha)/2^t$*. In particular, the values* $(C_1, \bar{b}, C_2, C_3, C_4, C_5, C_6)$ *are set to be* $(1, 0, 2, 1, 1, 1, 1/2)$ *for ANNS-Predicate-1,* $(1, 1, 3/4, 1, 1, 1/2, 1/4)$ *for ANNS-Predicate-2 and* $(1/2, 1, 1/4, 1/2, 1/2, 1/2, 1/4)$ *for ANNS-Predicate-3.*

Proof. In the following, we mainly analyze Algorithm 3 for the ANNS-Predicate-2 case. The results for ANNS-Predicate-1/3 can be obtained in the same way. For any pair of vectors $\mathbf{x}, \mathbf{y} \in L'_0$, $|\mathbf{x} + \mathbf{y}| = \omega$ is satisfied with probability $\delta(k+l, \omega, \omega') = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'}$ and $\hat{\mathbf{H}}(\mathbf{x}+\mathbf{y}) = \mathbf{0}$ is satisfied with probability $1/2^t$, where $d = (2\omega' - \omega)/2$. Note that $q(\mathbf{x}, \mathbf{y}, C_f, \alpha) = \Pr[\exists \mathbf{c} \in \mathcal{C}_f : \mathbf{c} \in \mathcal{B}_\alpha(\mathbf{x}) \cap \mathcal{B}_\alpha(\mathbf{y})]$ is the probability that $\mathbf{x}$ and $\mathbf{y}$ are added into one same bucket. Thus, for any pair of vectors $\mathbf{x}, \mathbf{y} \in L'_0$ or $\mathbf{x}, \mathbf{y} \in L'_1$, $\mathbf{x} + \mathbf{y}$ will be added into the list L_0 with probability

$$q = \frac{\delta(k+l, \omega, \omega') \cdot q(\mathbf{x}, \mathbf{y}, C_f, \alpha)}{2^t} = \frac{\binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d} \cdot q(\mathbf{x}, \mathbf{y}, C_f, \alpha)}{\binom{k+l}{\omega'} 2^t}. \quad (3.5)$$

Let $\bar{N}$ be the number of the pair of vectors $\mathbf{x}, \mathbf{y} \in L'_0$ or $\mathbf{x}, \mathbf{y} \in L'_1$ and let $|L'_0| = |L'_1| = N'/2$ for ANNS-Predicate-2. Obviously, $\bar{N} = 2 \cdot \binom{N'/2}{2} = N'^2/4 - N'/2 \approx N'^2/4$. Let $X_1, ..., X_{\bar{N}}$ be $\bar{N}$ independently and identically distributed random variables,[13] where X_i is 1 with probability q and 0 with probability $1-q$. Let $\delta = 1 - N/(2\bar{N}q)$. Thus, according to Lemma 2.1, we have $\Pr[X_1 + X_2 + \cdots + X_{\bar{N}} \leq (1-\delta)\bar{N}q = N/2] \leq \exp(-\bar{N}q\delta^2/2)$. Since the required size of the list L_0 is exactly $N/2$, we have $\Pr[|L_0| = N/2] \geq 1 - \exp(-\bar{N}q\delta^2/2)$. Similarly, we have $\Pr[|L_1| = N/2] \geq 1 - \exp(-\bar{N}q\delta^2/2)$. Thus, both lists L_0 and L_1 have size $N/2$ with probability

$$p \geq (1 - \exp(-\bar{N}q\delta^2/2))^2.$$

The running time of Algorithm 3 consists of two parts: the time of the bucketing phase and the time of the checking phase. In the bucketing phase, the valid filters are identified in time $T_{\text{ValidFilters}}$ and the triples $(0/1, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$ are written

[13] This is satisfied under the binary sieve heuristic.

into buckets (Lines 4 and 7) in time $\omega' \log(k+l)+t+1$.[14] The weight of vectors in L'_0 and L'_1 is ω', thus the computation of $\hat{\mathbf{H}}_t\mathbf{x}$ just needs to sum ω' columns of $\hat{\mathbf{H}}_t$. Thus, the expected time of the bucketing phase is

$$T_{\text{Bucket}} = N' \cdot (T_{\text{ValidFilters}} + E[|\mathcal{B}_\alpha(\mathbf{x})|] \cdot (\omega' \log(k+l) + t + 1) + (\omega' - 1) \cdot t).$$

Next, we show how the triple $(j, \mathbf{y}, \hat{\mathbf{s}}_t + \hat{\mathbf{H}}_t\mathbf{x})$ (Line 15) can be identified. Note that in the bucketing phase, table-I with elements $(flag, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$ and index $flag||\mathbf{x}$ ($flag$ is 0 or 1) is saved. Then, in the checking phase, one can first construct table-II with index $\mathbf{a}$ and elements $(\mathbf{a}, \hat{\mathbf{s}}_t + \mathbf{a})$, where $\mathbf{a} \in \{0,1\}^t$. Then, the pair $(j, \mathbf{y}, \hat{\mathbf{s}}_t + \hat{\mathbf{H}}_t\mathbf{x})$ can be easily identified using table-I and table-II without overhead. In detail, given $flag$ and $\mathbf{x}$, one can first find $\hat{\mathbf{H}}_t\mathbf{x}$ with table-I, then find $\hat{\mathbf{s}}_t + \hat{\mathbf{H}}_t\mathbf{x}$ via table-II with index $\hat{\mathbf{H}}_t\mathbf{x}$, finally identify candidates of $\mathbf{y}$.[15] The triple $(0/1, \mathbf{y}, \hat{\mathbf{H}}_t\mathbf{x})$ (Line 12 and Line 22) can be identified in the same way. The worst-case time of constructing table-II is $2^t \cdot t$. In particular, if $N'/2 < 2^t$, we choose to construct table-II directly with index $\hat{\mathbf{H}}_t\mathbf{x}$ where $\mathbf{x} \in L_0^{'}$. Thus, the time T_{table} is $\min\left\{\frac{N'}{2}, 2^t\right\} \cdot t$.

Then, for each $\mathbf{x} \in L'_0$, we only check the bucket that contains $\mathbf{x}$. Thus, only $E[|\mathcal{B}_\alpha(\mathbf{x})|]$ buckets are checked for a given $\mathbf{x} \in L'_0$. In particular, given $\mathbf{x} \in L'_0$, only $(0, \mathbf{y}, \hat{\mathbf{H}}_t\mathbf{x})$ or $(1, \mathbf{y}, \hat{\mathbf{s}}_t + \hat{\mathbf{H}}_t\mathbf{x})$ is checked in the bucket. Then, for each $\mathbf{x} \in L'_1$, we check $(1, \mathbf{y}, \hat{\mathbf{H}}_t\mathbf{x})$ in every bucket. For each pair $(\mathbf{x}, \mathbf{y})$, we need to check whether $|\mathbf{x} + \mathbf{y}| = \omega$ holds.[16] Thus, the expected time of the checking phase is $T_{\text{Check}} = T_{table} +$

$$\begin{aligned}
&\frac{N'}{2} \cdot E[|\mathcal{B}_\alpha(\mathbf{x})|] \cdot \left(\frac{E[|\text{Bucket}_\alpha(\mathbf{c})|]}{2^t \cdot 2} \cdot 2 + \frac{E[|\text{Bucket}_\alpha(\mathbf{c})|]}{2^t \cdot 2}\right) \cdot \omega' \log(k+l) \\
&= N' \cdot \frac{3 \cdot E[|\mathcal{B}_\alpha(\mathbf{x})|] \cdot E[|\text{Bucket}_\alpha(\mathbf{c})|] \cdot \omega' \log(k+l)}{2^{t+2}} + T_{table}.
\end{aligned}$$

Then the total running time of Algorithm 3 for ANNS-Predicate-2 is

$$\begin{aligned}
T = T_{table} + N' \cdot \Bigg(& T_{\text{ValidFilters}} + E[|\mathcal{B}_\alpha(\mathbf{x})|](\omega' \log(k+l) + t + 1) \\
& + (\omega' - 1)t + \frac{3 \cdot E[|\mathcal{B}_\alpha(\mathbf{x})|] \cdot E[|\text{Bucket}_\alpha(\mathbf{c})|] \cdot \omega' \log(k+l)}{2^{t+2}} \Bigg).
\end{aligned}$$

[14] The time of recording such a triple (vector) is counted by 1 in [19]'s conservative complexity model. In order to precisely estimate the gate complexity of our algorithm, we count this time by the length of triple (vector).

[15] When the triples $(flag, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$ are added into the $\text{Bucket}_\alpha(\mathbf{c})$ in the bucketing phase (Lines 4 and 7), we assume these triples are put into different sub-buckets according to the value of $\hat{\mathbf{H}}_t\mathbf{x}$.

[16] For low-weight vectors including $\mathbf{x}$ and $\mathbf{y}$, we just need to record the positions where "1" appears. Then, we can optimize the vector operations in the same way as in [19]. But, in order to make an accurate estimation, this check time is counted by $\omega' \log(k+l)$ in this paper rather than ω' used in [19].

Except the lists L'_0 and L'_1, we also need to store all the buckets and the aforementioned tables. Thus, $M = N'((\omega' \log(k+l)+t+1)+E[|\mathcal{B}_\alpha(\mathbf{x})|]\cdot(\omega' \log(k+l)+t+1))+N\cdot\omega\log(k+l)+2^{t+1}\cdot t$. □

LSF via GJN. In GJN-LSF [19,11], the filter set $\mathcal{C}_f = \mathcal{S}_d^{k+l}$ and $\alpha = d$. If for each $\mathbf{x} \in L'_0$ ($\mathbf{x} \in L'_1$, resp.) and each $\mathbf{c} \in \mathcal{C}_f$, $|\mathbf{x} \wedge \mathbf{c}| = d$ holds, then $\mathbf{x}$ is added into $\text{Bucket}_\alpha(\mathbf{c})$. Thus, if $\mathbf{x}$ and $\mathbf{y}$ are both in $\text{Bucket}_\alpha(\mathbf{c})$, we have $|\mathbf{x}+\mathbf{y}| \le \omega$.

Given $\mathbf{x}$ in L'_0 or L'_1, the way to identify all valid filters is to do a simple enumeration. Thus, the function **ValidFilters** is defined as

$$\textbf{ValidFilters}(\mathcal{S}_d^{k+l}, \mathbf{x}, d) \text{ returns } \mathcal{B}_{\mathcal{S}_d^{k+l},d}(\mathbf{x}) := \left\{\mathbf{c} \in \mathcal{S}_d^{k+l} : |\mathbf{x}\wedge\mathbf{c}| = d\right\}. \tag{3.6}$$

Then the time $T_{\text{ValidFilters}}$ to identify all valid filters is given by $T_{\text{ValidFilters}} = |\mathcal{B}_{\mathcal{S}_d^{k+l},d}(\mathbf{x})| = \binom{\omega'}{d}$. Note that every pair $\mathbf{x},\mathbf{y} \in S_{\omega'}^{k+l}$ such that $|\mathbf{x}+\mathbf{y}| = \omega$ will be added into at least one bucket due to the choice of $\mathcal{C}_f$ in GJN-LSF. Thus, we have $q(\mathbf{x},\mathbf{y},\mathcal{C}_f,\alpha) = 1$.

Corollary 3.1 (Complexity of ANNS-Predicate with GJN-LSF). *Let $\alpha = d$, $\mathcal{C}_f = \mathcal{S}_d^{k+l}$ and the function **ValidFilters** is defined in (3.6). Then, Algorithms 3 can solve the ANNS-Predicate-Problem in the expected time and expected memory*

$$T = T_{table} + N'(\omega'-1)t + N'\left(C_1\binom{\omega'}{d}(1+(\omega'-d)\log(k+l)+t+\bar{b})+\right.$$
$$\left.\binom{\omega'}{d}\frac{C_2N'\cdot\binom{\omega'}{d}}{2^t\binom{k+l}{d}}(\omega'-d)\log(k+l)\right)$$
$$M = N'((\omega'\log(k+l)+t+\bar{b})+C_3\binom{\omega'}{d}\cdot((\omega'-d)\log(k+l)+t+\bar{b}))+$$
$$C_4N\omega\log(k+l)+2^{t+1}\cdot t$$

with probability $p \ge (1-\exp(-\bar{N}q\delta^2/2))^2$ and $T_{table} = \min\{C_5N', 2^t\}\cdot t$, where $\bar{N} \approx C_6{N'}^2$, $\delta = 1-N/(2\bar{N}q)$ and $q = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'}\cdot 1/2^t$. In particular, the values $(C_1,\bar{b},C_2,C_3,C_4,C_5,C_6)$ are set to be $(1,0,2,1,1,1,1/2)$ for ANNS-Predicate-1, $(1,1,3/4,1,1,1/2,1/4)$ for ANNS-Predicate-2 and $(1/2,1,1/4,1/2,1/2,1/2,1/4)$ for ANNS-Predicate-3.

Proof. The time to find all valid filters is $T_{\text{ValidFilters}} = |\mathcal{B}_{\mathcal{S}_d^{k+l},d}(\mathbf{x})| = \binom{\omega'}{d}$. Note that the expected size of the bucket is $E[|\text{Bucket}_\alpha(\mathbf{c})|] = N'\cdot E[|\mathcal{B}_\alpha(\mathbf{x})|]/|\mathcal{C}_f| = N'\cdot\binom{\omega'}{d}/\binom{k+l}{d}$. In addition, according to the definition of filter set and bucket in GJN-LSF, one can easily observe that $\mathbf{x}\wedge\mathbf{c} = \mathbf{c}$ is satisfied for any $(0/1, \mathbf{x}, \hat{\mathbf{H}}\mathbf{x}) \in \text{Bucket}_\alpha(\mathbf{c})$. Thus, considering that $\mathbf{x}+(\mathbf{x}\wedge\mathbf{c})$ has smaller weight (that is $\omega'-d$) than $\mathbf{x}$ (with weight ω'), a minor optimization[17] can be implemented. In

[17] Take the Category 1 of HQC as an example, this minor optimization will bring a 0.6 bit improvement of time complexity for PRO-GJN.

detail, during the Bucketing phase, one can just record $(0/1, \mathbf{x}+\mathbf{c}, \hat{\mathbf{H}}_t\mathbf{x})$ rather $(0/1, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$. Accordingly, $\mathbf{x}+\mathbf{y}$ in the checking phase can be replaced by $(\mathbf{x}+\mathbf{c})+(\mathbf{y}+\mathbf{c})$. By substituting these quantities in (3.2) and (3.3) we can get the expected time and expected memory of Algorithm 3 via GJN-LSF. □

In [11], the authors notice that if the filter set is chosen by randomly picking a subset with suitable size in $\mathcal{S}_v^{k+l}$ $(v < d)$, then the asymptotic time complexity of sieving can be significantly improved. In particular, four probabilistic LSFs, CH, CH-OPT, RPC, RPC-OPT, have been proposed and can be used to instantiate Algorithm 3. Revisiting the asymptotic analysis in [11] carefully, we can derive the concrete complexities of Algorithm 3 with LSFs including CH, CH-OPT, RPC, RPC-OPT. We present these results in Appendix D.

3.3 Complexity of the Progressive Sieve

According to Theorem 3.1, in order to get the whole complexity of the progressive sieve, we just need to evaluate the time complexity T_i, memory complexity M_i and success probability p_i of the sieving in layer-i. Instantiating Algorithm 2 with different LSFs, we can easily derive these values based on the analysis in Sec. 3.2 and Appendix D. Let d_i be the overlap for a near neighbor pair $(\mathbf{x}_{i-1}, \mathbf{y}_{i-1})$ in the layer-i for $1 \le i \le l'$, where $\mathbf{x}_{i-1}, \mathbf{y}_{i-1} \in \mathcal{S}_{\omega_{i-1}}^{k+l}$. Then we have $d_i = \frac{2\omega_{i-1}-\omega_i}{2}$ and the probability that $\mathbf{x}_{i-1}+\mathbf{y}_{i-1} \in \mathcal{S}_{\omega_i}^{k+l}$ i.e. the neighbor pair $(\mathbf{x}_{i-1}, \mathbf{y}_{i-1})$ overlaps exactly at d_i positions is $\binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i}/\binom{k+l}{\omega_{i-1}}$. Let $\mathcal{C}_f^{(i)}$ be the filter set in layer-i. We denote $q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)$ as the probability that a near neighbor pair $(\mathbf{x}_{i-1}, \mathbf{y}_{i-1})$ is added to a bucket for layer-i, which is

$$q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i) = \Pr[\exists \mathbf{c}_i \in \mathcal{C}_f^{(i)} : \mathbf{c}_i \in \mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}) \cap \mathcal{B}_{\alpha_i}(\mathbf{y}_{i-1})]. \tag{3.7}$$

According to Sec. 3.2, the values N_{i-1} and N_i should satisfy Equation (3.8).

$$\frac{C_6^i N_{i-1}^2 \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i} q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)}{\binom{k+l}{\omega_{i-1}} 2^{t_i}} \ge \frac{N_i}{2}. \tag{3.8}$$

In addition, the list size in each layer should be less than the number of expected solutions.[18] That is, in addition to $N_0 \le \binom{k+l}{\omega_0}$, we also require that for $1 \le i \le l'$,

$$N_i/2 \le \binom{k+l}{\omega_i} / 2^{\sum_{j=1}^{i} t_j}. \tag{3.9}$$

The calculation methods of T_i, M_i, p_i with different LSFs are given by Appendix E.

[18] This implicit constraint is indispensable, and overlooked in [19]'s original concrete bit security analysis of NIST-KEMs. In appendix F, we provide a detailed discussion of parameter constraint (3.9).

4 Concrete cryptanalysis on NIST-PQC code-based KEMs

Instantiating the oracle in line 4 of Algorithm 1 with Algorithm 2, we obtain our progressive sieving-ISD. In this section, we will present the overall complexity of our progressive sieving-ISD and analyze the bit security of Classic McEliece, BIKE and HQC using this new sieving-ISD algorithm.

4.1 Overall Complexity of Progressive Sieving-style ISD

Outer Iteration. At the beginning of the algorithm, we first do a permutation. The probability of getting a good permutation such that the permuted error has the desired weight distribution is $P_1 = \binom{k+l}{\hat{\omega}}\binom{n-k-l}{\omega-\hat{\omega}}/\binom{n}{\omega}$. Besides, the probability that the solution is included in the output list is $P_2 = \min\left\{1, N \cdot 2^l/\binom{k+l}{\hat{\omega}}\right\}$ where N is the size of the output list. We can expect to get a good permutation by iterating $1/(P_1 \cdot P_2)$ times in expectation.

Gaussian Elimination. After permutation, we perform Gaussian elimination on the parity check matrix. Consistent with other work analyzing ISD algorithms [15,18,19], we also employ the *Method of Four Russian* (M4RI) [4,5,1] in this step. An open source version of estimating the complexity of M4RI is given by [15], and for a fair comparison with other work, we also use this Python script.

Progressive Sieve. Following the methods given by Sec. 3, we can directly obtain the expected running time $T_{sieve} = \sum_{i=1}^{l'} T_i$, expected memory $M_{sieve} = \max_{1\le i\le l'} M_i$, success probability $p_{sieve} = \prod_{1\le i\le l'} p_i$, where the evaluation of T_i, M_i and p_i with different LSFs is given by Appendix E. Note that two different pairs of vectors can produce the same sum, leading to a collision of newly produced vectors. That is, the vectors in L_i^0 and L_i^1 may be reduplicate. Therefore, in order to make a precise complexity analysis, we need to consider the effect of collisions on the probability p of the whole algorithm. Thus, the constraint between N_i and N_{i-1} given by Equation (3.8) should be updated when considering the duplicated vectors. In this paper, we follow [11, Lemma 6.1] to estimate the number of unique vectors in L_i^0 and L_i^1. Note that for the layer-i ($1 \le i \le l'$), the expected number of newly generated vector in $\mathcal{S}_{\omega_i}^{k+l}$ is given by

$$\bar{N}_i \approx C_6^i N_{i-1}^2 \cdot \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i}\Big/\binom{k+l}{\omega_{i-1}}, \tag{4.1}$$

where C_6^i is $1/2$ for $i = 1$ and $1/4$ for $2 \le i \le l'$. The expected number D_i of vectors that lie in $\mathcal{S}_{\omega_i}^{k+l}$ and satisfy all the equations in first $i-1$ layers is $\binom{k+l}{\omega_i}/2^{\sum_{j=1}^{i-1} t_j}$.

Lemma 4.1 (Unique Solutions [11, Lemma 6.1]). *Let $U_{\bar{N}_i}$ be a random variable that counts the number of different elements obtained in a list consisting of $\bar{N}_i$ elements uniformly and independently sampled from a set of size D_i for any given layer $1 \leq i \leq l'$. It is expected that $E[U_{\bar{N}_i}] = (1 - c_i^{\bar{N}_i})/(1 - c_i)$, where $c_i = 1 - 1/D_i$.*

Thus, based on Lemma 4.1, we can get the expected number $E[U_{\bar{N}_i}]$ of unique vectors in L_i^0 and L_i^1, which should satisfy the following equation.

$$D_i \geq E[U_{\bar{N}_i}] = \frac{(1 - c_i^{\bar{N}_i})}{1 - c_i} \geq \frac{N_i \cdot 2^{t_i}}{2 \cdot q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)}. \tag{4.2}$$

Thus, the constraint between N_i and N_{i-1} given by Equation (3.8) should be updated by Equation

$$2D_{i-1} \geq N_{i-1} \geq \sqrt{\frac{\log[1 - \frac{(1-c_i)\cdot N_i \cdot 2^{t_i}}{2 \cdot q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)}]}{C_6^i \cdot \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i}/\binom{k+l}{\omega_{i-1}} \cdot \log c_i}}. \tag{4.3}$$

Remark 4.1. Note that in the calculation of probability p_i in Sec. 3.3 and Appendix E, the duplicated vectors are also neglected. In the following concrete cryptoanalysis, we take this into account and replace the expected number of newly vectors in layer-i that is used in the estimation of p_i by $E[U_{\bar{N}_i}] \cdot 1/2^{t_i} \cdot q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)$.

Check. At the end of the algorithm, we need to compute $\bar{\mathbf{e}} = \bar{\mathbf{H}}\hat{\mathbf{e}} + \bar{\mathbf{s}}$ and check if its weight is $\omega - \hat{\omega}$. If $\bar{\mathbf{e}}$ satisfies the weight assumption, then add it to the list of solutions. Since the length of every row in $\bar{\mathbf{H}}$ is $n-k-l$ and the weight of $\hat{\mathbf{e}}$ is $\hat{\omega}$, the cost time of this step as estimated in [19] is $T_{check} = (\hat{\omega}+1)\cdot(n-k-l)\cdot N_{l'}/2$.

Then, we can obtain the total running time and memory complexity of the progressive sieving-style ISD algorithm.

Theorem 4.1 (Bit complexity of the progressive sieving-style ISD). *Let $P_{outer} = P_1 \cdot P_2$. The expected time complexity and memory complexity of the progressive sieving-style ISD algorithm are*

$$T_{ISD} = 1/P_{outer} \cdot \left(T_{gauss} + p^{-1} \cdot T_{sieve} + T_{check}\right), M_{ISD} = M_{sieve} + M_{gauss},$$

where T_{gauss} and M_{gauss} are respectively the expected time and memory of Gaussian elimination.

4.2 Cryptanalysis of Classic McEliece, BIKE and HQC

In this section, we will present the concrete cryptanalysis on Classic McEliece, BIKE and HQC. In particular, Classic McEliece relies on Goppa codes with

relatively large weight $\omega = \Theta(n/\log(n))$, while BIKE and HQC use small weight $\omega = \Theta(\sqrt{n})$. The parameter sets for these three schemes are listed in Table H.1 in Appendix H. As we can see, both BIKE and HQC have a code rate of $\frac{k}{n} = 1/2$ and are sparser than Classic McEliece. The bit security of Classic McEliece under attacks via our progressive sieving-ISD can be directly obtained by following the complexity estimation method in Sec. 4.1.

Cryptanalysis on BIKE (key-recovery). Both BIKE and HQC use the double circulant codes with code rate 1/2 (that is, $n = 2k$) to improve the efficiency of the KEM schemes. Such a cyclic structure will introduce k instances of the original SD problem, where a solution to any of the k instances is just a cyclic rotation of the original solution. This technique is widely known as "decoding one out of many" (DOOM).

In details, given $\mathbf{H} = (\mathbf{H}_1, \mathbf{H}_2)(\mathbf{H}_1, \mathbf{H}_2 \in \mathbb{F}_2^{k\times k})$ and $\mathbf{s} \in \mathbb{F}_2^k$ in the standard SD problem, one is asked to find a vector $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2)$ $(\mathbf{e}_1, \mathbf{e}_2 \in \mathbb{F}_2^k)$ satisfying $\mathbf{He} = \mathbf{s}$. Let $\mathrm{rot}_i(\mathbf{x})$ be the cyclic left rotation of $\mathbf{x}$ by i positions. Thus, the cyclicity leads to that for any $i \in \{0, 1, \ldots, k-1\}$, we have

$$\mathbf{H}(\mathrm{rot}_i(\mathbf{e}_1), \mathrm{rot}_i(\mathbf{e}_2)) = (\mathbf{H}_1, \mathbf{H}_2)(\mathrm{rot}_i(\mathbf{e}_1), \mathrm{rot}_i(\mathbf{e}_2)) = \mathrm{rot}_i(\mathbf{s}).$$

Therefore, a solution to any of the instance $(\mathbf{H}, \mathrm{rot}_i(\mathbf{s}), \omega)$ can yield the original solution $\mathbf{e}$.

For the key-recovery on BIKE, the syndrome $\mathbf{s}$ is exactly the zero vector $\mathbf{0}$. Thus, all $(\mathrm{rot}_i(\mathbf{e}_1), \mathrm{rot}_i(\mathbf{e}_2))$ $(i \in \{0, 1, \ldots, k-1\})$ are the solutions of the involved SD problem. For the key-recovery on BIKE, any ISD algorithm obtains a speedup of k [18].

Cryptanalysis on BIKE (message-recovery) and HQC. In this case, the syndrome $\mathbf{s}$ is not $\mathbf{0}$ and the ISD algorithms, including Stern, MMT and BJMM, can achieve a speedup of $\sqrt{k}$ [24,18].

Progressive sieving-ISD for the DOOM setting. In [19], the authors sketched the idea of how to adapt the sieving-ISD algorithm to benefit from the DOOM technique. However, due to the lack of analysis of concrete complexity and success probability, they did not give the bit security results for BIKE (message-recovery) and HQC in the DOOM setting.

In this work, thanks to our simplified method of combining vectors used in the sieving step and our concise analysis of the duplicated vectors,[19] we extend

[19] On average, $1/2^{t_i}$ parts of the vectors in layer-$(i-1)$ are already satisfy $\hat{\mathbf{H}}_{[t_i]}\mathbf{x} = \mathbf{s}_{[t_i]}$. Thus, these vectors are directly forwarded to layer-i in [19]. But, such an operation will make the analysis of duplicated vectors complicated [11]. In order to follow the concise analysis of the duplicated vectors given by [11], we disregard those vectors in the sieving as in [11]. In addition, a collision-filtering technique is also proposed to reduce the memory complexity in [19]. In order to simplify the complexity analysis, we also remove this technique in GJN-LSF.

[19]'s idea to give our progressive sieving-ISD for the DOOM setting and at the same time strictly analyze its concrete complexity. In particular, different from [19]'s idea, we employ an unbalanced weight configuration (vectors in L_i^0 ($L_i^j (j \geq 1)$, resp.) have weight ω_i+1 (ω_i, resp.)) to adapt our progressive sieving-ISD algorithm to further benefit from the DOOM technique. This unbalanced trick is important for the improvements in the DOOM setting, see Remark 4.2 for the reason.

Now, we first describe how to adapt the progressive sieving-ISD to the DOOM setting. Let $(\mathbf{H}, \mathbf{s}_i = \mathrm{rot}_i(\mathbf{s}_0), \omega)$ $(0 \leq i < k)$ be the k SD instances and $\mathbf{He}_i = \mathbf{s}_i$. As in the non-DOOM setting, one chooses a random permutation matrix $\mathbf{P}$ and performs a Gaussian elimination $\mathbf{G}$ on $\mathbf{HP}$ and $\mathbf{s}_i$. That is, we have

$$\mathbf{H}' = \mathbf{GHP} = \begin{bmatrix} \hat{\mathbf{H}} & \mathbf{0} \\ \bar{\mathbf{H}} & \mathbf{I}_{n-k-l} \end{bmatrix}, \mathbf{e}_i' = \mathbf{P}^{-1}\mathbf{e}_i = (\widehat{\mathbf{e}}_i, \bar{\mathbf{e}}_i), \mathbf{s}_i' = \mathbf{G}\mathbf{s}_i = (\widehat{\mathbf{s}}_i, \bar{\mathbf{s}}_i).$$

Then, the DOOM version of Progressive Sieving (called Progressive DOOM-Sieving in the following, see Algorithms 4 and 5 in Appendix G for the complete algorithm description) is invoked with input $(\hat{\mathbf{H}}, \{\widehat{\mathbf{s}}_i\}_{0\leq i<k}, \hat{\omega}, L_0^0, L_0^1)$ (list $L_0^0 \subseteq \mathcal{S}_{\omega_0+1}^{k+l}$ with size $N_0/2$ and list $L_0^1 \subseteq \mathcal{S}_{\omega_0}^{k+l}$ with size $N_0/2$) to obtain k lists $\{L_{l'}^i\}_{1\leq i\leq k}$, where for any $\widehat{\mathbf{e}}_i \in L_{l'}^i$ with weight $|\widehat{\mathbf{e}}_i| = \hat{\omega}$, we have $\hat{\mathbf{H}}\widehat{\mathbf{e}}_i = \widehat{\mathbf{s}}_i$. At last, for any $1 \leq i \leq k$ and $\widehat{\mathbf{e}}_i \in L_{l'}^i$, check whether the weight of $\bar{\mathbf{e}}_i = \bar{\mathbf{H}}\widehat{\mathbf{e}}_i + \bar{\mathbf{s}}_i$ is $\omega - \hat{\omega}$ and return $\mathbf{P}^{-1}(\widehat{\mathbf{e}}_i, \bar{\mathbf{e}}_i)$ if the check passes.

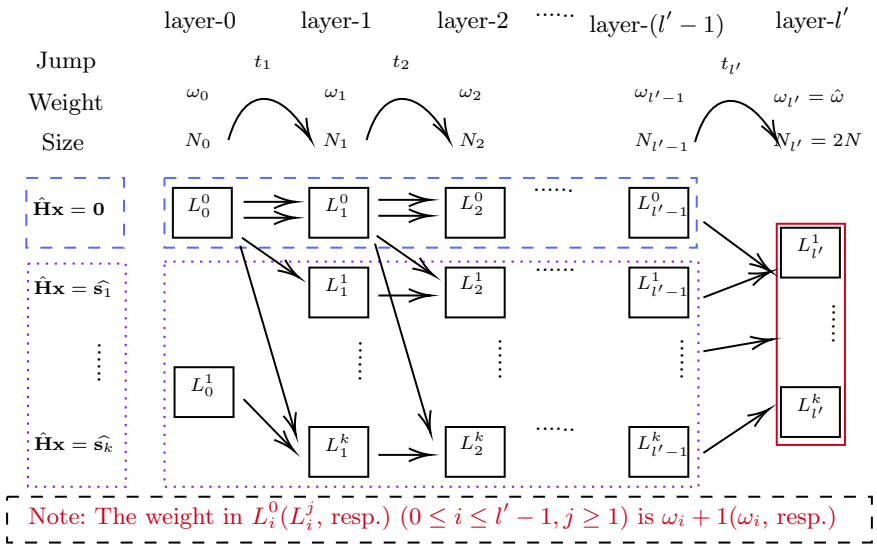

Fig. 4.1: The Framework of Progressive DOOM-Sieving

The skeleton of the Progressive DOOM-Sieving is given by Fig. 4.1. Unlike the Progressive sieving described in Section 3, the weight of the elements in list

$L_i^0 (0 \le i \le l'-1)$ is set to $\omega_i + 1$. This adjustment makes it easier for the list size of L_i^0 to satisfy the Equation (4.9). The goal of the Progressive DOOM-Sieving is to find $k+1$ list towers $\{L_1^j, \ldots, L_{l'-1}^j\}_{0 \le j \le k}$ such that for any $i \in \{1, \ldots, l'-1\}$ and $1 \le j \le k$ we have $L_i^0 \subseteq S_{\omega_{i+1}}^{k+l}$, $L_i^j \subseteq S_{\omega_i}^{k+l}$, $|L_i^0| = N_i/2$, $|L_i^j| = N_i/2k$ and $\hat{\mathbf{H}}_{1\to i}\mathbf{x} = \mathbf{0}$ ($\hat{\mathbf{H}}_{1\to i}\mathbf{x} = \hat{\mathbf{s}}_{1\to i}^j$, resp.) for any $\mathbf{x} \in L_i^0$ ($\mathbf{x} \in L_i^j$, resp.), where $\widehat{\mathbf{s}}_j = [\hat{\mathbf{s}}_1^j, \hat{\mathbf{s}}_2^j, ..., \hat{\mathbf{s}}_{l'}^j]$ and $\hat{\mathbf{s}}_{1\to i}^j = [\hat{\mathbf{s}}_1^j, \hat{\mathbf{s}}_2^j, ..., \hat{\mathbf{s}}_i^j]$. As in the non-DOOM case, for the layer-l', we require $\omega_{l'} = \hat{\omega}$ and $N_{l'} = 2N$. Thus, the $\{L_{l'}^j\}_{1 \le j \le k}$ is the expected k lists $\{L_{i+1}\}_{0 \le i < k}$ to return (the size of the list $L_{l'}^j$ is N/k.). The initial list L_0^0 (L_0^1, resp.) is obtained by uniformly sampling elements from $\mathcal{S}_{\omega_0+1}^{k+l}$ ($\mathcal{S}_{\omega_0}^{k+l}$, resp.) at random, with each list having size $N_0/2$. The whole Progressive DOOM-Sieving is given in Appendix G by Algorithm 4, where the embedded ANNS-Predicate-DOOM algorithms are given by Algorithm 5.

In layer-1, the elements in L_1^0 are obtained by summing pairs of vectors in list L_0^0, and the elements in L_1^j ($j \ge 1$) are obtained by combining the elements in list L_0^0 and the elements in list L_0^1. That is,

$$L_1^0 = \{\mathbf{x}+\mathbf{y} : \mathbf{x}, \mathbf{y} \in L_0^0, \hat{\mathbf{H}}_1(\mathbf{x}+\mathbf{y}) = \mathbf{0} \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_1 + 1\}$$

$$L_1^j = \{\mathbf{x}+\mathbf{y} : \mathbf{x} \in L_0^0, \mathbf{y} \in L_0^1, \hat{\mathbf{H}}_1(\mathbf{x}+\mathbf{y}) = \hat{\mathbf{s}}_1^j \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_1\}.$$

In the layer-i ($2 \le i \le l'-1$), the elements in L_i^0 are obtained by summing pairs of vectors in list L_{i-1}^0[20], and the elements in L_i^j are obtained by combining the elements in list L_{i-1}^0 and the elements in list L_{i-1}^j ($j \ge 1$). That is, for $2 \le i \le l'-1$, we have

$$\begin{aligned} L_i^0 =& \{\mathbf{x}+\mathbf{y} : \mathbf{x}, \mathbf{y} \in L_{i-1}^0, \hat{\mathbf{H}}_{1\to i}(\mathbf{x}+\mathbf{y}) = \mathbf{0} \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_i + 1\}, \\ L_i^j =& \{\mathbf{x}+\mathbf{y} : \mathbf{x} \in L_{i-1}^0, \mathbf{y} \in L_{i-1}^j, \hat{\mathbf{H}}_{1\to i}(\mathbf{x}+\mathbf{y}) = \hat{\mathbf{s}}_{1\to i}^j \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_i\}. \end{aligned}$$

In the layer-l', the list $L_{l'}^j$ ($j \ge 1$) is obtained by summing the elements in list $L_{l'-1}^0$ and the elements in list $L_{l'-1}^j$. That is,

$$L_{l'}^j = \left\{\mathbf{x}+\mathbf{y} : \mathbf{x} \in L_{l'-1}^0, \mathbf{y} \in L_{l'-1}^j, \hat{\mathbf{H}}_{1\to l'}(\mathbf{x}+\mathbf{y}) = \hat{\mathbf{s}}_{1\to l'}^j \text{ and } |\mathbf{x}+\mathbf{y}| = \omega_{l'}\right\}.$$

Complexity Analysis. Since one of the solutions in the k SD instances implies all the solutions, the success probability of the outer iteration is given by

$$P_{outer}^{DOOM} = 1 - (1 - P_1 P_2^{DOOM})^k \approx k P_1 P_2^{DOOM},$$

[20] In the non-DOOM case, the vectors in both L_{i-1}^0 and L_{i-1}^1 are used to generate the vectors in L_i^0. But, in the DOOM case, the number of pairs in all L_{i-1}^j ($j \ge 1$) is $k \cdot \binom{\frac{N_{i-1}}{2^k}}{2} \approx \frac{(N_{i-1})^2}{8k}$, which is quite smaller than the number $\frac{(N_{i-1})^2}{8}$ of pairs in L_{i-1}^0. Thus, to make a concise presentation, we just combine the vectors in L_{i-1}^0 and ignore the vectors in L_{i-1}^j ($j \ge 1$). Take Category 1 of HQC as an example, including these pairs in L_{i-1}^j ($j \ge 1$) will only reduce the time complexity by less than 1 bit.

where $P_1 = \binom{k+l}{\hat{\omega}}\binom{n-k-l}{\omega-\hat{\omega}}/\binom{n}{\omega}$ and $P_2^{DOOM} = \min\left\{1, \frac{N}{k}\cdot 2^l/\binom{k+l}{\hat{\omega}}\right\}$. We also employ M4RI to calculate the time of the Gaussian elimination step. Since the size of $\cup_{i=1}^{k} L_{l'}^i$ is $N_{l'}/2$, thus the running time of the Check step is the same as the ones in the non-DOOM case.

Following the complexity analysis method of progressive sieving, we can know that the DOOM-sieving step has running time complexity T_{sieve}^{DOOM}, the memory complexity M_{sieve}^{DOOM}, and the success probability p^{DOOM}, where

$$T_{sieve}^{DOOM} = \sum_{i=1}^{l'} T_i^{DOOM},\ M_{sieve}^{DOOM} = \max_{i=1}^{l'} M_i^{DOOM},\ p^{DOOM} = \prod_{i=1}^{l'} p_i^{DOOM}.$$

Next, we show how to estimate the values of T_i^{DOOM}, M_i^{DOOM} and p_i^{DOOM}. Let $T^i_{\text{ValidFilters}}(0)$ ($T^i_{\text{ValidFilters}}(1)$, resp.) be the time for each $\mathbf{x}_{i-1} \in L_{i-1}^0$ ($\mathbf{x}_{i-1} \in L_{i-1}^j (j \geq 1)$, resp.) to find valid filters, $E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}, 0)|]$ ($E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}, 1)|]$, resp.) be the expected number of valid filters assigned to each $\mathbf{x}_{i-1} \in L_{i-1}^0$ ($\mathbf{x}_{i-1} \in L_{i-1}^j (j \geq 1)$, resp.) and let $E[|\text{Bucket}_{\alpha_i}(\mathbf{c}_i, 0)|]$ ($E[|\text{Bucket}_{\alpha_i}(\mathbf{c}_i, 1)|]$, resp.) denote the number of the elements $(0, \mathbf{x}_{i-1}, \hat{\mathbf{H}}\mathbf{x}_{i-1})$ (*all* $(1, \mathbf{x}_{i-1}, \hat{\mathbf{H}}\mathbf{x}_{i-1})$, $(2, \mathbf{x}_{i-1}, \hat{\mathbf{H}}\mathbf{x}_{i-1}), ..., (k, \mathbf{x}_{i-1}, \hat{\mathbf{H}}\mathbf{x}_{i-1})$, resp.) in the bucket $E[|\text{Bucket}_{\alpha_i}(\mathbf{c}_i)|]$. In particular, we have

$$\begin{aligned}
T_i^{DOOM} = \frac{N_{i-1}}{2}\Big(& C_1^i\Big(T^i_{\text{ValidFilters}}(0) + E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}, 0)|]((\omega_{i-1}+1)\log(k+l) \\
& + t_i + \log(k+1))\Big) + T^i_{\text{ValidFilters}}(1) + E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}, 1)|](\omega_{i-1}\log(k+l) \\
& + t_i + \log(k+1)) + (2\omega_{i-1}-1)t_i + E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}, 0)|]\cdot 1/2^{t_i}\cdot\Big((C_2^i)^{(0)}\cdot \\
& E[|\text{Bucket}_{\alpha_i}(\mathbf{c}_i, 0)|] + (C_2^i)^{(1)}E[|\text{Bucket}_{\alpha_i}(\mathbf{c}_i, 1)|]\Big)\cdot(\omega_{i-1}+1)\log(k+l)\Big) \\
& + T_{table}^i, \qquad (4.4)
\end{aligned}$$

$$\begin{aligned}
M_i^{DOOM} = \frac{N_{i-1}}{2}\Big(& ((2\omega_{i-1}+1)\log(k+l) + t_i) \\
& + C_3^i E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}, 0)|]((\omega_{i-1}+1)\log(k+l) + t_i + \log(k+1)) \\
& + E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1}, 1)|](\omega_{i-1}\log(k+l) + t_i + \log(k+1))\Big) \\
& + \frac{N_i}{2}(C_4^i\omega_i + C_5^i)\log(k+l) + 2^{t_i+1}t_i k, \qquad (4.5)
\end{aligned}$$

$$p_i^{DOOM} \geq (1 - \exp(-(C_6^i)^{(0)}N_{i-1}^2 q_0(\delta_0^i)^2/2))(1 - \exp(-(C_6^i)^{(1)}N_{i-1}^2 q_1(\delta_1^i)^2/2))^k, \qquad (4.6)$$

where $q_0 = \binom{\omega_{i-1}+1}{d_i}\binom{k+l-\omega_{i-1}-1}{\omega_{i-1}+1-d_i}/\binom{k+l}{\omega_{i-1}+1}\cdot q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)/2^{t_i}$, $T_{table}^i = \min\{N_{i-1}/2, 2^{t_i}\}\cdot t_i \cdot k$, $d_i = (2\omega_{i-1}+1-\omega_i)/2$, $q_1 = \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}+1-d_i}/\binom{k+l}{\omega_{i-1}+1}\cdot$

$q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)/2^{t_i}$, and the values of $(C_1^i, (C_2^i)^{(0)}, (C_2^i)^{(1)}, \delta_0^i, C_3^i, \delta_1^i, C_4^i, C_5^i, (C_6^i)^{(0)}, (C_6^i)^{(1)})$ are set to $(1, 1, k, 1 - N_i/(2(C_6^i)^{(0)} N_{i-1}^2 q_0), 1, 1 - N_i/(2k(C_6^i)^{(1)} N_{i-1}^2 q_1)\ , 2, 1, 1/8, 1/4)$ for $i = 1$, $(1, 1, 1, 1 - N_i/(2(C_6^i)^{(0)} N_{i-1}^2 q_0), 1, 1 - N_i/(2k (C_6^i)^{(1)} N_{i-1}^2 q_1), 2, 1, 1/8, 1/(4k))$ for $2 \le i \le l' - 1$ and $(0, 0, 1, 1 - N_i/(2(C_6^i)^{(0)} N_{i-1}^2 q_0), 0, 1 - N_i/(2k(C_6^i)^{(1)} N_{i-1}^2 q_1), 1, 0, 1/8, 1/(4k))$ for $i = l'$.

The values N_{i-1} and N_i should satisfy Equations (4.7) and (4.8).

$$\frac{(C_6^i)^{(0)} N_{i-1}^2 \binom{\omega_{i-1}+1}{d_i} \binom{k+l-\omega_{i-1}-1}{\omega_{i-1}+1-d_i} q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)}{\binom{k+l}{\omega_{i-1}+1} 2^{t_i}} \ge \frac{N_i}{2}, 1 \le i \le l' - 1. \tag{4.7}$$

$$\frac{(C_6^i)^{(1)} N_{i-1}^2 \binom{\omega_{i-1}}{d_i} \binom{k+l-\omega_{i-1}}{\omega_{i-1}+1-d_i} q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i)}{\binom{k+l}{\omega_{i-1}+1} 2^{t_i}} \ge \frac{N_i}{2k}, 1 \le i \le l' \tag{4.8}$$

In addition, the list size in each layer should be less than the number of expected solutions. That is, in addition to

$$\frac{N_0}{2} \le \min\left(\binom{k+l}{\omega_0}, \binom{k+l}{\omega_0+1}\right),$$

we also require N_i satisfying

$$\frac{N_i}{2} \le \binom{k+l}{\omega_i+1} / 2^{\sum_{j=1}^{i} t_j}, \text{for } 1 \le i \le l' - 1 \tag{4.9}$$

$$\frac{N_i}{2k} \le \binom{k+l}{\omega_i} / 2^{\sum_{j=1}^{i} t_j}, \text{for } 1 \le i \le l'. \tag{4.10}$$

Remark 4.2. Note that $\frac{\binom{k+l}{\omega_i+1}}{k\binom{k+l}{\omega_i}} = \frac{k+l-\omega_i}{k(\omega_i+1)} \approx \frac{1}{\omega_i}$, since l and ω_i are much smaller than k. That is, the condition (4.9) is *slightly* stronger the condition (4.10). If we directly follow [19]'s DOOM strategy and replace $\omega_i + 1$ in condition (4.9) by ω_i, then condition (4.9) is *much* stronger than condition (4.10), and will impose significant restrictions on the attack parameter selection. In fact, conditions (4.9) and (4.10) can be further balanced, by scaling the list sizes and setting $|L_i^0| = N_i \cdot \frac{\omega_i}{\omega_i+1}$, $|L_i^j| = N_i \cdot \frac{1}{k(\omega_i+1)}$. This scaling trick can further reduce the complexity of our attack, though the improvement is slight and comes with the cost of more complicated presentation and analysis. In this paper, we choose not to incorporate this trick in our algorithm in order to maintain a concise presentation.

In the DOOM setting, the Equation (4.3) should be updated as follows

$$2(D_{i-1})^{(0)} \geq N_{i-1} \geq \sqrt{\frac{\log\left[1-\frac{(1-(c_i)^{(0)})N_i\cdot 2^{t_i}}{2\cdot q(\mathbf{x}_{i-1},\mathbf{y}_{i-1},\mathcal{C}_f^{(i)},\alpha)}\right]}{(C_6^i)^{(0)}\cdot\binom{\omega_{i-1}+1}{d_i}\binom{k+l-\omega_{i-1}-1}{\omega_{i-1}+1-d_i}/\binom{k+l}{\omega_{i-1}+1}\cdot\log\left((c_i)^{(0)}\right)}}, \quad \text{for } 1\leq i\leq l'-1 \tag{4.11}$$

$$2k(D_{i-1})^{(1)} \geq N_{i-1} \geq \sqrt{\frac{\log\left[1-\frac{(1-(c_i)^{(1)})N_i\cdot 2^{t_i}}{2k\cdot q(\mathbf{x}_{i-1},\mathbf{y}_{i-1},\mathcal{C}_f^{(i)},\alpha)}\right]}{(C_6^i)^{(1)}\cdot\binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}+1-d_i}/\binom{k+l}{\omega_{i-1}+1}\cdot\log\left((c_i)^{(1)}\right)}}, \quad \text{for } 1\leq i\leq l' \tag{4.12}$$

where $(D_i)^{(0)} = \binom{k+l}{\omega_i+1}/2^{\sum_{j=1}^{i-1}t_j}$, $(c_i)^{(0)} = 1-1/(D_i)^{(0)}$, $(D_i)^{(1)} = \binom{k+l}{\omega_i}/2^{\sum_{j=1}^{i-1}t_j}$ and $(c_i)^{(1)} = 1-1/(D_i)^{(1)}$.

We present the complete complexity analysis in Appendix G. The values of the terms $T^i_{\text{ValidFilters}}(j)$, $E[|\mathcal{B}_{\alpha_i}(\mathbf{x}_{i-1},j)|]$ and $E[|\text{Bucket}_{\alpha_i}(\mathbf{c}_i,j)|]$ (j is 0 or 1) depend on the concrete LSFs and can be determined by instantiating various LSFs in the same way as in Appendix D. Additionally, the duplicated vectors also need to be removed in the same way as in Sec. 4.1. Finally, we can estimate the complexity of the progressive sieving-ISD under the DOOM setting according to Theorem 4.2.

Theorem 4.2 (Complexity of progressive sieving-ISD under DOOM). *The expected running time complexity of the progressive sieving-ISD algorithm under DOOM is*

$$T_{ISD}^{DOOM} = \frac{1}{P_{outer}^{DOOM}}\cdot\left(T_{gauss}^{DOOM} + (p^{DOOM})^{-1}\cdot T_{sieve}^{DOOM} + T_{check}\right)$$

while the expected memory complexity $M_{ISD}^{DOOM} = M_{sieve}^{DOOM} + M_{gauss}^{DOOM}$, where T_{gauss}^{DOOM} and M_{gauss}^{DOOM} are respectively the expected time and memory of Gaussian elimination.

4.3 Numerical Results

As in [19], the complexities are estimated in the RAM model with unit-time memory access. For comparison with previous ISD algorithms, we follow the work [19] to mainly focus on MMT and BJMM variants and opt not to include Both-May [6] and May-Ozerov [21] variants (those are unclear how the polynomial overhead from using Nearest-neighbor Search can affect the concrete complexity). In order to make a fair comparison, we use [19]'s Python script[21] to reproduce the bit security of Stern [25], MMT [20], BJMM [3], and GJN [19], by making the following slight adjustments.

1. The implicit constraint in (3.9) is added.
2. The probability P_2 in Theorem 2.1 is included.

[21] https://github.com/vunguyen95/Review-ISD-Sieving

3. The time of the recording/reading a vector is counted by "the vector length" rather than 1.

The bit security results using our progressive sieving-ISD with five different LSFs are obtained by following the estimation method in Sec. 4.2 and optimizing the attack parameters. Considering that the estimation of duplicated vectors in [19] is different from ours, we also re-estimate the GJN complexity in our model, denoted by NO-PRO-GJN. In detail, all the N_i (and ω_i) are equal and all t_i are set to be 1 in NO-PRO-GJN. Our code for calculating the complexities listed in Table 1.1 and 1.2 can be found at anonymized repository https://github.com/yodong912/Code-For-Progressive-Sieving-ISD. The optimal parameters used in PRO-GJN are provided in Appendix I. In addition, in the aforementioned anonymized repository, we also provide a csv file that contains the optimal values of N_i, ω_i, t_i, and the code for verifying the constraints (3.8), (3.9), (4.3), (4.7)-(4.12). Figures I.2 and I.3 in Appendix I show that these constraints are satisfied by our optimal attack parameters.

Table 1.1 shows the bit security estimates for Classic McEliece. As we can see, our progressive sieving-ISD with GJN-LSF (denoted by PRO-GJN) achieves the best time complexity, and brings an improvement over the non-progressive version NO-PRO-GJN by 7-12 bits, over the state-of-the-art results by 4-7 bits.

The bit security estimates for HQC and BIKE are presented in Table 1.2. In [19, Table I], the DOOM improvements are not considered. In this paper, we slightly modify [19]'s script by adding the DOOM improvement of the ISD algorithms including Stern, MMT and BJMM. For key recovery of BIKE, the bit security results based on [19]'s algorithm are accordingly modified by reducing $\log k$ bits. But for message recovery of BIKE, we leave [19]'s results without modification (denoted by GJN* in Table 1.2) due to the lack of a rigorous DOOM improvement analysis. As shown by Table 1.2, for BIKE and HQC, PRO-GJN achieves the best time complexity, and brings an improvement over NO-PRO-GJN by 5-10 bits. In particular, PRO-GJN achieves an improvement over the state-of-the-art results of HQC by 7-9 bits.

The comparison among different LSFs. For the SD problem in the full distance setting (that is, the value ω matches the Gilbert-Varshamov bound), the probabilistic LSFs have been proved to asymptotically outperform the deterministic GJN-LSF. However, in the parameter regions of NIST-KEMs ($\omega = \Theta(n/\log(n))$ or $\sqrt{n}$), our results show that progressive sieving-ISD with deterministic GJN-LSF outperforms the ones based on the aforementioned probabilistic LSFs. This is due to the following two reasons.

- First, the asymptotic improvement of probabilistic LSFs mainly comes from optimizing the size v of the filter center vector and the bucket parameter α, where $\alpha \leq v < d$. However, in the concrete security analysis of NIST-KEMs, the value of d is small which limits such an optimization.
- Second, some polynomial overheads are neglected when only considering the asymptotic complexity and these overheads are large for the real-world parameter setting. In particular, in order to compensate for the decreased

probability $q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i) < 1$ (see Equation (3.7)) (which is approximately taken as 1 in the asymptotic analysis), the list size N_{i-1} is required to be increased to guarantee the success probability of p_i and p_i^{DOOM},[22] which in turn increases the running time complexity.

In addition, for the RPC-LSF and RPC-LSF-OPT, $k'(k+l)((k+l)/m+1)^{3m}$ permutations are additionally performed to guarantee the appropriate weight distribution of elements in the list. Although such a quantity is polynomial, it also brings a significant overhead of running time in concrete settings.

5 Connection between progressive sieving-ISD and BJMM

5.1 Understanding MMT/BJMM in the progressive sieving-ISD framework

As shown by Tables 1.1 and 1.2, BJMM (with two depths) achieves the best time complexities as a conventional ISD algorithm when attacking NIST-KEMs. Both BJMM and progressive sieving-ISD follow Dumer's ISD framework [12] in Algorithm 1. That is, the main difference between BJMM and progressive sieving-ISD is the sub-algorithm solving the SMD problem. In the progressive sieving-ISD, the SMD problem is reduced to the ANNS-predicate problem (see Definition 3.1), which is solved by our ANNS-predicate algorithm 3. As shown by Fig. 5.1, the underlying SMD problem in BJMM is also reduced to the ANNS-predicate problem. In detail, take the BJMM with two depths as an example, the merge algorithm in depth-2 (depth-1, resp.) essentially solves the ANNS-predicate-3 problem (a variant,[23] resp.).

In BJMM, the merging algorithm is a simple collision-searching algorithm that includes the following two main steps:

1. First find the set that contains all the colliding pairs, $S = \{(\mathbf{x}, \mathbf{y}) : \hat{\mathbf{H}}\mathbf{x} + \hat{\mathbf{H}}\mathbf{y} = \hat{\mathbf{s}}\}$;
2. Then, check the elements in S and filter out all the pairs in S such that $|\mathbf{x} + \mathbf{y}| = \omega$.

We call such a method BJMM in short. In fact, this method is also used in Stern, MMT. Note that both $\mathbf{x}$ and $\mathbf{y}$ have weight $\omega' = \omega/2 + \varepsilon$. Thus, the probability $|\mathbf{x} + \mathbf{y}| = \omega$ is $P_{filter} = \frac{\binom{\omega'}{\varepsilon}\binom{k+l-\omega'}{\omega'-\varepsilon}}{\binom{k+l}{\omega'}}$. If $\varepsilon = 0$, then $P_{filter} = \frac{\binom{k+l-\omega'}{\omega'}}{\binom{k+l}{\omega'}} \approx 1$ since $\omega' \ll k$ in concrete analysis of NIST-KEMs. Thus, in this

[22] For the DOOM setting, this brings a greater influence due to the fact that we need to guarantee that every list $\{L_i^j\}_{i,j}$ satisfies the size requirements.

[23] The ANNS-predicate problem in depth-1 requires that the length of the input vector is half that of the output vector. As in [19], a hybrid method that introduces this trick into the initial stage will not affect our current results of PRO-GJN.

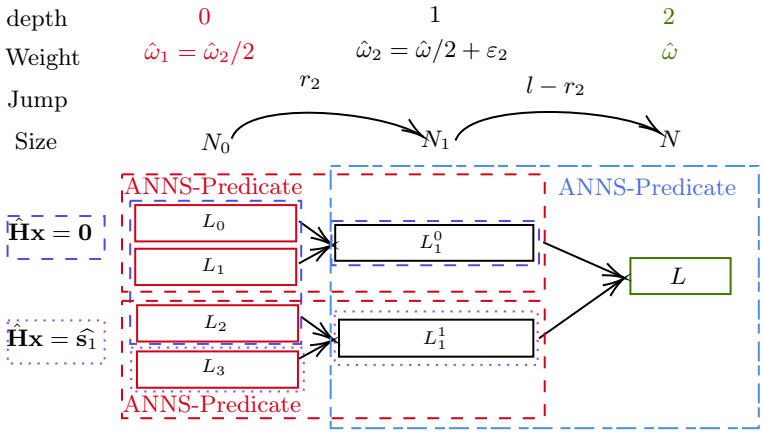

Fig. 5.1: The ANNS-Predicate problems in the BJMM framework

case, for the BJMM merging algorithm, almost all the pairs in S that pass the first step will be desired pairs and will also pass the second step.

However, in order to increase the number of "representations",[24] ε is chosen from the integers in $(0, \omega/2]$. Since ω' is very small and k is much larger than ω', $P_{filter} \approx (k+l)^{-\varepsilon}$ is very small. That is, most pairs in S will be discarded in the second step of the BJMM merging algorithm, which essentially performs a simple quadratic sieve.

Our ANNS-Predicate algorithm 3 can be seen as an extension of the BJMM merging algorithm. In particular, the simple quadratic sieve is replaced by the advanced LSF-based sieving [19,11] and the above two steps are integrated. In the following, we show the performance comparison between the BJMM merging algorithm and our ANNS-Predicate algorithm 3.

5.2 Performance comparison for the ANNS-Predicate problem

In this section, we will first show the asymptotic performance, and then present the concrete performance in the NIST-KEM case.

Asymptotic experiment. In the asymptotic setting, any parameter a is scaled by $a/(k+l)$. The list size N is set to be the expected number of solutions $\binom{k+l}{\hat{\omega}}/2^l$, where $\hat{\omega}/(k+l)$ and $l/(k+l)$ are chosen based on the optimal parameters of progressive Sieving-ISDs for the category 1 of BIKE (key recovery), BIKE (message recovery), HQC and Classic McEliece respectively. The list size N' is obtained based on Equation (3.8). The running time exponent v is set such that $T = N^v$.

[24] Please refer to [3] for the representation technique.

The asymptotic complexity of different ANNS-Predicate algorithms[25] is estimated as follows

- BJMM: $\max\left\{(\frac{N'}{2})^2 \cdot \frac{1}{2^t}, \frac{N'}{2}\right\}$
- Algorithm 3 with GJN: $N'\binom{\omega'}{d}\left(1+\frac{N'\binom{\omega'}{d}}{2^t\cdot\binom{k+l}{d}}\right)+\min\left\{2^t, \frac{N'}{2}\right\}\cdot 2^t$
- Algorithm 3 with CH: $N'\binom{\omega'}{v}\left(\binom{r}{v}^{-1}+\frac{N'\binom{\omega'}{v}}{2^r\cdot 2^t\cdot\binom{k+l}{v}}\right)+\min\left\{2^t, \frac{N'}{2}\right\}\cdot 2^t$
- Algorithm 3 with RPC: $N'\frac{\binom{\omega'}{\alpha}\binom{k+l-\omega'}{v-\alpha}}{\not{\#}^{k+l}_{\omega',v,\alpha}}\left(1+\frac{N'\binom{\omega'}{\alpha}\binom{k+l-\omega'}{v-\alpha}}{2^t\binom{k+l}{v}}\right)+\min\left\{2^t, \frac{N'}{2}\right\}2^t$

Fig. 5.2 shows the asymptotic results with variable $t/(k+l)$ that lies in $(0, t_{ub}/(k+l)]$. The values $t_{ub}, \omega/(k+l), \varepsilon/(k+l)$ are chosen based on the optimal parameter of PRO-GJN. As we can see, when t is small, LSF-based ANNS-Predicate algorithms significantly outperform the BJMM merging algorithm. In appendix I.1, we also show the asymptotic performance comparison with variable ε. All the comparison results show that LSF-based ANNS-Predicate algorithms significantly outperform the BJMM merging algorithm as ε increases.

Non-asymptotic experiment. In the aforementioned asymptotic analysis, the polynomial overhead is neglected. In order to show the concrete advantages of LSF-based ANNS-Predicate algorithms, we take the underlying ANNS-Predicate problem (depth-2 in Fig. I.1) based on the optimal parameter configuration of PRO-GJN for category 1 of the three code-based NIST-KEMs as examples. The concrete complexities shown by Fig. 5.3 are estimated based on the methods given in Sec. 4. As we can see, all the ANNS-Predicate algorithms except RPC significantly outperform BJMM. In addition, different from the asymptotic case, the deterministic GJN-LSF achieves the best performance compared with the probabilistic ones, such as CH and RPC.

[25] Since we just consider the time complexity in this section, we do not consider CH-OPT and RPC-OPT.

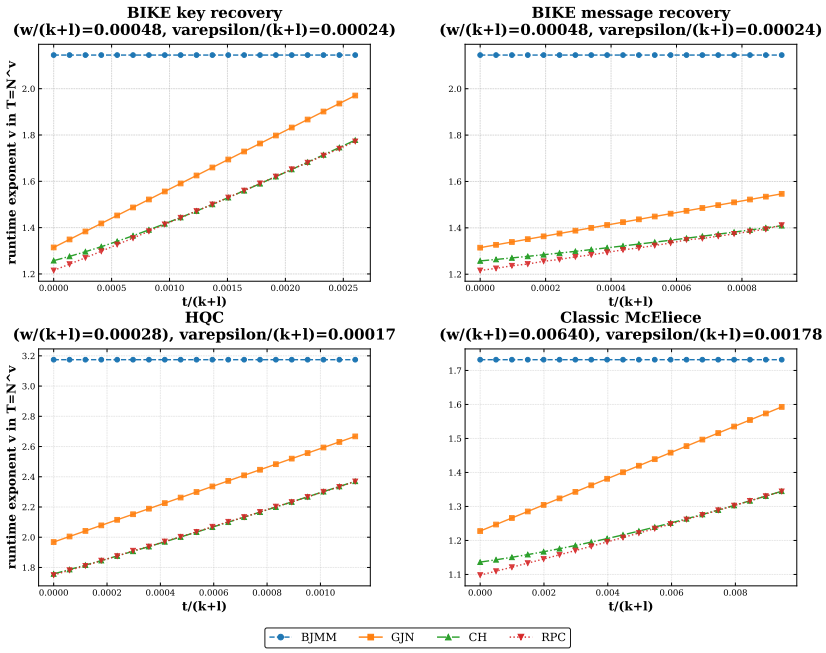

Fig. 5.2: Asymptotic time complexity of different ANNS-Predicate algorithms with variational $t/(k+l)$

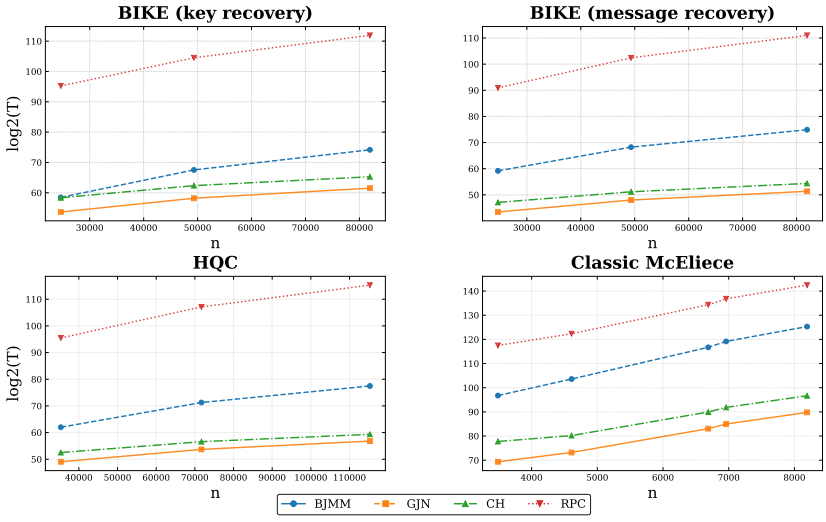

Fig. 5.3: Concrete time complexity comparison among different ANNS-Predicate algorithms

References

1. Bard, G.V.: Algorithms for Solving Linear and Polynomial Systems of Equations over Finite Fields with Applications to Cryptanalysis. Ph.D. thesis, University of Maryland, College Park, MD, USA (2007), https://hdl.handle.net/1903/7202
2. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Krauthgamer, R. (ed.) Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016. pp. 10–24. SIAM (2016), https://doi.org/10.1137/1.9781611974331.ch2
3. Becker, A., Joux, A., May, A., Meurer, A.: Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In: Pointcheval, D., Johansson, T. (eds.) Advances in Cryptology - EUROCRYPT 2012. Lecture Notes in Computer Science, vol. 7237, pp. 520–536. Springer (2012), https://doi.org/10.1007/978-3-642-29011-4_31
4. Bernstein, D.J., Lange, T., Peters, C.: Attacking and defending the mceliece cryptosystem. In: Buchmann, J., Ding, J. (eds.) Post-Quantum Cryptography, Second International Workshop, PQCrypto 2008. Lecture Notes in Computer Science, vol. 5299, pp. 31–46. Springer (2008), https://doi.org/10.1007/978-3-540-88403-3_3
5. Bernstein, D.J., Lange, T., Peters, C.: Smaller decoding exponents: Ball-collision decoding. In: Rogaway, P. (ed.) Advances in Cryptology - CRYPTO 2011. Lecture Notes in Computer Science, vol. 6841, pp. 743–760. Springer (2011), https://doi.org/10.1007/978-3-642-22792-9_42
6. Both, L., May, A.: Decoding linear codes with high error rate and its impact for LPN security. In: Lange, T., Steinwandt, R. (eds.) Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018. Lecture Notes in Computer Science, vol. 10786, pp. 25–46. Springer (2018), https://doi.org/10.1007/978-3-319-79063-3_2
7. Carrier, K., Debris-Alazard, T., Meyer-Hilfiger, C., Tillich, J.: Statistical decoding 2.0: Reducing decoding to LPN. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 13794, pp. 477–507. Springer (2022). https://doi.org/10.1007/978-3-031-22972-5_17, https://doi.org/10.1007/978-3-031-22972-5_17
8. Carrier, K., Debris-Alazard, T., Meyer-Hilfiger, C., Tillich, J.: Reduction from sparse LPN to lpn, dual attack 3.0. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI. Lecture Notes in Computer Science, vol. 14656, pp. 286–315. Springer (2024). https://doi.org/10.1007/978-3-031-58754-2_11, https://doi.org/10.1007/978-3-031-58754-2_11
9. Carrier, K., Meyer-Hilfiger, C., Shen, Y., Tillich, J.: Assessing the impact of a variant of matzov's dual attack on kyber. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I. Lecture Notes in Computer Science, vol. 16000, pp. 444–476. Springer (2025). https://doi.org/10.1007/978-3-032-01855-7_15, https://doi.org/10.1007/978-3-032-01855-7_15

10. Devadas, S., Ren, L., Xiao, H.: On iterative collision search for LPN and subset sum. In: Kalai, Y., Reyzin, L. (eds.) Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part II. Lecture Notes in Computer Science, vol. 10678, pp. 729–746. Springer (2017), https://doi.org/10.1007/978-3-319-70503-3_24
11. Ducas, L., Esser, A., Etinski, S., Kirshanova, E.: Asymptotics and improvements of sieving for codes. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI. Lecture Notes in Computer Science, vol. 14656, pp. 151–180. Springer (2024), https://doi.org/10.1007/978-3-031-58754-2_6
12. Dumer, I.: On minimum distance decoding of linear codes. In: Proc. 5th Joint Soviet-Swedish Int. Workshop Inform. Theory. pp. 50–52 (1991)
13. Engelberts, L., Etinski, S., Loyer, J.: Quantum sieving for code-based cryptanalysis and its limitations for ISD. CoRR **abs/2408.16458** (2024), https://doi.org/10.48550/arXiv.2408.16458
14. Esser, A.: Revisiting nearest-neighbor-based information set decoding. In: Quaglia, E.A. (ed.) Cryptography and Coding - 19th IMA International Conference, IMACC 2023, London, UK, December 12-14, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14421, pp. 34–54. Springer (2023). https://doi.org/10.1007/978-3-031-47818-5_3, https://doi.org/10.1007/978-3-031-47818-5_3
15. Esser, A., Bellini, E.: Syndrome decoding estimator. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) Public-Key Cryptography - PKC 2022. Lecture Notes in Computer Science, vol. 13177, pp. 112–141. Springer (2022), https://doi.org/10.1007/978-3-030-97121-2_5
16. Esser, A., Heuer, F., Kübler, R., May, A., Sohler, C.: Dissection-bkw. In: Shacham, H., Boldyreva, A. (eds.) Advances in Cryptology - CRYPTO 2018. Lecture Notes in Computer Science, vol. 10992, pp. 638–666. Springer (2018), https://doi.org/10.1007/978-3-319-96881-0_22
17. Esser, A., Kübler, R., Zweydinger, F.: A faster algorithm for finding closest pairs in hamming metric. In: Bojanczyk, M., Chekuri, C. (eds.) 41st IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2021, December 15-17, 2021, Virtual Conference. LIPIcs, vol. 213, pp. 20:1–20:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021), https://doi.org/10.4230/LIPIcs.FSTTCS.2021.20
18. Esser, A., May, A., Zweydinger, F.: Mceliece needs a break - solving mceliece-1284 and quasi-cyclic-2918 with modern ISD. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology - EUROCRYPT 2022. Lecture Notes in Computer Science, vol. 13277, pp. 433–457. Springer (2022), https://doi.org/10.1007/978-3-031-07082-2_16
19. Guo, Q., Johansson, T., Nguyen, V.: A new sieving-style information-set decoding algorithm. IEEE Trans. Inf. Theory **70**(11), 8303–8319 (2024), https://doi.org/10.1109/TIT.2024.3457150
20. May, A., Meurer, A., Thomae, E.: Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In: Lee, D.H., Wang, X. (eds.) Advances in Cryptology - ASIACRYPT 2011. Lecture Notes in Computer Science, vol. 7073, pp. 107–124. Springer (2011), https://doi.org/10.1007/978-3-642-25385-0_6
21. May, A., Ozerov, I.: On computing nearest neighbors with applications to decoding of binary linear codes. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology - EUROCRYPT 2015. Lecture Notes in Computer Science, vol. 9056, pp. 203–228. Springer (2015), https://doi.org/10.1007/978-3-662-46800-5_9

22. Minder, L., Sinclair, A.: The extended k-tree algorithm. J. Cryptol. **25**(2), 349–382 (2012). https://doi.org/10.1007/S00145-011-9097-Y, https://doi.org/10.1007/s00145-011-9097-y
23. Prange, E.: The use of information sets in decoding cyclic codes. IRE Trans. Inf. Theory **8**(5), 5–9 (1962), https://doi.org/10.1109/TIT.1962.1057777
24. Sendrier, N.: Decoding one out of many. In: Yang, B. (ed.) Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011. Lecture Notes in Computer Science, vol. 7071, pp. 51–67. Springer (2011), https://doi.org/10.1007/978-3-642-25405-5_4
25. Stern, J.: A method for finding codewords of small weight. In: Cohen, G.D., Wolfmann, J. (eds.) Coding Theory and Applications, 3rd International Colloquium. Lecture Notes in Computer Science, vol. 388, pp. 106–113. Springer (1988), https://doi.org/10.1007/BFb0019850
26. Wagner, D.A.: A generalized birthday problem. In: Yung, M. (ed.) Advances in Cryptology - CRYPTO 2002. Lecture Notes in Computer Science, vol. 2442, pp. 288–303. Springer (2002), https://doi.org/10.1007/3-540-45708-9_19

Appendices: supplementary material

A Basics of Locality Sensitive Filter

Let $\mathcal{C}_f \subseteq \mathbb{F}_2^n$ be a set of filter vectors that divides the Hamming space into regions, whose formal definition is given by Definition A.1.

Definition A.1 (Region). *Given a filter $\mathbf{c} \in \mathcal{C}_f$ and a parameter α, the $Region_{\mathbf{c},\alpha}$ is defined as $Region_{\mathbf{c},\alpha} = \{\mathbf{x} \in \mathbb{F}_2^n : |\mathbf{x} \wedge \mathbf{c}| = \alpha\}$.*

Roughly speaking, the sum of two vectors that lie in the same region usually achieves a small Hamming weight with a high probability due to the fact that such two vectors have a large overlapping support with a fixed vector $\mathbf{c}$. Thus, the key idea of LSF is to assign all vectors in the list to the buckets (see Definition A.2) and only check the pairs of vectors in the same bucket.

Definition A.2 (Bucket). *A bucket associated to filter center $\mathbf{c}$ and list L is defined as $Bucket_{\mathbf{c},\alpha} = Region_{\mathbf{c},\alpha} \cap L$.*

Definition A.3 (The set of valid filters). *Let $\mathcal{C}_f \subseteq \mathbb{F}_2^n$ be a central set. Given an $\mathbf{x} \in \mathbb{F}_2^n$ and the parameter α, the set of valid filters is defined as $\mathcal{B}_\alpha(\mathbf{x}) := \{\mathbf{c} \in \mathcal{C}_f : |\mathbf{x} \wedge \mathbf{c}| = \alpha\}$.*

In the probability analysis of LSFs in [11], we also need the following notions. The intersection of the sphere with a region is called a spherical cap.

Definition A.4 (Spherical cap). *Given $\mathbf{c} \in \mathcal{S}_\omega^n$, integer parameters $0 \leq \alpha, \omega \leq n$, a spherical cap is defined as $\mathcal{C}_{\mathbf{c},\omega,\alpha} := \mathcal{S}_\omega^n \cap Region_{\mathbf{c},\alpha}$.*

The volume of the spherical cap $\mathcal{C}_{\mathbf{c},\omega,\alpha}$ is $\mathscr{C}_{v,\omega,\alpha}^n = \binom{v}{\alpha}\binom{n-v}{\omega-\alpha}$. Further, the part where the two spherical caps intersect is called a spherical wedge.

Definition A.5 (Spherical wedge). *Given two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{F}_2^n$ of weight ω and integer parameters $0 \leq \alpha, v \leq n$, a spherical wedge is defined as*

$$\begin{aligned}\mathcal{W}_{\mathbf{x},\mathbf{y},v,\alpha}^n &:= \mathcal{C}_{\mathbf{x},v,\alpha} \cap \mathcal{C}_{\mathbf{y},v,\alpha} = \mathcal{S}_v^n \cap Region_{\mathbf{x},\alpha} \cap Region_{\mathbf{y},\alpha} \\ &= \{\mathbf{c} \in \mathcal{S}_v^n : |\mathbf{c} \wedge \mathbf{x}| = |\mathbf{c} \wedge \mathbf{y}| = \alpha\}.\end{aligned}$$

The volume of a spherical wedge is given by the following lemma.

Lemma A.1 (Wedge volume [13, Lemma 2.5]). *Given two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{F}_2^n$ of weight ω and integer parameters $0 \leq \alpha, v \leq n$. Define $\omega^* = |\mathbf{x} \wedge \mathbf{y}|$, then we have that*

$$\mathscr{W}_{\omega,v,\alpha}^n := |\mathcal{W}_{\mathbf{x},\mathbf{y},v,\alpha}^n| = \sum_{e=\max\{0,2\alpha-v\}}^{\min\{\alpha,\omega^*\}} \binom{\omega^*}{e}\binom{\omega-\omega^*}{\alpha-e}^2\binom{n-2\omega+\omega^*}{v-2\alpha+e}.$$

B Experiments for asymmetric binary heuristic assumptions

By following [11]'s experiments and codes for heuristic verification, we set $n = 256$, $k = 236$ then $l = n-k = 20$ and the list size in layer-l to be $N_l = 4000$. Then, we can derive the list size in other layers based on equation (4.3) in our paper from layer-$(l-1)$ to layer-1. We modify [11]'s code and the experiment result is shown in Fig. B.1. We can see that the data trends from the experiment and the theoretical model are almost identical. Although there is a slight discrepancy between the experimental data and the data derived based on the theoretical model in terms of the number of collisions. As described in [11], this is likely due to the dependencies among vectors and a slight bias in the theoretical model.

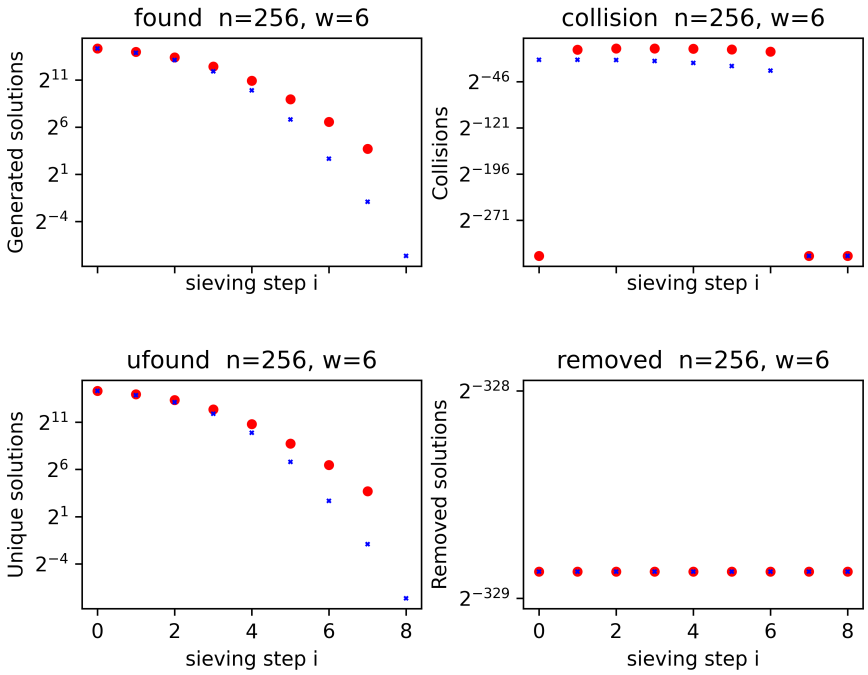

Fig. B.1: Comparison between experiment and theory (Changing the size of lists) (Red dots represent experimental data, blue dots represent the data with the theoretical models)

Furthermore, when we change the size of the list, we observe the same phenomenon as in [11], where the number of unique solutions in the experimental data exceeds the predicted value of the theoretical model.

In addition to the above experiments, we further changed the weights of the vectors in the list so that the weights of the vectors in the output list and input

list are inconsistent. We let ω' (ω, resp.) represent the weight of the vectors in the input (output, resp.) list. We keep the current sieving step at 1 and let the range of ω be $[2\lfloor\omega'/2\rfloor, 2\omega']$. Given the size N_2 of the output list, we derive the size N_1 of the input list using equation (4.3). Then we can compare the experimental data with the theoretical values.

As shown by Table B.1, the deviation between experimental data and the predicted values based on the theoretical model is also very small.

Table B.1: Comparison between experiment and theory ($\omega' = 5$ and $N_2 = 4000$)

ω'	N_1	Generated solutions		Unique Solutions		Collisions		Removed solutions ($n = 6960$)	
		Exp	Theory	Exp	Theory	Exp	Theory	Exp	Theory
4	23218	4813	4799	4813	4799	0	0	813	799
6	2548	4750	4794	4750	4794	0	0	750	794
8	457	4790	4764	4790	4764	0	0	790	764
10	145	4684	4693	4684	4693	0	0	684	693

C Proof of Theorem 2.1

Proof. The permutation in the first step is good if and only if the vector $\mathbf{e}$ of dimension n is $\hat{\omega}$-weight in the first $k+l$ positions and $(\omega - \hat{\omega})$-weight in the remaining positions. Thus, the success probability of getting a good permutation is

$$P_1 = \frac{\binom{k+l}{\hat{\omega}}\binom{n-k-l}{\omega-\hat{\omega}}}{\binom{n}{\omega}}.$$

The probability that the solution included in the output list is

$$P_2 = \min\left\{1, 1 - \left(1 - 2^l/\binom{k+l}{\hat{\omega}}\right)^N\right\} \approx \min\left\{1, N \cdot 2^l/\binom{k+l}{\hat{\omega}}\right\}.$$

Then, executing the algorithm $1/(P_1 \cdot P_2)$ times, we can expect to obtain a good permutation. After that, the algorithm performs Gaussian elimination, runs an oracle to find $\hat{\mathbf{e}}$, and checks whether the weight of $\bar{\mathbf{e}}$ satisfies the weight assumption. This process results in a single expected running time $T_{Gauss} + T_O + T_{check}$. Since we run the algorithm a total of $1/(P_1 \cdot P_2)$ times, the total expected running time of Algorithm 1 is

$$T = (P_1 \cdot P_2)^{-1} \cdot (T_{Gauss} + T_O + T_{check}).$$

□

D Algorithm 3 with probabilistic LSFs

LSF via CH. In CH-LSF, the random subset of $\mathcal{S}_v^{k+l}$ ($v < d$) is chosen by defining a hash function $\mathcal{H} : \mathcal{S}_v^{k+l} \to [2^r]$ and setting $\mathcal{C}_f := \{\mathbf{c} \in \mathcal{S}_v^{k+l} : \mathcal{H}(\mathbf{c}) = 0\}$, where the hash function is instantiated via a random binary $[k+l, k+l-r]$ code. Then the size of $\mathcal{C}_f$ becomes $\binom{k+l}{v}/2^r$. Further, α is set to be v, so that if $\mathbf{c} \in \mathcal{C}_f$ is a valid filter of an $\mathbf{x} \in L_0'$ or L_1', then $\mathbf{x} \wedge \mathbf{c} = \mathbf{c}$ holds. More formally, if $\mathbf{c}$ is a valid filter of $\mathbf{x}$, we have

$$\mathcal{H}(\mathbf{c}) = 0 \iff \mathbf{c} \in \mathcal{C}_f \iff \mathbf{c} \in \sigma_{\mathbf{x}}(\mathcal{C}_{\mathcal{H}})$$

where $\sigma_{\mathbf{x}}(\mathcal{C}_{\mathcal{H}}) = \{\mathbf{c} \in \mathcal{C}_{\mathcal{H}} : \mathbf{c} \wedge \bar{\mathbf{x}} = \mathbf{0}\}$ (the bar notation in $\bar{\mathbf{x}}$ denotes the complement here) is a shortened code. The function **ValidFilters** can be instantiated with Prange's algorithm as in [11, Algorithm 5] and can find all valid filters in time $T_{prange}(\omega', \omega' - r) = \binom{\omega'}{v}/\binom{r}{v} \cdot T_{Gauss}(\omega', \omega' - r)$. Since the filter set is just a random subset of $\mathcal{S}_v^{k+l}$, some valid pairs may not be detected by CH-LSF. Thus, the probability $q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$ that a good pair $(\mathbf{x}, \mathbf{y})$ falls in the same bucket (defined by equation 3.1) may not be equal to 1. We need Lemma D.1 given by [11, Lemma 4.3] to estimate the value of $q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$.

Lemma D.1 ([11, Lemma 4.3]). *Let $\mathbf{x}, \mathbf{y} \in \mathcal{S}_{\omega'}^{k+l}$ satisfy $|\mathbf{x} \wedge \mathbf{y}| = d$ where $k+l$ be sufficiently large and let $\alpha, v = \Theta(k+l) < d$. Further let $\mathcal{C}_f \subset \mathcal{S}_v^{k+l}$ be a random subset and satisfy $|\mathcal{C}_f| \geq k'(k+l) \cdot \frac{\binom{k+l}{v}}{\mathscr{W}_{\omega,v,\alpha}^{k+l}}$ where $k' > 0$ and $\mathscr{W}_{\omega,v,\alpha}^{k+l}$ is defined by Lemma A.1. Then, $q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha) = \Pr[\exists \mathbf{c} \in \mathcal{C}_f : \mathbf{c} \in \mathcal{B}_\alpha(\mathbf{x}) \cap \mathcal{B}_\alpha(\mathbf{y})] \geq 1 - \exp(-k'(k+l))$.*

Corollary D.1 (Complexity of ANNS-Predicate with CH-LSF). *Let $v = \Theta(k+l) < d$, $\alpha = v$, $k' > 0$, $\mathcal{C}_f = \mathcal{S}_v^{k+l} \cap \mathcal{C}_{\mathcal{H}}$ where $\mathcal{C}_{\mathcal{H}}$ is a random binary $[k+l, k+l-r]$ code for $r := \lfloor \log \binom{d}{v} - \log(k'(k+l)) \rfloor$, the function **ValidFilters** is defined in [11, Algorithm 5]. Then, Algorithms 3 can solve the ANNS-Predicate-Problem in the expected time and expected memory*

$$T = N'\left(C_1\left(T_{Prange}(\omega', \omega' - r) + \frac{\binom{\omega'}{v}}{2^r}(\omega' \log(k+l) + t + \bar{b})\right) + \frac{C_2 \omega' \log(k+l) N' \binom{\omega'}{v}^2}{2^r 2^t \cdot \binom{k+l}{v}}\right) + N'(\omega' - 1)t + T_{table}$$

$$M = N'\left((\omega' log(k+l) + t + \bar{b}) + \frac{C_3 \binom{\omega'}{v}(\omega' log(k+l) + t + \bar{b})}{2^r}\right) + C_4 N \omega \log(k+l) + 2^{t+1} \cdot t$$

with probability $p \geq (1 - \exp(-\bar{N} q \delta^2/2))^2$ and $T_{table} = \min\{C_5 N', 2^t\} \cdot t$, where $\bar{N} \approx C_6 N'^2$, $\delta = 1 - N/(2\bar{N}q)$ and $q = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'} \cdot 1/2^t \cdot q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$. In particular, the values $(C_1, \bar{b}, C_2, C_3, C_4, C_5, C_6)$ are set to be $(1, 0, 2, 1, 1, 1, 1/2)$ for ANNS-Predicate-1, $(1, 1, 3/4, 1, 1, 1/2, 1/4)$ for ANNS-Predicate-2 and $(1/2, 1, 1/4, 1/2, 1/2, 1/2, 1/4)$ for ANNS-Predicate-3.

Proof. The time to find valid filters is $T_{\text{ValidFilters}} = T_{\text{Prange}}(\omega', \omega'-r) = \binom{\omega'}{v}/\binom{r}{v} \cdot T_{Gauss}(\omega', \omega'-r)$. For any $\mathbf{x}$, the expected size of the valid filter set is

$$E[|\mathcal{B}_\alpha(\mathbf{x})|] = E[|\{\mathbf{c} \in \sigma_\alpha(\mathcal{C}_\mathcal{H}) : |\mathbf{c}| = v\}|] = \binom{\omega'}{v}/2^r.$$

Since $\mathcal{C}_f := \mathcal{S}_v^{k+l} \cap \mathcal{C}_\mathcal{H} = \{\mathbf{c} \in \mathcal{C}_\mathcal{H} : |\mathbf{c}| = v\}$, we have $E[|\mathcal{C}_f|] = \binom{k+l}{v}/2^r$. Then, the expected size of the bucket is $E[|\text{Bucket}_\alpha(\mathbf{c})|] = N' \cdot E[|\mathcal{B}_\alpha(\mathbf{x})|]/|\mathcal{C}_f| = N' \cdot \binom{\omega'}{v}/\binom{k+l}{v}$. By substituting these quantities in (3.2) and (3.3), we can get the expected time and expected memory of Algorithm 3 via CH-LSF. □

In [11], the authors also proposed a memory-optimized version of CH-LSF using a simple repetition technique. In this paper, we call this version CH-LSF-OPT. The idea of CH-LSF-OPT is that if we execute the algorithm on a filter set with reduced size $|\mathcal{C}_f'| = |\mathcal{C}_f|/2^{r'}$. Then, the reduced success probability can be compensated by repeating the algorithm $2^{r'}$ times, but the expected memory of the algorithm can be improved by a factor of $2^{r'}$. Thus, we can easily obtain the following corollary.

Corollary D.2 (Complexity of ANNS-Predicate with CH-LSF-OPT). *Let $v = \Theta(k+l) < d$, $\alpha = v$, $k' > 0$, $\mathcal{C}_f := \mathcal{S}_v^{k+l} \cap \mathcal{C}_\mathcal{H}$ where $\mathcal{C}_\mathcal{H}$ is a random binary $[k+l, k+l-r]$ code for $\lfloor \log \binom{d}{v} - \log k'(k+l) \rfloor \le r \le \lfloor \log \binom{\omega'}{v} - \log k'(k+l) \rfloor$, the function **ValidFilters** is defined in [11, Algorithm 5]. Define $r' = r - \lfloor \log \binom{d}{v} - \log k'(k+l) \rfloor$ and then $2^{r'}$ sequential repetitions of Algorithms 3 can solve the ANNS-Predicate-Problem in the expected time and expected memory*

$$\begin{aligned} T =& T_{table} + 2^{r'} N'(\omega'-1)t + 2^{r'} N' \Bigg(C_1 \Big(T_{Prange}(\omega', \omega'-r) + \\ & \frac{\binom{\omega'}{v}}{2^r} (\omega' \log(k+l) + t + \bar{b}) \Big) + \frac{C_2 \omega' \log(k+l) N' \binom{\omega'}{v}^2}{2^r \cdot 2^t \cdot \binom{k+l}{v}} \Bigg), \\ M =& N' \Bigg((\omega' \log(k+l) + t + \bar{b}) + \frac{C_3 \binom{\omega'}{v} (\omega' \log(k+l) + t + \bar{b})}{2^r} \Bigg) + \\ & C_4 N \omega \log(k+l) + 2^{t+1} t, \end{aligned}$$

with expected probability $p \ge (1 - \exp(-\bar{N} q \delta^2/2))^2$ and $T_{table} = \min\{C_5 N', 2^t\} \cdot t$, where $\bar{N} \approx C_6 N'^2$, $\delta = 1 - N/(2\bar{N}q)$ and $q = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'} 1/2^t q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$. In particular, the values $(C_1, \bar{b}, C_2, C_3, C_4, C_5, C_6)$ are set to be $(1, 0, 2, 1, 1, 1, 1/2)$ for ANNS-Predicate-1, $(1, 1, 3/4, 1, 1, 1/2, 1/4)$ for ANNS-Predicate-2 and $(1/2, 1, 1/4, 1/2, 1/2, 1/2, 1/4)$ for ANNS-Predicate-3.

LSF via RPC. In [11], the authors also proposed another probabilistic LSF called RPC-LSF. Different from CH-LSF, the random filter set $\mathcal{C}_f \subset \mathcal{S}_v^{k+l}$ is defined by random product code (see Definition D.1).

Definition D.1 (Random Product Code [11, Definition 4.3]). *An element C is called RPC if it is drawn uniformly at random from the set*

$$R_{n,v,m} := \left\{ C = C^{(1)} \times \cdots \times C^{(m)} : C^{(i)} \subseteq \mathcal{S}_{v/m}^{(k+l)/m} \text{ with size } |C^{(i)}| = \sqrt[m]{|C|} \right\}.$$

In particular, the filter set is chosen by $\mathcal{C}_f = \mathcal{C}_f^{(1)} \times \cdots \times \mathcal{C}_f^{(m)} \in R_{k+l,v,m}$. Accordingly, the set of valid filters for $\mathbf{x} = (\mathbf{x}_1, ..., \mathbf{x}_m) \in L'$ is defined by

$$\mathcal{B}_\alpha^{(m)}(\mathbf{x}) = \{ \mathbf{c} \in \mathcal{C}_f : \forall i, |\mathbf{x}_i \wedge \mathbf{c}_i| = \alpha/m \}. \tag{D.1}$$

The function **ValidFilters** is defined in [11, Algorithm 6] and can return the set $\mathcal{B}_\alpha^{(m)}(\mathbf{x})$ in time $m \cdot \sqrt[m]{|\mathcal{C}_f|} + |\mathcal{B}_\alpha^{(m)}(\mathbf{x})|$ according to [11, Lemma 4.5].

Lemma D.2 (Amount of Filters for RPC-LSF [11, Lemma 4.8]). [26] *Let $\mathbf{x}, \mathbf{y} \in \mathcal{S}_{\omega'}^{k+l}$ satisfy $|\mathbf{x} \wedge \mathbf{y}| = d$ where $k+l$ be sufficiently large, $\alpha, v = \Theta(k+l) < d$ and $m = o(\frac{k+l}{\log(k+l)})$ be integers. Further let $C \in R_{k+l,v,m}$ be an RPC and $P = \{\pi_i\}_{i \in [k'(k+l)((k+l)/m+1)^{3m}]}$ be a set of independently random permutations on $k+l$ elements. The probability that there exists a $\pi \in P$ and a $\mathbf{c} \in C$ such that $\mathbf{c} \in \mathcal{W}_{\pi(\mathbf{x}),\pi(\mathbf{y}),v,\alpha}$ is denoted by $q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$. Then*

$$q(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha) > (1 - exp(-k'(k+l))) \cdot q_1(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha) \cdots q_m(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$$

where

$$q_i(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha) = 1 - \left(1 - \mathcal{W}_{\omega'/m,v/m,\alpha/m}^{(k+l)/m} \Big/ \binom{(k+l)/m}{v/m} \right)^{|C^{(i)}|}. \tag{D.2}$$

Remark D.1. Note that given the values of k, l, m, v, α and ω, we can derive the value of $q_i(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha)$ according to Equation (D.2). Denote $q_i(\mathbf{x}, \mathbf{y}, \mathcal{C}_f, \alpha) = 1 - 2^{-k_q}$ ($k_q \geq 0$). Then, we can compute the value $|\mathcal{C}_f|$ via

$$|\mathcal{C}_f| = \left(\frac{-k_q}{\log\left(1 - \left(\mathcal{W}_{\omega'/m,v/m,\alpha/m}^{(k+l)/m} \Big/ \binom{(k+l)/m}{v/m}\right)\right)} \right)^m. \tag{D.3}$$

Corollary D.3 (Complexity of ANNS-Predicate with RPC). *Let $k, l, v, \alpha, d, \omega, \omega' \in \mathbb{N}$ such that $d = (2\omega' - \omega)/2$ and $m \leq \alpha \leq v < d$. Further let $\mathcal{C}_f \in R_{k+l,v,m}$ be an RPC and $P = \{\pi_i\}_{i \in [k'(k+l)((k+l)/m+1)^{3m}]}$ be a set of independently random permutations on $k+l$ elements. The function **ValidFilters** is instantiated by [11, Algorithm 6]. Let $\mathscr{C}_{\omega'/m,v/m,\alpha/m}^{(k+l)/m} = \binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}$ and $\bar{K} = k'(k+l)(\frac{k+l}{m}+1)^{3m}$. Then, the iterative execution of Algorithms 3 $\bar{K}$*

[26] In [11], the value q_i is estimated asymptotically. In this paper, we focus on the concrete bit security of code-based cryptographic algorithms. Thus, we leave the accurate q_i in [11, Lemma 4.8].

times, where the input list is permuted in each iteration using the permutation in P, can solve the ANNS-Predicate-Problem in the expected time

$$T = \bar{K}N'\Bigg(C_1\Big(m\sqrt[m]{|\mathcal{C}_f|} + |\mathcal{C}_f|\left(\frac{\mathscr{C}^{(k+l)/m}_{\omega'/m,v/m,\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m + |\mathcal{C}_f|\cdot\left(\frac{\binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m(\omega'\log(k+l)+t+\bar{b})\Big) + (\omega'-1)\cdot t + \frac{C_2\omega'\log(k+l)N'|\mathcal{C}_f|(\mathscr{C}^{(k+l)/m}_{\omega'/m,v/m,\alpha/m})^{2m}}{2^t\binom{(k+l)/m}{v/m}^{2m}}\Bigg) + T_{table}$$

and expected memory $M = C_4\cdot N\cdot\omega\log(k+l) + 2^{t+1}\cdot t +$

$$N'\cdot\left((\omega'\log(k+l)+t+b)+C_3|\mathcal{C}_f|\cdot\left(\frac{\mathscr{C}^{(k+l)/m}_{\omega'/m,v/m,\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m\cdot(\omega'\log(k+l)+t+\bar{b})\right)$$

with probability $p \geq (1-\exp(-\bar{N}q\delta^2/2))^2$ *and* $T_{table} = \min\{C_5N', 2^t\}\cdot t$, *where* $\bar{N} \approx C_6N'^2$, $\delta = 1-N/(2\bar{N}q)$ *and* $q = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'}\cdot 1/2^t\cdot q(\mathbf{x},\mathbf{y},\mathcal{C}_f,\alpha)$. *In particular, the values* $(C_1,\bar{b},C_2,C_3,C_4,C_5,C_6)$ *are set to be* $(1,0,2,1,1,1,1/2)$ *for ANNS-Predicate-1,* $(1,1,3/4,1,1,1/2,1/4)$ *for ANNS-Predicate-2 and* $(1/2,1,1/4,1/2,1/2,1/2,1/4)$ *for ANNS-Predicate-3.*

Proof. Note that the expected size of the valid filters set in RPC-LSF is

$$E[|\mathcal{B}^{(m)}_\alpha(\mathbf{x})|] = \sum_{\mathbf{c}\in\mathcal{C}_f}\Pr\left[\forall i, |\mathbf{c}_i\wedge\mathbf{x}_i| = \frac{\alpha}{m}\right] = |\mathcal{C}_f|\cdot\left(\frac{\binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m.$$

Because of the instantiation of the function **ValidFilters**, the expected time $T_{\text{ValidFilters}} =$

$$m\sqrt[m]{|\mathcal{C}_f|} + |\mathcal{B}^{(m)}_\alpha(\mathbf{x})| = m\sqrt[m]{|\mathcal{C}_f|} + |\mathcal{C}_f|\cdot\left(\frac{\binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m.$$

The expected size of the bucket is

$$E[|\text{Bucket}_\alpha(\mathbf{c})|] = \frac{N'\cdot E[|\mathcal{B}^{(m)}_\alpha(\mathbf{x})|]}{|\mathcal{C}_f|} = N'\cdot\left(\frac{\binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m.$$

Collecting all the expectations and applying Theorems 3.2, we can get the expected time and expected memory of Algorithm 3 via RPC-LSF. □

As in the CH-LSF case, the repetition technique that chooses a smaller set of filters and repeats Algorithms 3 for many of these smaller sets can also be

used to optimize the memory complexity of RPC-LSF. In the following, such an optimized version is called RPC-LSF-OPT, whose complexity is presented by Corollary D.4. In particular, the repetition number $\bar{D}$ in RPC-LSF-OPT is chosen such that the expected number of buckets for each $\mathbf{x}$ is 1.

Corollary D.4 (Complexity of ANNS-Predicate with RPC-LSF-OPT). *Let $k, l, v, \alpha, d, \omega, \omega' \in \mathbb{N}$ such that $d = (2\omega' - \omega)/2$ and $m \leq \alpha \leq v < d$. Further let $\mathcal{C}'_f \in R_{k+l,v,m}$ be an RPC and $P = \{\pi_i\}_{i\in[k'(k+l)((k+l)/m+1)^{3m}]}$ be a set of independently random permutations on $k+l$ elements. The function **ValidFilters** is instantiated by [11, Algorithm 6]. Let $\mathscr{C}^{(k+l)/m}_{\omega'/m,v/m,\alpha/m} = \binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}$, $\bar{K} = k'(k+l)(\frac{k+l}{m}+1)^{3m}$, $|\mathcal{C}'_f| = \frac{|\mathcal{C}_f|}{\bar{D}}$, where $\bar{D} = |\mathcal{C}_f|\left(\frac{\mathscr{C}^{(k+l)/m}_{\omega'/m,v/m,\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m$ and $|\mathcal{C}_f|$ is defined by Equation (D.3). Then $\bar{D}$ sequential repetitions of the $\bar{K}$ iterative execution of Algorithms 3 (the input list is permuted in each iteration using the permutation in P) can solve the ANNS-Predicate-Problem in the expected time*

$$T = T_{table} + \bar{K}\bar{D}N'\left(C_1\left(m\sqrt[m]{\frac{|\mathcal{C}_f|}{\bar{D}}} + 1 + (\omega'\log(k+l) + t + \bar{b})\right)\right.$$
$$\left. + (\omega'-1)t + \frac{C_2\omega'\log(k+l)N'(\mathscr{C}^{(k+l)/m}_{\omega'/m,v/m,\alpha/m})^m}{2^t\binom{(k+l)/m}{v/m}^m}\right)$$

and expected memory

$$M = N'\cdot\left(C_3\omega'\log(k+l) + 2t + 2\bar{b}\right) + C_4\cdot N\cdot\omega\log(k+l) + 2^{t+1}\cdot t$$

with probability $p \geq (1-\exp(-\bar{N}q\delta^2/2))^2$ and $T_{table} = \min\{C_5N', 2^t\}\cdot t$, where $\bar{N} \approx C_6N'^2$, $\delta = 1 - N/(2\bar{N}q)$ and $q = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'-d}/\binom{k+l}{\omega'}\cdot 1/2^t\cdot q(\mathbf{x},\mathbf{y},\mathcal{C}_f,\alpha)$. In particular, the values $(C_1,\bar{b},C_2,C_3,C_4,C_5,C_6)$ are set to be $(1,0,2,2,1,1,1/2)$ for ANNS-Predicate-1, $(1,1,3/4,2,1,1/2,1/4)$ for ANNS-Predicate-2 and $(1/2,1,1/4,3/2,1/2,1/2,1/4)$ for ANNS-Predicate-3.

Proof. Due to the setting of $\bar{D}$, we have the expected size of the valid filters set in RPC-LSF-OPT

$$E[|\mathcal{B}^{(m)}_\alpha(\mathbf{x})|] = \sum_{\mathbf{c}\in\mathcal{C}'_f}\Pr\left[\forall i, |\mathbf{c}_i\wedge\mathbf{x}_i| = \frac{\alpha}{m}\right] = |\mathcal{C}'_f|\cdot\left(\frac{\binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m = 1.$$

Thus, we can obtain the expected memory.

Because of the instantiation of the function **ValidFilters** , the expected time $T_{\text{ValidFilters}} = m\sqrt[m]{|\mathcal{C}'_f|} + |\mathcal{B}^{(m)}_\alpha(\mathbf{x})| = m\sqrt[m]{|\mathcal{C}_f|/\bar{D}} + 1$. The expected size of the bucket is

$$E[|\text{Bucket}_\alpha(\mathbf{c})|] = \frac{N'\cdot E[|\mathcal{B}^{(m)}_\alpha(\mathbf{x})|]}{|\mathcal{C}'_f|} = N'\cdot\left(\frac{\binom{\omega'/m}{\alpha/m}\binom{(k+l)/m-\omega'/m}{v/m-\alpha/m}}{\binom{(k+l)/m}{v/m}}\right)^m.$$

Collecting all the expectations and applying Theorems 3.2, we can get the expected time and memory as desired. □

E The whole complexity of progressive sieving

Having introduced the single-layer iterative process (see Fig. E.1 for the overview of the relation between parameters), we can now calculate the whole complexity of the progressive sieving.

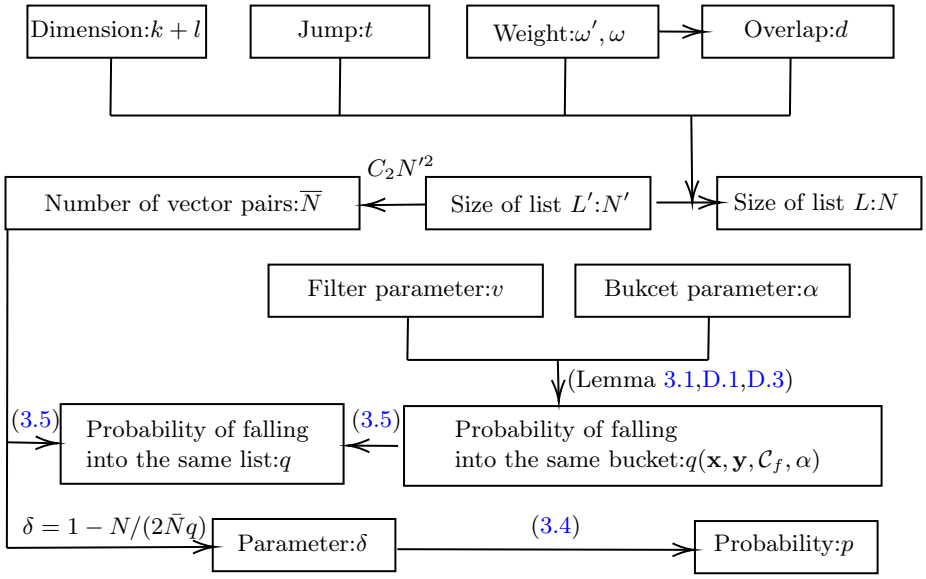

Fig. E.1: The relation between parameters for ANNS-predicate with lists L and L'.

Progressive sieve via GJN-LSF. Instantiating Algorithm 2 with GJN-LSF, we get list $L_0 \subseteq \mathcal{S}_{\omega_0}^{k+l}$ with size $|L_0| = N_0$, $\mathcal{C}_f^{(i)} := \mathcal{S}_{d_i}^{k+l}$ and $\alpha_i = d_i$ for any $1 \le i \le l'$. As in the single-layer case, thanks to the choice of $\mathcal{C}_f^{(i)}$, we have $q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}_f^{(i)}, \alpha_i) = 1$. According to Corollary 3.1, the expected time T_i, memory M_i and success probability p_i of layer-i ($1 \le i \le l'$) are given by

$$T_i = N_{i-1}\left(C_1^i\binom{\omega_{i-1}}{d_i}(1 + (\omega'_{i-1} - d_i)\log(k+l) + t_i + \bar{b}_i) + (\omega_{i-1} - 1)t_i + \binom{\omega_{i-1}}{d_i}\frac{C_2^i(\omega_{i-1} - d_i)\log(k+l)N_{i-1}\binom{\omega_{i-1}}{d_i}}{2^{t_i}\cdot\binom{k+l}{d_i}}\right) + T_{table}^i,$$

$$M_i = N_{i-1}\Big((\omega_{i-1}\log(k+l) + t_i + \bar{b}_i) + C_3^i\binom{\omega_{i-1}}{d_i}((\omega_{i-1} - d_i)\log(k+l) + t_i + \bar{b}_i)\Big) + C_4^i N_i\omega_i\log(k+l) + 2^{t_i+1}t_i,$$

$p_i \geq (1-\exp(-\bar{N}_i q_i \delta_i/2))^2$, where $T^i_{table} = \min\{C^i_5 N_i, 2^{t_i}\} \cdot t_i$, $\bar{N}_i \approx C^i_6 N^2_{i-1}$, $\delta_i = 1 - N_i/(2\bar{N}_i q_i)$ and $q_i = \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i}/\binom{k+l}{\omega_{i-1}} \cdot 1/2^{t_i}$. In particular, the values $(C^i_1, \bar{b}_i, C^i_2, C^i_3, C^i_4, C^i_5, C^i_6)$ are set to be $(1,0,2,1,1,1,1/2)$ for $i=1$, $(1,1,3/4,1,1,1/2,1/4)$ for $2 \leq i \leq l'-1$ and $(1/2,1,1/4,1/2,1/2,1/2,1/4)$ for $i = l'$. Plugging these quantities in Theorem 3.1, we can get the total running time, memory and success probability of the progressive sieve via GJN-LSF.

Progressive sieve via CH-LSF. Instantiating Algorithm 2 with CH-LSF, we set list $L_0 \subseteq \mathcal{S}^{k+l}_{\omega_0}$ with size $|L_0| = N_0$ and set $v_i < d_i$, $\alpha_i = v_i$, $k' > 0$, $\mathcal{C}^{(i)}_f := \mathcal{S}^{k+l}_{v_i} \cap \mathcal{C}_{\mathcal{H}_i}$ where $\mathcal{C}_{\mathcal{H}_i}$ is a random binary $[k+l, k+l-r_i]$ code for $r_i := \left\lfloor \log\binom{d_{i-1}}{v_i} - \log(k'(k+l)) \right\rfloor$ of layer-i $(1 \leq i \leq l')$. The function **ValidFilters** is defined by [11, Algorithm 5]. Then, according to Corollary D.1, the expected time T_i, expected memory M_i and success probability p_i of layer-i $(1 \leq i \leq l')$ are given by

$$
\begin{aligned}
T_i =& N_{i-1}\Bigg(C^i_1\Big(T^{(i)}_{\text{Prange}}(\omega_{i-1}, \omega_{i-1}-r_i) + \frac{\binom{\omega_{i-1}}{v_i}}{2^{r_i}}(\omega_{i-1}\log(k+l) + t_i + \bar{b}_i)\Big) \\
&+ (\omega_{i-1}-1)t_i + \frac{C^i_2 \omega_{i-1}\log(k+l) N_{i-1}\binom{\omega_{i-1}}{v_i}^2}{2^{r_i}\cdot 2^{t_i}\cdot\binom{k+l}{v_i}}\Bigg) + T^i_{table}, \\
M_i =& N_{i-1}\left((\omega_{i-1}\log(k+l) + t_i + \bar{b}_i) + \frac{C^i_3\binom{\omega_{i-1}}{v_i}(\omega_{i-1}\log(k+l)+t_i+\bar{b}_i)}{2^{r_i}}\right) \\
&+ C^i_4 \cdot N_i \cdot \omega_i \log(k+l) + 2^{t_i+1}\cdot t_i \\
p_i \geq& (1-\exp(-\bar{N}_i q_i \delta_i/2))^2
\end{aligned}
$$

where $T^i_{table} = \min\{C^i_5 N_i, 2^{t_i}\} \cdot t_i$, $\bar{N}_i \approx C^i_6 N^2_{i-1}$, $\delta_i = 1 - N_i/(2\bar{N}_i q_i)$ and $q_i = \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i}/\binom{k+l}{\omega_{i-1}} \cdot 1/2^{t_i} \cdot q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}^{(i)}_f, \alpha_i)$. In particular, the values $(C^i_1, \bar{b}_i, C^i_2, C^i_3, C^i_4, C^i_5, C^i_6)$ are set to be $(1,0,2,1,1,1,1/2)$ for $i=1$, $(1,1,3/4,1,1,1/2,1/4)$ for $2 \leq i \leq l'-1$ and $(1/2,1,1/4,1/2,1/2,1/2,1/4)$ for $i = l'$.

Plugging these quantities in Theorem 3.1, we can get the total running time, memory and success probability of the progressive sieve via CH-LSF.

Progressive sieve via CH-LSF-OPT. The progressive sieve based on CH-LSF can be optimized using the repetition technique, which is called CH-LSF-OPT. When instantiating Algorithm 2 with CH-LSF-OPT, we also set $v_i < d_i$, $\alpha_i = v_i$, $k' > 0$, $\mathcal{C}^{(i)}_f := \mathcal{S}^{k+l}_{v_i} \cap \mathcal{C}_{\mathcal{H}_i}$. But, different from the CH-LSF case, $\mathcal{C}_{\mathcal{H}_i}$ in CH-LSF-OPT is a random binary $[k+l, k+l-r_i]$ code for $\lfloor \log\binom{d_i}{v_i} - \log k'(k+l) \rfloor \leq r_i \leq \lfloor \log\binom{\omega_{i-1}}{v_i} - \log k'(k+l)\rfloor$. Define $r'_i = r_i - \lfloor \log\binom{d_i}{v_i} - \log k'(k+l)\rfloor$. Then, $2^{r'_i}$ sequential repetitions of Algorithms 3 can reduce the expected memory in each layer by a factor of $2^{r'_i}$. According to Corollary D.2, the expected time T_i, expected memory M_i and success probability p_i of layer-i $(1 \leq i \leq l')$ are given

by

$$
\begin{aligned}
T_i =& T^i_{table} + 2^{r'_i} N_{i-1} \cdot \Bigg(C^i_1 \Big(T^{(i)}_{\text{Prange}}(\omega_{i-1}, \omega_{i-1} - r_i) + \frac{\binom{\omega_{i-1}}{v_i}}{2^{r_i}} (\omega'_{i-1} \log(k+l) + t_i \\
&+ \bar{b}_i) \Big) + (\omega_{i-1} - 1) \cdot t_i + \frac{C^i_2 \cdot \omega_{i-1} \log(k+l) \cdot N_{i-1} \binom{\omega_{i-1}}{v_i}^2}{2^{r_i} \cdot 2^{t_i} \cdot \binom{k+l}{v_i}} \Bigg), \\
M_i =& N_{i-1} \cdot \left((\omega_{i-1} \log(k+l) + t_i + \bar{b}_i) + \frac{C^i_3 \binom{\omega_{i-1}}{v_i} (\omega_{i-1} \log(k+l) + t_i + \bar{b}_i)}{2^{r_i}} \right) \\
&+ C^i_4 \cdot N_i \cdot \omega_i \log(k+l) + 2^{t_i+1} \cdot t_i, \\
p_i \geq& (1 - \exp(-\bar{N}_i q_i \delta_i / 2))^2
\end{aligned}
$$

where $T^i_{table} = \min\left\{C^i_5 N_i, 2^{t_i}\right\} \cdot t_i$, $\bar{N}_i \approx C^i_6 N^2_{i-1}$, $\delta_i = 1 - N_i/(2\bar{N}_i q_i)$ and $q_i = \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i} / \binom{k+l}{\omega_{i-1}} \cdot 1/2^{t_i} \cdot q(\mathbf{x}_{i-1}, \mathbf{y}_{i-1}, \mathcal{C}^{(i)}_f, \alpha_i)$. In particular, the values $(C^i_1, \bar{b}_i, C^i_2, C^i_3, C^i_4, C^i_5, C^i_6)$ are set to be $(1, 0, 2, 1, 1, 1, 1/2)$ for $i = 1$, $(1, 1, 3/4, 1, 1, 1/2, 1/4)$ for $2 \leq i \leq l' - 1$ and $(1/2, 1, 1/4, 1/2, 1/2, 1/2, 1/4)$ for $i = l'$. Plugging these quantities in Theorem 3.1, we can get the total running time, memory and success probability of the progressive sieve via CH-LSF-OPT.

Progressive sieve via RPC-LSF. Instantiating Algorithm 2 with RPC-LSF, we set list $L_0 \subseteq \mathcal{S}^{k+l}_{\omega_0}$ with size $|L_0| = N_0$ and set $m_i \leq \alpha_i \leq v_i < d_i$, $\mathcal{C}^{(i)}_f \in R_{k+l,v_i,m_i}$ be an RPC and $P = \{\pi_i\}_{i \in [k'(k+l)\left((k+l)/m_i+1\right)^{3m_i}]}$ be a set of independently random permutations on $k+l$ elements.

Given $k_q > 0$, according to Equation (D.3), we have

$$
|\mathcal{C}^{(i)}_f| = \left(\frac{-k_q}{\log\left(1 - \left(\mathscr{W}^{(k+l)/m_i}_{\omega_{i-1}/m_i, v_i/m_i, \alpha_i/m_i} \Big/ \binom{(k+l)/m_i}{v_i/m_i} \right)\right)} \right)^{m_i}. \tag{E.1}
$$

The function **ValidFilters** is defined by [11, Algorithm 6]. Let $\mathscr{C}^{(k+l)/m_i}_{\omega_{i-1}/m_i, v_i/m_i, \alpha_i/m_i} = \binom{\omega_{i-1}/m_i}{\alpha_i/m_i}\binom{(k+l)/m_i - \omega_{i-1}/m_i}{v_i/m_i - \alpha_i/m_i}$ and $\bar{K}_i = k'(k+l)(\frac{k+l}{m_i} + 1)^{3m_i}$. According to Corollary D.3, the expected time T_i, expected memory M_i and success probability p_i of layer-i $(1 \leq i \leq l')$ are given by $T_i = T^i_{table} +$

$$
\begin{aligned}
&\bar{K}_i N_{i-1} \Bigg(C^i_1 \Big(m_i \sqrt[m_i]{|\mathcal{C}^{(i)}_f|} + |\mathcal{C}^{(i)}_f| \left(\frac{\mathscr{C}^{(k+l)/m_i}_{\omega_{i-1}/m_i, v_i/m_i, \alpha_i/m_i}}{\binom{(k+l)/m_i}{v_i/m_i}} \right)^{m_i} \\
&+ |\mathcal{C}^{(i)}_f| \left(\frac{\mathscr{C}^{(k+l)/m_i}_{\omega_{i-1}/m_i, v_i/m_i, \alpha_i/m_i}}{\binom{(k+l)/m_i}{v_i/m_i}} \right)^{m_i} (\omega_{i-1} \log(k+l) + t_i + \bar{b}_i) \Big) \\
&+ \frac{C^i_2 \omega_{i-1} \log(k+l) N_{i-1} |\mathcal{C}^{(i)}_f| (\mathscr{C}^{(k+l)/m_i}_{\omega_{i-1}/m_i, v_i/m_i, \alpha_i/m_i})^{2m_i}}{2^{t_i} \binom{(k+l)/m_i}{v_i/m_i}^{2m_i}} + (\omega_{i-1} - 1) \cdot t_i \Bigg),
\end{aligned}
$$

$$M_i = N_{i-1}\Bigg((\omega_{i-1}\log(k+l)+t_i+\bar{b}_i)+C_3^i|\mathcal{C}_f^{(i)}|\left(\frac{\mathscr{C}^{(k+l)/m_i}_{\omega_{i-1}/m_i,v_i/m_i,\alpha_i/m_i}}{\binom{(k+l)/m_i}{v_i/m_i}}\right)^{m_i}$$

$$(\omega_{i-1}\log(k+l)+t_i+\bar{b}_i)\Bigg)+C_4^i\cdot N_i\omega_i\log(k+l)+2^{t_i+1}\cdot t_i,$$

$$p_i \geq (1-\exp(-\bar{N}_i q_i\delta_i/2))^2,$$

where $T^i_{table} = \min\{C_5^i N_i, 2^{t_i}\}\cdot t_i$, $\bar{N}_i \approx C_6^i N_{i-1}^2$, $\delta_i = 1 - N_i/(2\bar{N}_i q_i)$ and $q_i = \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i}/\binom{k+l}{\omega_{i-1}}\cdot 1/2^{t_i}\cdot q(\mathbf{x}_{i-1},\mathbf{y}_{i-1},\mathcal{C}_f^{(i)},\alpha_i)$. In particular, the values $(C_1^i,\bar{b}_i,C_2^i,C_3^i,C_4^i,C_5^i,C_6^i)$ are set to be $(1,0,2,1,1,1,1/2)$ for $i=1$, $(1,1,3/4,1,1,1/2,1/4)$ for $2\leq i\leq l'-1$ and $(1/2,1,1/4,1/2,1/2,1/2,1/4)$ for $i=l'$. Plugging these quantities in Theorem 3.1, we can get the total running time, memory and success probability of the progressive sieve via RPC-LSF.

Progressive sieve via RPC-LSF-OPT. We can also save memory through repetitions for the progressive sieve via RPC-LSF and get the progressive sieve via RPC-LSF-OPT. Unlike the RPC-LSF case, the size of filter set is set to be $|(\mathcal{C}_f')^{(i)}| = \frac{|\mathcal{C}_f^{(i)}|}{\bar{D}_i}$ where $\bar{D}_i = |\mathcal{C}_f^{(i)}|\left(\frac{\mathscr{C}^{(k+l)/m_i}_{\omega_{i-1}/m_i,v_i/m_i,\alpha_i/m_i}}{\binom{(k+l)/m_i}{v_i/m_i}}\right)^{m_i}$ and $|\mathcal{C}_f^{(i)}|$ (the size of filter set in RPC-LSF) is defined in (E.1). According to Corollary D.4, the success probability p_i is the same as the setting in the RPC-LSF case, but the expected time T_i and expected memory M_i are updated by $T_i = T^i_{table}+$

$$\bar{K}_i\bar{D}_i N_{i-1}\Bigg(C_1^i\Big(m_i\sqrt[m_i]{|\mathcal{C}_f^{(i)}|/\bar{D}_i}+1+(\omega_{i-1}log(k+l)+t_i+\bar{b}_i)\Big)$$

$$+(\omega_{i-1}-1)t_i+\frac{C_2^i\omega_{i-1}\log(k+l)N_{i-1}(\mathscr{C}^{(k+l)/m_i}_{\omega_{i-1}/m_i,v_i/m_i,\alpha_i/m_i})^{m_i}}{2^{t_i}\binom{(k+l)/m_i}{v_i/m_i}^{m_i}}\Bigg),$$

$$M_i = N_{i-1}\cdot(C_3^i\omega_{i-1}\log(k+l)+2t_i+2\bar{b}_i)+C_4^i\cdot N_i\cdot\omega_i\log(k+l)$$

$$+2^{t_i+1}\cdot t_i.$$

where $T^i_{table} = \min\{C_5^i N_i, 2^{t_i}\}\cdot t_i$, $\bar{N}_i \approx C_6^i N_{i-1}^2$, $\delta_i = 1 - N_i/(2\bar{N}_i q_i)$ and $q_i = \binom{\omega_{i-1}}{d_i}\binom{k+l-\omega_{i-1}}{\omega_{i-1}-d_i}/\binom{k+l}{\omega_{i-1}}\cdot 1/2^{t_i}\cdot q(\mathbf{x}_{i-1},\mathbf{y}_{i-1},\mathcal{C}_f^{(i)},\alpha_i)$. In particular, the values $(C_1^i,\bar{b}_i,C_2^i,C_3^i,C_4^i,C_5^i,C_6^i)$ are set to be $(1,0,2,2,1,1,1/2)$ for $i=1$, $(1,1,3/4,2,1,1/2,1/4)$ for $2\leq i\leq l'-1$ and $(1/2,1,1/4,3/2,1/2,1/2,1/4)$ for $i=l'$.

F Discussion of parameter constraint (3.9)

The constraint (3.9) essentially requires that the size of each list in sieving-ISD should be less than the number of expected solutions. This constraint (3.9) is

overlooked in [19]'s analysis.[27] A similar constraint is also required by conventional ISDs, such as the Both-May algorithm [6]. In particular, this constraint is also neglected in [6]'s original analysis, and afterwards pointed out by [7].

In order to show that such a constraint is indeed necessary, we perform several experiments. We conducted the experiments with the simple quadratic sieve to verify the constraint (3.9). According to [19], the list size N is set as follows,

$$N = \frac{2 \cdot 1.5\binom{\hat{\omega}}{d}\binom{k+l-\hat{\omega}}{\hat{\omega}-d}}{\binom{k+l}{\hat{\omega}}}$$

For each specific set of parameters k, l and $\hat{\omega}$, we do the experiment 1000 times, where the size and weight are fixed in each layer. Then, the required number $(N/2)$ of solutions, the expected number of solutions, and the actual average number of solutions found in the experiment are respectively shown in Table F.1.

Table F.1: Experiments for the constraint (3.9)

	k	l	$\hat{\omega}$	$N/2$	$E[\lvert solutions \rvert]$	$\frac{N/2}{E[\lvert solutions \rvert]}$	Average found
Exp-1	128	9	2	52	36	1.44	12
Exp-2	128	10	2	52	18	2.89	4
Exp-3	128	11	2	53	9	5.89	1
Exp-4	164	9	2	65	58	1.12	21
Exp-5	164	10	2	66	29	2.28	7
Exp-6	164	11	2	66	15	4.40	2

G The progressive DOOM-sieving and its complexity

The complete progressive DOOM-Sieving is presented by Algorithm 4, where the embedded ANNS-Predicate-DOOM algorithms are given by Algorithm 5.

Before analyzing the complexity of progressive DOOM-sieving, we first give the complexity results of Algorithm 5 by Theorems G.1 similar to Section 3.2. That is, we choose to mainly analyze Algorithm 5 for the ANNS-Predicate-2-DOOM case, and directly give analysis for ANNS-Predicate-1/3-DOOM accordingly.

Theorem G.1 (Complexity of Algorithm 5). *Given matrix* $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_{t'}, \hat{\mathbf{H}}_t] \in \mathbb{F}_2^{(t'+t)\times(k+l)}$, *vector* $\hat{\mathbf{s}}_{1\to k} = (\hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2, ..., \hat{\mathbf{s}}_k)$ *where* $\hat{\mathbf{s}}_j = [(\hat{\mathbf{s}}_j)_{t'}, (\hat{\mathbf{s}}_j)_t] \in \mathbb{F}_2^{t'+t}$ *for*

[27] If we do not consider the constraint (3.9), the security analysis results of attacking NIST-KEMs using progressive sieving-ISDs will be thrilling but wrong (31-36 bit-improvements for Classic McEliece, and 16-20 bit-improvements for HQC/BIKE over the state-of-the-art results).

Algorithm 4: Progressive DOOM-Sieving

Input : List $L_0^0 \subseteq \mathcal{S}_{\omega_0+1}^{k+l}$ with size $N_0/2$, List $L_0^1 \subseteq \mathcal{S}_{\omega_0}^{k+l}$ with size $N_0/2$, matrix $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_1, \hat{\mathbf{H}}_2, ..., \hat{\mathbf{H}}_{l'}] \in \mathbb{F}_2^{l\times(k+l)}$ and vector $\widehat{\mathbf{s}}_j = (\hat{\mathbf{s}}_1^j, \hat{\mathbf{s}}_2^j, ..., \hat{\mathbf{s}}_{l'}^j) \in \mathbb{F}_2^l$ where $1 \le j \le k$, $\hat{\mathbf{H}}_i \in \mathbb{F}_2^{t_i\times(k+l)}$, $\hat{\mathbf{s}}_i^j \in \mathbb{F}_2^{t_i}$ for any $1 \le i \le l'$, and $\sum_{i=1}^{l'} t_i = l$

Output: Lists $L_{l'}^{1\to k} = \{L_{l'}^1, L_{l'}^2, ..., L_{l'}^k\}$ where $L_{l'}^j \subseteq \mathcal{S}_{\hat{\omega}}^{k+l}$ with size $|L_{l'}^1| = \cdots = |L_{l'}^k| = N_{l'}/2k = N/k$ such that $\hat{\mathbf{H}}\mathbf{z} = \widehat{\mathbf{s}}_j$ for any $\mathbf{z} \in L_{l'}^j$ where $1 \le j \le k$

$L_1^0, L_1^{1\to k} \leftarrow \emptyset$;
$L_1^0, L_1^{1\to k} \leftarrow$ ANNS-Predicate-1-DOOM$(L_0^0, L_0^1, \hat{\mathbf{H}}_1, \hat{\mathbf{s}}_1^{1\to k})$;
for $i = 2$ *to* $l' - 1$ **do**
  $L_i^0, L_i^{1\to k} \leftarrow \emptyset$;
  $L_i^0, L_i^{1\to k} \leftarrow$ ANNS-Predicate-2-DOOM$(L_{i-1}^0, L_{i-1}^{1\to k}, [\hat{\mathbf{H}}_{i-1}, \hat{\mathbf{H}}_i], [\hat{\mathbf{s}}_{i-1}, \hat{\mathbf{s}}_i]^{1\to k})$;
$L_{l'}^{1\to k} \leftarrow$ ANNS-Predicate-3-DOOM$(L_{l'-1}^0, L_{l'-1}^{1\to k}, [\hat{\mathbf{H}}_{l'-1}, \hat{\mathbf{H}}_{l'}], [\hat{\mathbf{s}}_{l'-1}, \hat{\mathbf{s}}_{l'}]^{1\to k})$;
return $L_{l'}^{1\to k}$

$1 \le j \le k$, list $L_0' \subseteq \mathcal{S}_{\omega'+1}^{k+l}$ satisfying $\hat{\mathbf{H}}_{t'}\mathbf{x} = \mathbf{0}$ for any $\mathbf{x} \in L_0'$ and list $L_j' \subseteq \mathcal{S}_{\omega'}^{k+l}$ satisfying $\hat{\mathbf{H}}_{t'}\mathbf{x} = (\hat{\mathbf{s}}_j)_{t'}$ for any $\mathbf{x} \in L_j'$. Then Algorithm 5 has expected time

$$T = T_{table} + \frac{N'}{2}\Big(C_1\Big(T_{ValidFilters}(0) + E[|\mathcal{B}_\alpha(\mathbf{x},0)|]((\omega'+1)\log(k+l) + t + \log(k+1))\Big) + T_{ValidFilters}(1) + E[|\mathcal{B}_\alpha(\mathbf{x},1)|](\omega'\log(k+l) + t + \log(k+1)) + (2\omega'-1)t + 1/2^t \cdot E[|\mathcal{B}_\alpha(\mathbf{x},0)|]\Big((C_2)^{(0)}E[|Bucket_\alpha(\mathbf{c},0)|] + (C_2)^{(1)} E[|Bucket_\alpha(\mathbf{c},1)|]\Big)(\omega'+1)\log(k+l)\Big)$$

and expected memory

$$M = \frac{N'}{2}\Big(((2\omega'+1)\log(k+l) + t) + C_3 E[|\mathcal{B}_\alpha(\mathbf{x},0)|]((\omega'+1)\log(k+l) + t + \log(k+1)) + E[|\mathcal{B}_\alpha(\mathbf{x},1)|](\omega'\log(k+l) + t + \log(k+1))\Big) + \frac{N}{2}(C_4\omega + C_5)\log(k+l) + 2^{t+1}\cdot t\cdot k$$

with probability $p \ge (1-\exp(-(C_6)^{(0)}N'^2 q_0 \delta_0^2/2))(1-\exp(-(C_6)^{(1)}N'^2 q_1\delta_1^2/2))^k$, $d = (2\omega'+1-\omega)/2$ and $T_{table} = \min\{N'/2, 2^t\}\cdot t\cdot k$, where $q_0 = \binom{\omega'+1}{d}\binom{k+l-\omega'-1}{\omega'+1-d}/\binom{k+l}{\omega'+1}\cdot q(\mathbf{x},\mathbf{y},\mathcal{C}_f,\alpha)/2^t$, $q_1 = \binom{\omega'}{d}\binom{k+l-\omega'}{\omega'+1-d}/\binom{k+l}{\omega'+1}\cdot q(\mathbf{x},\mathbf{y},\mathcal{C}_f,\alpha)/2^t$. In particular, the values of $(C_1, (C_2)^{(0)}, (C_2)^{(1)}, \delta_0, C_3, \delta_1, C_4, C_5, (C_6)^{(0)}, (C_6)^{(1)})$ are set to

Algorithm 5: ANNS-Predicate-DOOM

Input : matrix $\hat{\mathbf{H}} = [\hat{\mathbf{H}}_{t'}, \hat{\mathbf{H}}_t] \in \mathbb{F}_2^{(t'+t)\times(k+l)}$, vector $\hat{\mathbf{s}}_{1\to k} = (\hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2, ..., \hat{\mathbf{s}}_k)$ where $\hat{\mathbf{s}}_j = [(\hat{\mathbf{s}}_j)_{t'}, (\hat{\mathbf{s}}_j)_t] \in \mathbb{F}_2^{t'+t}$ for $1 \le j \le k$, list $L'_0 \subseteq \mathcal{S}_{\omega'+1}^{k+l}$ satisfying $\hat{\mathbf{H}}_{t'}\mathbf{x} = \mathbf{0}$ for any $\mathbf{x} \in L'_0$, $L'_{1\to k} = (L'_1, L'_2, ..., L'_k)$ where list $L'_j \subseteq \mathcal{S}_{\omega'}^{k+l}$ satisfying $\hat{\mathbf{H}}_{t'}\mathbf{x} = (\hat{\mathbf{s}}_j)_{t'}$ for any $\mathbf{x} \in L'_j$, the filter set $\mathcal{C}_f$ and the bucket parameter α

Output: list $L_0 \subseteq \mathcal{S}_{\omega+1}^{k+l}$ such that $\hat{\mathbf{H}}\mathbf{z} = \mathbf{0}$ for any $\mathbf{z} \in L_0$ and $L_{1\to k} = (L_1, L_2, ..., L_k)$ where list $L_j \subseteq \mathcal{S}_{\omega}^{k+l}$ such that $\hat{\mathbf{H}}\mathbf{z} = \hat{\mathbf{s}}_j$ for $1 \le j \le k$ and $\mathbf{z} \in L_j$

// ANNS-Predicate-1-DOOM: $t' = 0$, $\hat{\mathbf{H}}_{t'} = \mathbf{0}$, $|L'_0| = |L'_1| = N'/2$, $L'_i = \emptyset$ $(2 \le i \le k)$

// ANNS-Predicate-2-DOOM: $t' \neq 0$, $|L'_0| = N'/2$, $|L'_i| = N'/(2k)$, $|L_0| = N/2$, $|L_i| = N/(2k)$ $(1 \le i \le k)$

// ANNS-Predicate-3-DOOM: $t' \neq 0$, $L_0 = \emptyset$

1 Bucketing Phase
2 **for** $\mathbf{x} \in L'_0$ **do**
3 **for** $\mathbf{c} \in$ ValidFilters$(\mathcal{C}_f, \mathbf{x}, \alpha)$ **do**
4 add $(0, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$ to Bucket$_\alpha(\mathbf{c})$;
5 **for** $i \in \{1, ..., k\}$ **do**
6 **for** $\mathbf{x} \in L'_i$ **do**
7 **for** $\mathbf{c} \in$ ValidFilters$(\mathcal{C}_f, \mathbf{x}, \alpha)$ **do**
8 add $(i, \mathbf{x}, \hat{\mathbf{H}}_t\mathbf{x})$ to Bucket$_\alpha(\mathbf{c})$;
9 Checking Phase
10 $L_0, L_1, ..., L_k \leftarrow \emptyset$;
11 **for** $\mathbf{x} \in L'_0$ **do**
12 **for** $\mathbf{c} \in$ ValidFilters$(\mathcal{C}_f, \mathbf{x}, \alpha)$ **do**
13 **for** $(0, \mathbf{y}, \hat{\mathbf{H}}_t\mathbf{x}) \in Bucket_\alpha(\mathbf{c})$ **do**
14 **if** $|\mathbf{x} + \mathbf{y}| = \omega + 1$ **then**
15 store $\mathbf{x} + \mathbf{y}$ in L_0
16 Discard some elements if $|L_0| > N/2$;
17 **for** $1 \le i \le k$ **do**
18 **for** $(j, \mathbf{y}, (\hat{\mathbf{s}}_i)_t + \hat{\mathbf{H}}_t\mathbf{x}) \in Bucket_\alpha(\mathbf{c})$// Flag j is 1 for ANNS-Predicate-1-DOOM, and i for ANNS-Predicate-2/3-DOOM
19 **do**
20 **if** $|\mathbf{x} + \mathbf{y}| = \omega$ **then**
21 store $\mathbf{x} + \mathbf{y}$ in L_i
22 Discard some elements if $|L_i| > N/2k$;

// Lines 2-4, 13-16 can be removed for ANNS-Predicate-3-DOOM

23 **return** $L_0, L_{1\to k}$

$(1,1,k,\ 1-N/(2(C_6)^{(0)}N'^2q_0),1,1-N/(2k(C_6)^{(1)}N'^2q_1),2,1,1/8,1/4)$ *for* $i=1$, $(1,1,1,\ 1-N/(2(C_6)^{(0)}N'^2q_0),\ 1,1-N/(2k(C_6)^{(1)}N'^2q_1),2,1,1/8,1/(4k))$ *for* $2\le i\le l'-1$ *and* $(0,0,1,1-N/(2(C_6)^{(0)}N'^2q_0),\ 0,1-N/(2k(C_6)^{(1)}N'^2q_1),1,0,1/8,1/(4k))$ *for* $i=l'$.

Proof. Let $\bar{N}_0$ be the number of pairs of vectors $\mathbf{x},\mathbf{y}\in L'_0$. Obviously, $\bar{N}_0=\binom{N'/2}{2}\approx\frac{1}{8}\cdot N'^2$. Let $\delta_0=1-N/(2\bar{N}_0q_0)$ and $\bar{N}_i$ be the number of pairs of vectors $\mathbf{x}\in L'_0$ and $\mathbf{y}\in L'_i$ for $1\le i\le k$. Then, we have $\bar{N}_i=N'/2\cdot N'/(2k)$. Let $\delta_i=1-N/(2k\bar{N}_iq_1)=1-N/(2k\bar{N}_1q_1)$. Let $N_i^{expected}$ be $N/2$ for $i=0$ and $N/(2k)$ for $1\le i\le k$. According to Lemma 2.1, for $0\le i\le k$ we have

$$\Pr[|L_i|\ge N_i^{expected}]\ge 1-\exp\left(-\bar{N}_iq_j\delta_i^2/2\right)$$

where $j=0$ for $i=0$ and $j=1$ for $i\ge 1$. Thus, the probability that all the lists $\{L_i\}_{0\le i\le k}$ satisfy the required size is

$$\begin{aligned}p=\prod_{i=0}^{k}\Pr[|L_i|\ge N_i^{expected}]&\ge\prod_{i=0}^{k}(1-\exp\left(-\bar{N}_iq_j\delta_i^2/2\right))\\&=(1-\exp\left(-\bar{N}_0q_0\delta_0^2/2\right))(1-\exp\left(-\bar{N}_1q_1\delta_1^2/2\right))^k.\end{aligned}$$

The expected time of the bucketing phase is

$$\begin{aligned}T_{\text{Bucket}}&=\frac{N'}{2}\Big(T_{\text{ValidFilters}}(0)+E[|\mathcal{B}_\alpha(\mathbf{x},0)|]((\omega'+1)\log(k+l)+t\\&+\log(k+1))\Big)+\frac{N'}{2}\omega' t+k\frac{N'}{2k}\Big(T_{\text{ValidFilters}}(1)+E[|\mathcal{B}_\alpha(\mathbf{x},1)|](\omega'\log(k+l)\\&+t+\log(k+1))\Big)+\frac{N'}{2}(\omega'-1)t\\&=\frac{N'}{2}\Big(T_{\text{ValidFilters}}(0)+E[|\mathcal{B}_\alpha(\mathbf{x},0)|]((\omega'+1)\log(k+l)+t+\log(k+1))\\&+T_{\text{ValidFilters}}(1)+E[|\mathcal{B}_\alpha(\mathbf{x},1)|](\omega'\log(k+l)+t+\log(k+1))+(2\omega'-1)t\Big).\end{aligned}$$

Next, we show how the triple $(j,\mathbf{y},(\hat{\mathbf{s}}_i)_t+\hat{\mathbf{H}}_t\mathbf{x})$ (Line 18) can be identified. Note that in the bucketing phase, table-I with elements $(\mathbf{x},\hat{\mathbf{H}}_t\mathbf{x})$ and index $\mathbf{x}$ is saved for each $\mathbf{x}\in L_0^{'}$. Then, in the checking phase, one can first construct table-II$_i$ $(1\le i\le k)$ with index $\mathbf{a}$ and elements $(\mathbf{a},(\hat{\mathbf{s}}_i)_t+\mathbf{a})$, where $\mathbf{a}\in\{0,1\}^t$. Then, the pair $(j,\mathbf{y},(\hat{\mathbf{s}}_i)_t+\hat{\mathbf{H}}_t\mathbf{x})$ can be easily identified using table-I and table-II$_i$ without overhead. In detail, one given $\mathbf{x}$ $(\mathbf{x}\in L_0^{'})$ can first find $\hat{\mathbf{H}}_t\mathbf{x}$ with table-I, then find $(\hat{\mathbf{s}}_i)_t+\hat{\mathbf{H}}_t\mathbf{x}$ via table-II$_i$ with index $\hat{\mathbf{H}}_t\mathbf{x}$, finally identify candidates of $\mathbf{y}$. The triple $(0,\mathbf{y},\hat{\mathbf{H}}_t\mathbf{x})$ (Line 13) can be identified in the same way. The worst-case time to construct k table-II is $2^t\cdot t\cdot k$. In particular, if $N'/2<2^t$, we choose to construct each table-II$_i$ directly with index $\hat{\mathbf{H}}_{[t]}\mathbf{x}$ where $\mathbf{x}\in L_0^{'}$. Thus, the time T_{table} is $\min\left\{\frac{N'}{2},2^t\right\}\cdot t\cdot k$.

In the Checking phase, given $\mathbf{x} \in L'_0$, we first check $(0, \mathbf{y}, \hat{\mathbf{H}}_t\mathbf{x})$ (Line 13). For each pair $(\mathbf{x}, \mathbf{y})$, we need to compute $\mathbf{x}+\mathbf{y}$ to check whether $|\mathbf{x}+\mathbf{y}| = \omega+1$ is hold. Then, we also need to check $(j, \mathbf{y}, (\hat{\mathbf{s}}_i)_t + \hat{\mathbf{H}}_t\mathbf{x})$ (Line 18) for $1 \le i \le k$ and for each pair $(\mathbf{x}, \mathbf{y})$, we check whether $|\mathbf{x}+\mathbf{y}| = \omega$ is hold. Thus, we have

$$\begin{aligned} T_{\text{Check}} =& T_{table} + \frac{N'}{2} E[|\mathcal{B}_\alpha(\mathbf{x}, 0)|](\frac{E[|\text{Bucket}_\alpha(\mathbf{c}, 0)|]}{2^t} + k\frac{E[|\text{Bucket}_\alpha(\mathbf{c}, 1)|]}{2^t}\frac{1}{k}) \\ & (\omega'+1)\log(k+l) \\ =& \frac{N'}{2}\frac{E[|\mathcal{B}_\alpha(\mathbf{x}, 0)|] \cdot \Big(E[|\text{Bucket}_\alpha(\mathbf{c}, 0)|] + E[|\text{Bucket}_\alpha(\mathbf{c}, 1)|]\Big)}{2^t} \\ & \cdot (\omega'+1)\log(k+l) + T_{table}. \end{aligned}$$

Then the total time of Algorithm 5 is $T = T_{\text{Bucket}} + T_{\text{Check}} = T_{table} +$

$$\begin{aligned} & \frac{N'}{2}\Bigg(T_{\text{ValidFilters}}(0) + E[|\mathcal{B}_\alpha(\mathbf{x}, 0)|]((\omega'+1)\log(k+l) + t + \log(k+1)) \\ & + T_{\text{ValidFilters}}(1) + E[|\mathcal{B}_\alpha(\mathbf{x}, 1)|](\omega'\log(k+l) + t + \log(k+1)) + (2\omega'-1)t \\ & + \frac{E[|\mathcal{B}_\alpha(\mathbf{x}, 0)|]\Big(E[|\text{Bucket}_\alpha(\mathbf{c}, 0)|] + E[|\text{Bucket}_\alpha(\mathbf{c}, 1)|]\Big) \cdot (\omega'+1)\log(k+l)}{2^t}\Bigg). \end{aligned}$$

Except for the lists L'_0 and $L'_{1\to k}$, we also need to store all the buckets and tables. Thus, $M =$

$$\begin{aligned} & \frac{N'}{2}\Big((\omega'+1)\log(k+l) + \omega'\log(k+l) + t + E[|\mathcal{B}_\alpha(\mathbf{x}, 0)|]((\omega'+1)\log(k+l) + t \\ & + \log(k+1)) + E[|\mathcal{B}_\alpha(\mathbf{x}, 1)|](\omega'\log(k+l) + t + \log(k+1))\Big) \\ & + \frac{N}{2}\Big((\omega+1)\log(k+l) + \omega\log(k+l)\Big) + 2^{t+1}tk. \end{aligned}$$

□

H Parameter sets for Classic McEliece, BIKE and HQC

I Appendix: optimal parameters of PRO-GJN

On the optimal parameter set. The framework of our PRO-GJN with a depth of 2 that we adopted is shown in Fig I.1[28] and the optimal parameter sets of our PRO-GJN are given by Table I.1.

We also provide P_1, P_2, p_i, T_{check} and T_{gauss} in the optimal parameter setting of PRO-GJN. We show values P_1, T_{check} and T_{gauss} in Table I.2. In our Python

[28] If $\hat{\omega}_1$ is odd, then $\hat{\omega}_1 = \hat{\omega}_2/2 + 1$.

Table H.1: Parameter sets for Classic McEliece, BIKE and HQC

Scheme	Category	n	k	ω
Classic McEliece	1	3488	2720	64
	3	4608	3360	96
	5	6688	5024	128
	5	6960	5413	119
	5	8192	6528	128
BIKE(key)	1	24646	12323	142
	3	49318	24659	206
	5	81946	40973	274
BIKE(message)	1	24646	12323	134
	3	49318	24659	199
	5	81946	40973	264
HQC	1	35338	17669	132
	3	71702	35851	200
	5	115274	57637	262

scripts, we set mpmath.mp.dps to 300 and under the optimal parameters of PRO-GJN for NIST-KEMs, $p_i \approx 1$. That is, $(1 - p_i) < 10^{-300}$.

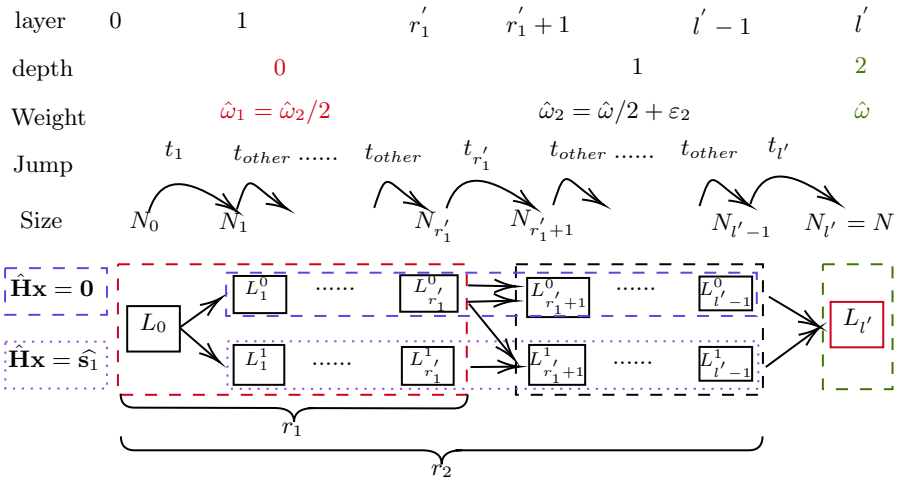

Fig. I.1: The framework of PRO-GJN with a depth of 2

The variation of N_i. Here, in Fig. I.2 and Fig. I.3, we provide the variation of N_i and its relationship with Equations (3.8) and (3.9) which in the DOOM

Table I.1: Optimal parameters of PRO-GJN and NO-PRO-GJN (shown in parentheses)

Scheme	Category	$\hat{\omega}$	l	$\hat{\omega}_2$	$\hat{\omega}_1$	t_1	$t_{r_1'}$	$t_{l'}$	t_{other}	r_1	r_2
Classic McEliece	1	18(8)	93(46)	14	8	5	35	28	1	23	65
	3	20(16)	105(91)	16	8	5	35	27	1	13	78
	5	28(20)	145(117)	22	12	7	35	37	1	31	108
	5	30(18)	156(107)	26	14	9	35	18	1	35	138
	5	30(18)	160(110)	26	14	11	35	19	1	36	141
BIKE(key)	1	6(6)	50(41)	4	2	1	14	34	1	2	16
	3	6(6)	54(44)	4	2	1	15	37	1	2	17
	5	6(6)	57(46)	4	2	1	16	39	1	2	18
BIKE(message)	1	5(5)	49(41)	5	3	13	21	12	1	16	37
	3	5(7)	53(44)	5	3	13	23	13	1	17	40
	5	5(5)	57(46)	5	3	12	24	15	1	18	42
HQC	1	5(5)	51(42)	5	3	13	21	13	1	17	38
	3	5(3)	56(30)	5	3	13	23	15	1	18	41
	5	5(5)	59(47)	5	3	11	24	16	1	19	43

Table I.2: P_1, T_{check} and T_{gauss} in the optimal parameter setting of PRO-GJN

Scheme	category	P_1	P_2	T_{check}	T_{gauss}
Classic McEliece	1	64	0.51	74	31
	3	96	0.51	84	32
	5	134	0.51	118	33
	5	126	0.51	125	33
	5	155	0.51	129	34
BIKE(key)	1	110	0.51	38	40
	3	171	0.51	41	42
	5	236	0.51	43	45
BIKE(message)	1	107	0.51	41	42
	3	169	0.51	44	44
	5	232	0.51	45	47
HQC	1	105	0.51	43	43
	3	170	0.51	45	46
	5	230	0.51	47	48

setting correspond to Equations (4.7)-(4.10) under the optimal parameters of PRO-GJN for BIKE/HQC/Classic McEliece.

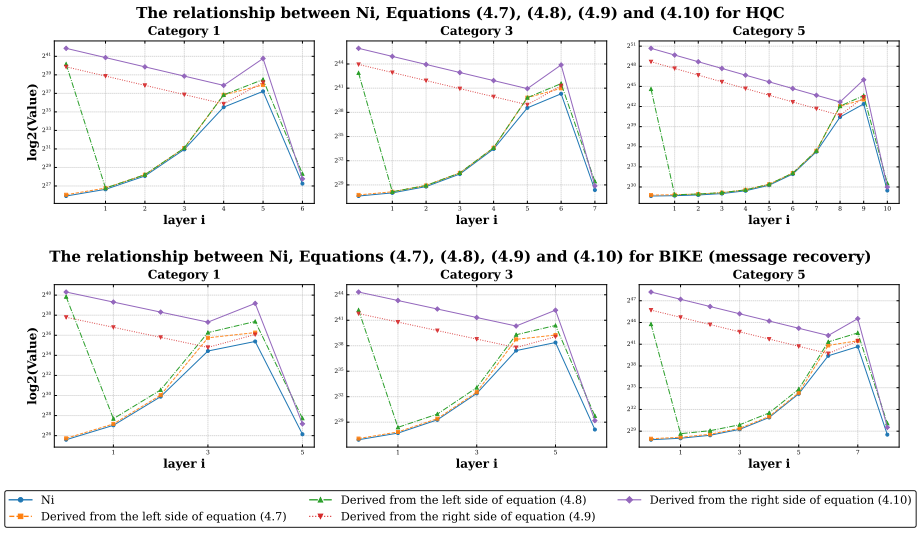

Fig. I.2: The relationship between the variable $\log N_i$, Equations (4.7), (4.8), (4.9) and (4.10) for HQC and BIKE (message recovery)

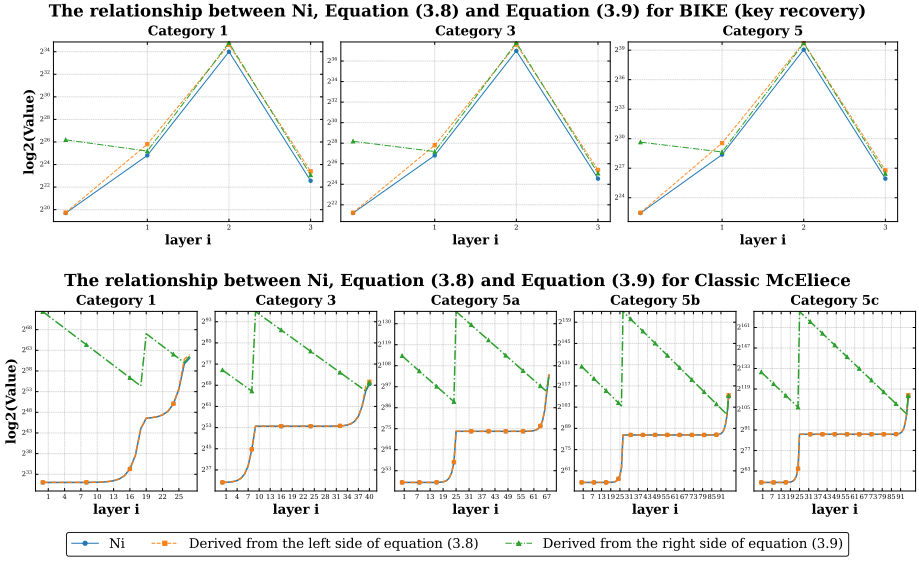

Fig. I.3: The relationship between the variable $\log N_i$, Equation (3.8) and Equation (3.9) for Classic McEliece and BIKE (key recovery)

I.1 Supplementary performance comparison for the ANNS-Predicate problem

Figs. I.4 ($t = 1$) and I.5 ($t > 1$) show the asymptotic results with variable $\varepsilon/(k+l)$ that lies in $(0, \frac{\hat{\omega}}{2(k+l)}]$. The values t, $\hat{\omega}/(k+l)$ are chosen based on the optimal parameter of PRO-GJN.

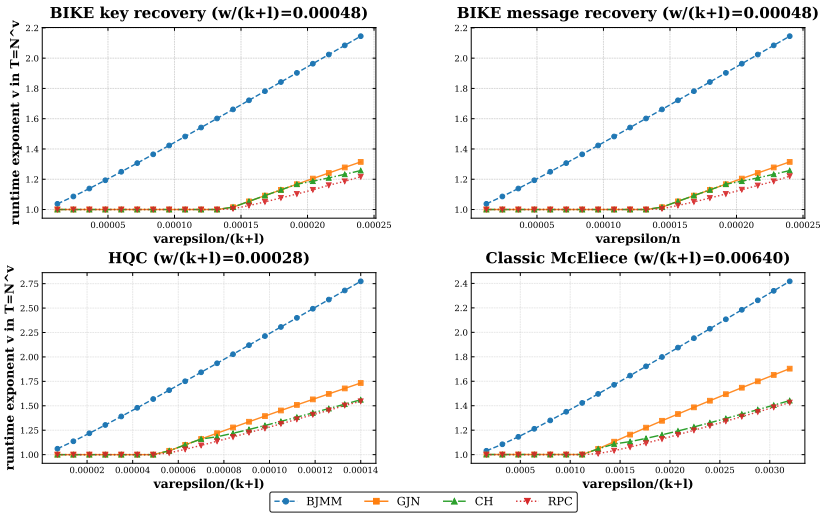

Fig. I.4: Asymptotic time complexity of different ANNS-Predicate algorithms with variable ε ($t = 1$)

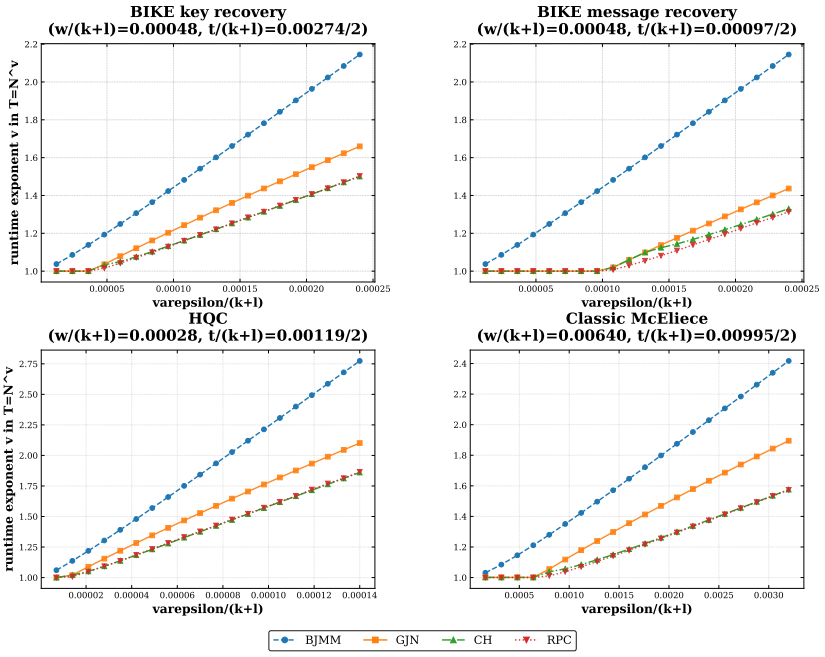

Fig. I.5: Asymptotic time complexity of different ANNS-Predicate algorithms with variable ε ($t > 1$)